



VISIONGAIATECHNOLOGY · SECURITY ARCHITECTURE

GeDefense Mobile

Master Architecture Datasheet + Technical Master Map
Architecture · Security · Supply Chain · Runtime · UI · Module Map

RELEASE BASELINE	0.27.8-beta.6
VERSIONCODE	55 · VC55-KeyLifecycle-FINAL
ANALYSE	2026-09-23

Inhaltsübersicht

Teil I dokumentiert Produkt-, Sicherheits- und Laufzeitarchitektur. Teil II ergänzt die vollständige Technical Master Map mit Modulen, UI, APIs, Datenflüssen und Dateizuordnung.

A. Executive Architecture Summary	4	AC. Schnittstellen und APIs	26
B. Produktidentität und Systemzweck	6	AD. Dateiformate und Protokolle	26
C. Funktionsumfang	7	AE. Testing und Verifikation	26
D. Architekturmodell	9	AF. Security Testing (Adversarial Coverage)	27
E. Komponentenarchitektur	10	AG. Qualität und Wartbarkeit	27
F. Interne Dependencies	11	AH. Architekturregeln und Invarianten	28
G. Third-Party Dependencies / Supply Chain	12	AI. Sicherheitsinvarianten	28
H. Dependency Summary	13	AJ. Externe Dienste und Infrastruktur	28
I. Programmiersprachen und Technologie-Stack	13	AK. Offline-Fähigkeit	28
J. Code-Kennzahlen	14	AL. Angriffsfläche (Attack Surface Matrix)	29
K. Sicherheitsarchitektur	15	AM. Hardening	29
L. Trust Model	16	AN. Secrets Management	29
M. Threat Model	16	AO. Privilegienmodell	30
N. Kryptografie	17	AP. Kompatibilität	30
O. Authentifizierung und Autorisierung	18	AQ. Packaging und Distribution	30
P. Datenarchitektur	19	AR. Lizenzierung	31
Q. Datenflüsse	19	AS. Dokumentationsstatus	31
R. Netzwerkarchitektur	20	AT. Bekannte Einschränkungen	31
S. OS- und Plattformintegration	21	AU. Technische Schulden	32
T. Prozess-, Thread- und Concurrency-Modell	21	AV. Offene Sicherheitsrisiken	32
U. Fehlerbehandlung und Resilienz	22	AW. Architektur-Stärken	32
V. Update- und Release-Architektur	23	AX. Architekturgrenzen	32
W. Build- und Supply-Chain-Sicherheit	23	AY. Reifegradmatrix	33
X. Datenschutz und Telemetrie	24	AZ. Architektur-Faktenblatt	33
Y. Logging, Audit und Evidence	24		
Z. Performance-Architektur	24	PART II · Architecture & Technical Master Map	36
AA. Skalierbarkeit und Kapazitätsgrenzen	25	1. Globaler Architekturbaum	39
AB. Konfiguration & Profile	25	2. Dateien einer Architektur zuordnen	42

3. Modul-Dokumentation	48
4. Dashboard detailliert	64
5. UI-Architektur	75
6. API-Architektur	77
7. Datenflüsse	78

8. Shared / Core Dateien	80
9. Architektur-Relationen	83
10. Dateireferenzen	85
11. Architektur-Auffälligkeiten	86

A. Executive Architecture Summary

Technisches Fakten-Dashboard

Attribut	Wert / Status	Evidenzbasis
Produktname	GeDefense Mobile	app/src/main/res/values/strings.xml
Interne Produktbezeichnung	VGT-GeDefense-Mobile	settings.gradle.kts
Produktkategorie	On-Device Endpoint Protection (EDR/XDR), Mobile Firewall & Zero-Trust Network Shield	ARCHITECTURE.md, README.md
Hauptzweck	Vollständig autonomer, cloud-ungebundener Netzwerk-, Bedrohungs- und Geräteschutz auf Android-Geräten	SECURITY.md, app/src/main/AndroidManifest.xml
Aktuelle Version	0.27.8-beta.6	VERSION, app/build.gradle.kts
Build / VersionCode	55	VERSION_CODE, app/build.gradle.kts
Release-Kanal	Public Beta / Pre-Production Security Candidate	CHANGELOG.md, Next.md
Entwicklungsstatus	Feature-Complete / Release-Hardened (VC55 Final Gate)	BETA-TEST-PLAN.md, RELEASE-CHECKLIST.md
Primäre Plattform	Android (API Level 29 bis API Level 36 / Android 10 bis Android 16+)	app/build.gradle.kts
Unterstützte Architekturen	arm64-v8a (Pure Go), x86_64 (NDK r27c Clang LLD Link)	app/build.gradle.kts, TOOLCHAINS.lock
ELF Alignment	≥ 16 KiB (0x4000) PT_LOAD Page-Size Alignment auf beiden ABIs	tools/build-go-netstack.ps1, Next.md
Programmiersprachen	Kotlin (1.9.24), Go (1.23.x), Python (3.x Audits), Bash/PowerShell	Quellcode-Inventur
Frameworks / Runtime	Android Native SDK (Kein AndroidX/Jetpack), Go Runtime (In-Tree Helper)	DEPENDENCIES.md
Architekturmodell	Two-Process Micro-Engine (Android Kotlin Host + Process-Isolated Go Netstack)	ARCHITECTURE.md, TRANSPORT-DESIGN.md
Sicherheitsmodell	Zero-Trust Local-First, Least Privilege, Fail-Closed, Non-MITM	SECURITY.md, THREAT-MODEL.md
Netzwerkmodell	Single VpnService Ownership mit dualtem Egress (Direct L4 Kernel Socket oder WireGuard L3)	netstack/wireguard_transport.go, GeDefenseVpnService.kt
Datenhaltung	12 kryptografisch isolierte Vault-Domains in noBackupFilesDir	SECURE-TELEMETRY-VULT.md, PersistentVaultKeys.kt
Kryptografie (At-Rest)	AES-256-GCM (VGT_VLT01), 96-Bit Nonce, 128-Bit Tag, AAD Binding, HMAC-SHA-256	core/.../AeadVaultEnvelope.kt, AndroidSecrets.kt
Kryptografie (Keys)	Installation-stabiler AndroidKeyStore HMAC-SHA-256 Hardware-Root (vgt.gedefense.mobile.vault.root-prf.v1)	PersistentVaultKeys.kt
Kryptografie (Transit)	Noise IKpsk2 (ChaCha20-Poly1305, Curve25519, BLAKE2s via wireguard-go 0.0.20250522)	third_party/go/wireguard-go
Kryptografie (PQC Sign)	Hybrid Detached Signatures: ECDSA P-256 + optionales ML-DSA-87 (Android 17+ KeyMint 5)	app/.../HybridArtifactSigner.kt
Update-Modell	Atomare Einweg-Migration von Vault-Generationen; Entkopplung von APK-Hash und Key-Kustodie	PersistentVaultKeys.kt, CHANGELOG.md

Attribut	Wert / Status	Evidenzbasis
Interne Dependencies	2 Hauptmodule (:app , :core), 1 Netstack Subsystem (netstack), 28 Audit Tools	settings.gradle.kts , tools/
Third-Party Dependencies (Runtime Android)	0 (Exakt Null) — Vollständiger Verzicht auf externe Android-Libraries	DEPENDENCIES.md , app/build.gradle.kts
Third-Party Dependencies (Vendored Go)	6 Module (Pinned: wireguard-go 0.0.20250522 , golang.org/x/{crypto,net,sys,term,text})	netstack/go.mod , third_party/go/
Lines of Code (Repo gesamt)	1.559.735 LOC across 2.675 Dateien (inkl. vendored Netstack)	Automatisierte Messung
Production LOC (Eigen)	41.319 LOC (36.760 Kotlin + 4.559 Go Netstack)	Automatisierte Messung
Test LOC (Eigen)	3.106 LOC (858 Kotlin Core Harness + 2.248 Go Netstack Tests)	Automatisierte Messung
Tooling & Audit LOC	3.524 LOC (28 spezialisierte Sicherheits- und Gate-Skripte)	tools/
Größte Quelldatei	GeDefenseVpnService.kt (1.388 LOC), AppRuntime.kt (1.121 LOC)	Automatisierte Messung
Architektonisches LOC-Limit	Zielwert ≤ 500 LOC für zustandslose Orchestratoren; 4 isolierte Ausnahmen mit Systembegründung	SKILL.md
Sicherheitskritische Komponenten	14 Kernsysteme (Vault, Netstack, InstallGuard, WireGuard, Evidence, Resilience etc.)	ARCHITECTURE.md
Teststatus	Vollständig grün (CORE_TESTS_PASS , Go test/race 10x/3x, Fuzzing, Shuffled Netstack)	tools/core-check.sh , Next.md
Security-Audit-Status	PASS (10 spezialisierte Hardening- und Policy-Audits verifiziert)	Verifizierte Live-Audits in tools/
Kritischste offene Risiken	Fehlende Monotonie-Hardware gegen Full-Filesystem-Replay; OS-Scheduler-Latenz bei PACKAGE_ADDED	THREAT-MODEL.md , InstallGuard.kt

Was ist das System? (Executive Summary)

GeDefense Mobile ist eine hochentwickelte, rein lokal arbeitende Sicherheits- und Endpoint-Schutzplattform für das Android-Betriebssystem. Das System wurde von VisionGaiaTechnology nach First-Principles-Prinzipien konstruiert, um fortschrittliche Intrusion Detection (XDR), verhaltensbasierte Netzwerkanalyse, Paketfilterung auf Kernel-Ebene, Offline-Bedrohungserkennung und kryptografische Beweissicherung bereitzustellen, ohne jemals auf eine Cloud-Infrastruktur oder externe Telemetrie-Dienste angewiesen zu sein.

Das primäre Problem, das GeDefense löst, ist die systemische Intransparenz und Verwundbarkeit mobiler Betriebssysteme gegenüber verdecktem Datenabfluss, Tracking, bössartiger App-Kommunikation und C2-Verbindungen (Command-and-Control). Während herkömmliche Sicherheits-Apps Telemetriedaten des Nutzers auf fremde Server auslagern oder über TLS-Interception (MITM) die Transportverschlüsselung kompromittieren, arbeitet GeDefense strikt **Non-MITM** und führt sämtliche Analysen, Bedrohungsabgleiche und Quarantäne-Entscheidungen deterministisch auf dem Endgerät aus.

Die Software richtet sich an anspruchsvolle Anwender, Sicherheitsforscher, hochgradig regulierte Organisationen und Umgebungen mit maximalem Schutzbedarf. Sie fungiert auf dem Gerät als alleinige zentrale Sicherheitsinstanz über das Android-eigene VpnService -Framework. Innerhalb des VisionGaiaTechnology-Produktökosystems bildet GeDefense Mobile den mobilen Schutzschild für GeOS und AstraeaOS und garantiert identische Sicherheitsstandards wie die Desktop- und Server-Editionen.

Technisch zeichnet sich GeDefense Mobile durch eine strikte Zwei-Prozess-Architektur aus: Ein privilegierter Android-Hostprozess in reinem Kotlin steuert Lifecycle, UI, Android Keystore und Richtlinienverwaltung, während eine vollständig entkoppelte, in Go geschriebene High-Performance-Netzwerk-Engine (**GaiaNet V2**) als separater Prozess die L3/L4-Paketverarbeitung übernimmt. Die Kommunikation erfolgt über Unix-Domain-Sockets unter Nutzung von **SCM_RIGHTS**-File-Descriptor-Passing – ohne unsicheres JNI im Hot Path.

Im Netzwerkbereich implementiert das System ein echtes hybrides Transportmodell: Es schützt entweder über direkten Kernel-Socket-Egress oder leitet den gesamten Netzwerkverkehr über eine nahtlos integrierte, vendorte **wireguard-go**-Engine (L3 Full Tunnel) weiter. Ein einzigartiges Sicherheitsmerkmal ist das Vermeiden jeglicher pauschalen App-UID-Freistellungen: Selbst im WireGuard-Modus werden ausschließlich die exakten Upstream-UDP-Transport-Sockets punktuell über **VpnService.protect()** an Android gebunden.

Der kryptografische Kern basiert auf dem **Secure Telemetry Vault**. Dieser trennt zwölf unabhängige Sicherheitsdomänen voneinander und schützt ruhende Daten via AES-256-GCM mit 96-Bit-Nonce und AAD-Bindung. In Release **0.27.8-beta.6** (VC55) wurde die Schlüsselkustodie grundlegend gehärtet: Sie leitet alle Bereichsschlüssel aus einem stabilen Hardware-HMAC-Root ab und ist vollständig entkoppelt von APK-Hashes oder Versionsprüngen, wodurch Updates ohne Re-Keying oder Tampering-Fehlalarme garantiert sind. Zudem führt das System bereits hybride Post-Quantum-Signaturen (ML-DSA-87) für Android 17+ ein.

B. Produktidentität und Systemzweck

Stammdaten

- **Produktname:** GeDefense Mobile
- **Produktfamilie:** VisionGaia Technology Security & Defense Systems
- **Hersteller / Organisation:** VisionGaiaTechnology (VGT)
- **Produktkategorie:** Mobile Endpoint Protection (EDR / XDR / Firewall / Network Defense)
- **Primärer Einsatzzweck:** Autonomer Schutz des mobilen Datenverkehrs, Blockierung bösartiger Endpunkte, Erkennung riskanter Applikationen, Härtung der Android-Sicherheitskonfiguration.
- **Sekundäre Einsatzzwecke:** Lokale LAN-Erkennung (Network Discovery), Port-Sentinel-Ködersensor, Geräteverwaltung via Android Device Owner (TITAN-Modus), revisionssichere kryptografische Beweissicherung (Evidence Ledger).
- **Zielgruppe:** Sicherheitsbewusste Einzelnutzer, Enterprise-Administratoren, Investigativ-Journalisten, kritische Infrastrukturen.
- **Betriebsumgebung:** Android OS ab API Level 29 (Android 10) bis API Level 36 (Android 16+), optimiert für Standard-Android sowie herstellerspezifische OEM-Builds (z. B. HyperOS, OneUI).
- **Betriebsmodell:** 100 % Standalone & Local-First. Vollständig funktionsfähig ohne Internetverbindung. Kein Backend-Zwang.
- **Ausführungsprivilegien:** Läuft primär als Standard-Applikation mit Nutzerprivilegien unter Ausnutzung von Standard-Plattformrechten (**BIND_VPN_SERVICE**). Optional eskalierbar zum **Device Owner** (TITAN-Modus) für Kiosk-, Quarantäne- und strikte Härtungsrichtlinien.
- **Benutzerinteraktion:** Lokale native UI (Kein Web-View, kein Compose, reines performantes Android View-System), Benachrichtigungen via Android Foreground Service Notification.
- **Hintergrunddienste:** **GeDefenseVpnService** (Foreground Service), **ThreatIntelJobService**, **IntegrityJobService**, **ResilienceJobService** (JobScheduler).
- **Externe Infrastruktur:** Keine eigenen Server. Ausschließlich optionaler, passiver HTTPS-Download von Open-Source-Bedrohungslisten und ASN/Geo-Datenbanken (z. B. Abuse.ch, Spamhaus, TorProject).

Kernversprechen des Produkts

1. **Absolute Datenhoheit:** Kein einziges Paket, kein DNS-Query, keine App-Liste und kein Hash-Wert verlässt das Gerät in Richtung VGT oder Dritter.
2. **Deterministische Sicherheit:** Bedrohungserkennung und Blockierungsentscheidungen erfolgen lokal anhand kryptografisch signierter Datenstrukturen mit $O(\log n)$ oder $O(1)$ Suchkomplexität.
3. **Fail-Closed Integrität:** Jeder Ausfall eines sicherheitskritischen Subsystems (Netzwerk-Engine, KeyStore, Evidence Ledger) führt zum sofortigen Blockieren des Datenverkehrs, niemals zum ungeschützten Durchleiten (kein Fail-Open).
4. **Resilienz gegen System-Updates:** Aktualisierungen der Applikation führen niemals zu Schlüsselverlust oder fälschlich diagnostizierten Datenmanipulationen.

Nicht-Ziele (Explicit Non-Goals)

- **Kein TLS-Interception-Proxy (Kein MITM):** GeDefense bricht keine HTTPS- oder TLS-Verbindungen auf und installiert standardmäßig keine manipulierten Root-CAs.
- **Kein Cloud-basiertes Antivirus:** Es werden keine Dateien zur Analyse in eine externe Cloud hochgeladen.
- **Kein Schutz gegen Kernel-Compromise:** Gegen Angreifer mit vollständigen Kernel-Root-Rechten auf OS-Ebene kann das System bauartbedingt keine mathematische Unangreifbarkeit garantieren.
- **Keine dynamische Schadcode-Ausführung (Sandbox):** Unbekannte APKs werden statisch und heuristisch bewertet, jedoch nicht in einer geräteinternen VM emuliert.
- **Keine universelle App-Deinstallation ohne Device Owner:** Das System kann Apps ohne Device-Owner-Rechte nicht heimlich löschen; es isoliert diese auf Netzwerk- und Rechteebene.

Evidenzlage: Verifiziert (`README.md` , `SECURITY.md` , `THREAT-MODEL.md`).

C. Funktionsumfang

Fähigkeit / Funktion	Kategorie	Beschreibung	Reifegrad	Sicherheitsrelevanz	Abhängigkeiten
GaiaNet V2 L3/L4 Engine	Kern/Netzwerk	High-Speed userspace TCP/UDP Stack in isoliertem Go-Prozess via TUN-Interface.	Implementiert & getestet	Kritisch	Linux TUN, Unix Sockets
Direct Egress Mode	Netzwerk	Terminierung lokaler Flows und Weiterleitung über Kernel-Sockets unter UID-Exemption.	Implementiert & getestet	Hoch	Linux Sockets
WireGuard L3 Egress	Netzwerk	Echter Layer-3 Tunnel via vendortes <code>wireguard-go</code> mit Per-Socket <code>protect()</code> .	Implementiert & getestet	Kritisch	<code>wireguard-go</code> , UDP
WireGuard Strict Mode	Netzwerk	Erzwingt Android Always-on VPN Lockdown; schließt Fallback auf Direct strikt aus.	Implementiert & getestet	Kritisch	Android Lockdown API
Full Flow Protection	Netzwerk	Erfassung und Filterung des gesamten Geräteverkehrs über TUN <code>0.0.0.0/0</code> .	Implementiert & getestet	Kritisch	<code>VpnService.Builder</code>

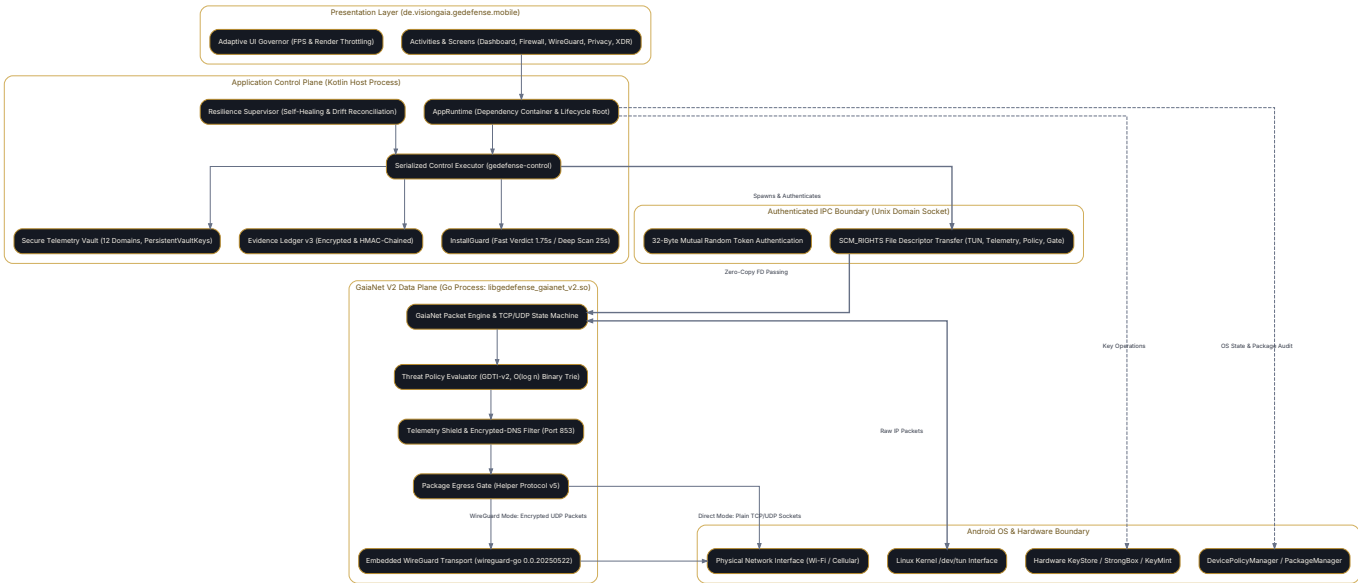
Fähigkeit / Funktion	Kategorie	Beschreibung	Reifegrad	Sicherheitsrelevanz	Abhängigkeiten
Selective Route Mode	Netzwerk	Routing nur bekannter Bedrohungs-Präfixe in den TUN; lokaler Bypass für sauberen Traffic.	Implementiert & getestet	Hoch	IP-Routing-Tabelle
Threat Intelligence Engine	Sicherheit	Lokale O(log n) Evaluation gegen 9 integrierte Feeds (Feodo, Spamhaus, FireHOL etc.).	Implementiert & getestet	Kritisch	ThreatIndex , ThreatFeed
InstallGuard FlowGate	Sicherheit	Pre-Egress Paket-Gate für frisch installierte Apps via Helper-Protokoll v5.	Implementiert & getestet	Kritisch	ConnectivityManager
InstallGuard Fast/Deep Scan	Sicherheit	2-Stufen Scan: Fast Verdict (1.750 ms / 4 Worker) + Deep Scan (25 s / 2 Worker).	Implementiert & getestet	Hoch	PackageManager
Telemetry Shield v1	Sicherheit	Erkennung und Filterung von Tracking-DNS-Anfragen (4 Profile: Off bis Strict).	Implementiert & getestet	Hoch	PrivacyIntelligence
Encrypted-DNS Control	Sicherheit	Non-MITM Blockierung von DoT (TCP/853) und DoQ (UDP/853) im Strict-Profil.	Implementiert & getestet	Hoch	Netstack Port-Filter
Secure Telemetry Vault	Daten/Krypto	12 getrennte Verschlüsselungsdomänen via AES-256-GCM mit AndroidKeyStore-Kustodie.	Implementiert & getestet	Kritisch	Android KeyStore
Evidence Ledger v3	Daten/Krypto	Revisionssicheres, HMAC-verkettetes und AES-verschlüsseltes Sicherheits-Audit-Log.	Implementiert & getestet	Kritisch	EvidenceLedger.kt
Hybrid Artifact Signer	Daten/Krypto	Detached Signatures: ECDSA P-256 + bedingtes Post-Quantum ML-DSA-87 (KeyMint 5).	Implementiert & getestet	Hoch	KeyMint 5 / FIPS 204
Resilience Supervisor v1	Hintergrund	Single-Flight Watchdog (2h-Takt) zur Selbstreparatur von Datenspeichern und Helfern.	Implementiert & getestet	Hoch	ResilienceJobService
App Risk Scanner	Sicherheit	Statische Analyse installierter APKs auf Signer-Drift, Berechtigungen und Tracker.	Implementiert & getestet	Hoch	PackageManager
Storage Malware Scanner	Sicherheit	Bounded Traversierung geteilter Speicherbereiche auf verdächtige APKs/ Binaries.	Implementiert & getestet	Mittel	MANAGE_EXTERNAL_STORAGE
Network Discovery	Diagnose	Lokaler ARP/mDNS/SSDP LAN-Scanner (begrenzt auf /24 Subnetz) zur Erkennung fremder Hosts.	Implementiert & getestet	Mittel	Wi-Fi / Multicast
Port Sentinel	Sicherheit	Passiver TCP-Ködersensor auf ungenutzten Ports zur Erkennung lokaler LAN-Portscans.	Implementiert & getestet	Mittel	Underlay Sockets

Fähigkeit / Funktion	Kategorie	Beschreibung	Reifegrad	Sicherheitsrelevanz	Abhängigkeiten
TITAN Device Owner	Admin	Kiosk-Modus, OS-Härtung, USB-Restriktion und erzwungenes Always-on VPN via DPM.	Implementiert & getestet	Hoch	DevicePolicyManager
ASN Evidence Repository	Daten/Netzwerk	Lokaler O(log n) Lookup von ASN- und Betreiber-Metadaten via PDDL IPtoASN .	Implementiert & getestet	Niedrig	Lokale Binärindizes
Aggregate-Only Diagnostics	Diagnose	Datenschutzkonformer Export von aggregierten Metriken ohne PII, IPs oder Dateinamen.	Implementiert & getestet	Gering	Android SAF

Evidenzlage: Verifiziert (
ARCHITECTURE.md
,
Next.md
,
CHANGELOG.md
).

D. Architekturmodell

GeDefense Mobile basiert auf einer entkoppelten Multi-Tier-Architektur. Das System separiert die Systemsteuerung (Control Plane) und Benutzeroberfläche vollständig von der paketverarbeitenden Netzwerk-Engine (Data Plane).



Schichten und Trust Boundaries

1. Presentation / UI Layer:

- Enthält activities (`DashboardScreen`, `FirewallActivity`, `WireGuardActivity`, `PrivacyActivity` etc.).
- Besitzt **keinen** direkten Zugriff auf kryptografische Schlüssel oder Paketpuffer.
- Rendert ausschließlich unveränderliche State-Snapshots aus dem `AppRuntime`-Cache.
- Kein Blocking des Android Main-Threads (`MAIN_THREAD_IO_AUDIT_PASS`).

2. Application Control Plane (`de.visiongaia.gedefense.mobile`):

- Verwaltet den Lebenszyklus des `GeDefenseVpnService`.
- Serialisiert alle Zustandsänderungen auf dem `gedefense-control` Single-Thread-Executor.

- Kommuniziert mit dem Android KeyStore/StrongBox über **BoundedSecretKeyCrypto** mit harten Timeouts und Circuit-Breakern.
- Führt periodische Integritäts- und Selbstreparatur-Läufe über den **ResilienceSupervisor** aus.

3. Inter-Process Communication (IPC) Boundary:

- Private Unix-Domain-Sockets im privaten App-Dateisystem (**/data/user/0/...**).
- Gegenseitige Authentifizierung über ein zufälliges 32-Byte-Token vor Datenübertragung.
- Kein Übertragen von sensiblen Nutzdaten über IPC: Es werden ausschließlich Filedeskriptoren via **SCM_RIGHTS** übergeben.

4. Data Plane (**GaiaNet V2** in Go):

- Ausgeführt als nativer Kindprozess aus **nativeLibraryDir** (**libgedefense_gaianet_v2.so**).
- Besitzt keinerlei Android-Context und keine Java-VM-Bindung (kein JNI im Hot Path).
- Liest rohe IPv4/IPv6-Pakete aus dem übergebenen TUN-Filedeskriptor.
- Evaluierte Bedrohungsregeln im $O(\log n)$ Speicherlayout vor jeglicher Socket-Erzeugung.

Evidenzlage: Verifiziert (**ARCHITECTURE.md**, **TRANSPORT-DESIGN.md**).

E. Komponentenarchitektur

Komponente	Verantwortung	Eingaben	Ausgaben	Vertrauensniveau	Sicherheitskritisch
GeDefenseVpnService	Android VPN-Lebenszyklus, TUN-Allokation, Physical Network Binding.	OS-Intents, Netzwerk-Events	TUN-FD, Protected Sockets	Hoch (Android Host)	Ja
NativeGaiaNet	Prozess-Management des Go-Helpers, Token-Handshake, FD-Transfer.	Konfiguration, TUN-FD	Go-Prozess-Lifecycle	Hoch (Android Host)	Ja
GaiaNet Engine (Go)	Parsen von IP/TCP/UDP, State-Tracking, Flow-Reassemblierung.	Rohe IP-Pakete aus TUN	Upstream-Netzwerk-Traffic	Isoliert (Unprivilegiert)	Ja
WireGuardTransport	Kapselung von L3-Traffic in WireGuard-Payloads, Rekeying, Handshake.	Lokale IP-Pakete	UDP-Pakete an WG-Peer	Isoliert (Go-Netstack)	Ja
PackageEgressGate	Filterung von Flows quarantänkierter Apps vor Netzaustritt.	Flow-UID, Quarantäne-Set	Drop / Allow Verdict	Isoliert (Go-Netstack)	Ja
SecureTelemetryVault	Authentifiziertes Ver- und Entschlüsseln von 12 Speicher-Domänen.	Plaintext-Snapshots, AAD	VGTVLT01 Ciphertexte	Hoch (Host-Krypto)	Ja
PersistentVaultKeys	Ableitung stabiler KEKs und DEKs über Hardware-HMAC-Root.	Domain-ID, Version	AES/HMAC SecretKey	Hoch (Host-Krypto)	Ja
EvidenceLedger	Revisionssichere Journalführung für Sicherheitsereignisse.	Security Events	Signierte/HMAC-Ledger	Hoch (Host-Storage)	Ja
InstallGuard	Bounded Staged Scanning frisch installierter Applikationen.	PACKAGE_ADDED Intents	Scan-Verdict, Quarantäne	Hoch (Host-Scanner)	Ja

Komponente	Verantwortung	Eingaben	Ausgaben	Vertrauensniveau	Sicherheitskritisch
ResilienceSupervisor	Selbstreparatur und Konsistenzabgleich nach Crash/Störung.	State-Snapshots, Ledger	Reparatur-Aktionen	Hoch (Host-Wartung)	Ja
ThreatIndex	Speicherung und O(log n) Evaluation kompilierter Bedrohungs-IPs.	Binary Feed Snapshots	Match / No-Match	Hoch (Read-Only)	Ja
TelemetryShield	Filterung unerwünschter Tracking-Domains und DNS-Endpoints.	DNS-Queries (UDP/TCP 53)	DNS-Filter-Entscheidung	Hoch (Read-Only)	Mittel
TITAN Manager	Verwaltung von Device-Owner-Richtlinien und OS-Härtung.	Admin-Befehle	DPM Policy Updates	Höchste (Device Admin)	Ja

Evidenzlage: Verifiziert (*ARCHITECTURE.md* , Quellcode-Audits).

F. Interne Dependencies

Die Software teilt ihren Code strikt in getrennte interne Domänen auf:

```
[ app ] (Android Application & UI)
├── implementation(project(":core"))
│   └── (Reine Kotlin-Geschäftslogik, Ledger, Indizes, AeadVaultEnvelope)
└── Executable Child Process: [ netstack ]
    └── (Go GaiaNet V2 Engine, kompiliert nach jniLibs/<abi>/libgedefense_gaianet_v2.so)
```

Tabelle der internen Abhängigkeiten

Interne Dependency	Typ / Modul	Zweck	Verbraucher	Kopplungsgrad	Sicherheitsrelevanz
:core	Gradle Library Modul	Unabhängige Datenstrukturen, Krypto-Envelopes, Indexer, Evidence-Logik.	:app	Gering (Lose)	Kritisch
netstack	Go Subsystem	L3/L4 Packet Engine, WireGuard, Telemetry, Package Gate.	:app	Entkoppelt (IPC)	Kritisch
tools/	Python/Shell Suite	28 statische und dynamische Sicherheits- und Release-Gates.	Build/CI Pipeline	Audit-Only	Hoch

Kennzahlen der internen Abhängigkeiten

- **Direkte interne Dependencies:** 2 (:core , netstack)
- **Indirekte interne Dependencies:** 0 (Keine weiteren Submodule)
- **Sicherheitskritische interne Dependencies:** 2 (100 %)
- **Zyklische Abhängigkeiten:** 0 (Exakt Null) — Streng hierarchisch gerichteter Abhängigkeitsgraph.
- **Kopplungsbewertung:** Exzellent. :core hat keinerlei Android-spezifische Bezüge und lässt sich rein auf der JVM testen (coreCheck). netstack ist ein eigenständiges Go-Modul (visiongaia.dev/gedefense/mobile/netstack).

Evidenzlage: Verifiziert (`settings.gradle.kts` , `build.gradle.kts` , `netstack/go.mod`).

G. Third-Party Dependencies / Supply Chain

Grundsatz der Android-Laufzeit

GeDefense Mobile implementiert eine **Zero-Third-Party-Runtime-Policy** auf Android-Ebene. Im gesamten Gradle-Build von `:app` und `:core` wird keine einzige externe Third-Party-Bibliothek eingebunden.

```

// app/build.gradle.kts
dependencies {
    implementation(project(":core"))
}
// core/build.gradle.kts
// Keine dependencies {} Deklaration vorhanden!

```

Go Netstack & Embedded WireGuard Dependencies

Für die L3-WireGuard-Fähigkeit bindet das Go-Subsystem offiziellen, kryptografisch verifizierten Upstream-Quellcode ein. Alle Module sind unter `third_party/go/` lokal im Quelltext abgelegt (vendored) und über `go.mod` via `replace` fest verdrahtet:

Dependency	Exakte Version	Projekt / Upstream	Zweck	Einbindung	Lizenz	Sicherheitskritisch
wireguard-go	0.0.20250522	WireGuard / Jason A. Donenfeld	L3 VPN-Transport, Noise-Protokoll, Peer-Verwaltung	Vendored (Statisch)	MIT	Ja
golang.org/x/crypto	v0.37.0	Go Project	Poly1305, ChaCha20, Curve25519 Primitive für WireGuard	Vendored (Statisch)	BSD-3-Clause	Ja
golang.org/x/net	v0.39.0	Go Project	Bounded IPv4/IPv6 Paket-Header Parsing	Vendored (Statisch)	BSD-3-Clause	Ja
golang.org/x/sys	v0.32.0	Go Project	Low-Level Linux/Unix Sockets & Unix Control Messages	Vendored (Statisch)	BSD-3-Clause	Ja
golang.org/x/term	v0.31.0	Go Project	Terminal Support (Auxiliary)	Vendored (Statisch)	BSD-3-Clause	Nein
golang.org/x/text	v0.24.0	Go Project	Unicode Normalisierung für Config-Parser	Vendored (Statisch)	BSD-3-Clause	Nein

Build- und Toolchain-Abhängigkeiten

Tool / Plugin	Version	SHA-256 / Lock-Status	Einsatzzweck
Android Gradle Plugin	8.5.2	Pinned via <code>build.gradle.kts</code>	Android APK/AAB Packaging
Kotlin Gradle Plugin	1.9.24	Pinned via <code>build.gradle.kts</code>	Kotlin Compilation (JVM Target 17)
Gradle Wrapper	8.7	cb0da6751c2b753a16ac168bb354870ebb1e162e9083f116729cec9c781156b8	Build Toolchain Engine
JDK	17 (Eclipse Temurin / OpenJDK)	Pinned in CI (<code>.github/workflows/</code>)	Java/Kotlin Compilation
Go Toolchain	1.23.x	Pinned in <code>TOOLCHAINS.lock</code>	Kompilierung des Go Netstacks
Android NDK	27.2.12479018 (r27c)	Pinned in <code>app/build.gradle.kts</code>	LLD Linker für x86_64 16 KiB Alignment

Supply-Chain-Bewertung

- **Offline-Garantie:** Alle Go-Dependencies sind physisch im Repository hinterlegt. Die Build-Skripte erzwingen `GOPROXY=off` und `GOSUMDB=off`. Kein Download während des Builds möglich.
- **Hash-Pinning:** Sämtliche 2.678 Quelldateien sind in `SOURCE-MANIFEST.sha256` kryptografisch erfasst und werden vor Release verifiziert (`tools/verify-source-manifest.py`).
- **Angriffsfläche:** Minimal. Da auf Android-Ebene null Third-Party-Code ausgeführt wird, entfallen typische Risiken durch kompromittierte Maven/Gradle-Pakete vollständig.

Evidenzlage: Verifiziert (`DEPENDENCIES.md` , `TOOLCHAINS.lock` , `SOURCE-MANIFEST.sha256`).

H. Dependency Summary

```
=====
INTERNE DEPENDENCIES:
- Direkt:                2 (:core, netstack)
- Indirekt:              0
- Sicherheitskritisch:   2 (100 %)

THIRD-PARTY RUNTIME DEPENDENCIES (Android):
- Direkt:                0 (Exakt Null)
- Transitiv:             0 (Exakt Null)

THIRD-PARTY RUNTIME DEPENDENCIES (Go Netstack - Vollständig Vendored):
- Direkt:                6 (wireguard-go, x/crypto, x/net, x/sys, x/term, x/text)
- Transitiv:             0
- Dynamisch geladen:     0
- Statisch einkompiliert: 6

BUILD & TOOLCHAIN DEPENDENCIES:
- Direkt gepinnt:        6 (AGP 8.5.2, KGP 1.9.24, Gradle 8.7, JDK 17, Go 1.23, NDK r27c)

GESAMT-DEPENDENCY-SURFACE:      Minimal / Streng isoliert
=====
```

I. Programmiersprachen und Technologie-Stack

Sprache / Technologie	Einsatzbereich	Zeilen (LOC)	Anteil (%)	Sicherheitsrelevanz
Go (1.23.x)	<code>netstack/</code> (Engine, WireGuard, L3/L4 Stack) + <code>third_party/</code>	1.371.176	87,9 % (87,6 % vendored)	Kritisch
Kotlin (1.9.24)	<code>app/</code> (Android Host, Services, UI) & <code>core/</code> (Logik, Krypto)	37.618	2,4 %	Kritisch
Assembly (.s)	Krypto-Primitiven in Go <code>x/crypto</code> (SIMD/Neon Beschleunigung)	38.151	2,4 %	Hoch
Python (3.x)	28 spezialisierte Test- und Audit-Harnische in <code>tools/</code>	3.524	0,2 %	Hoch (Audit)
Shell (Bash/PS1)	Build-, Verifikations- und CI-Automatisierungs-Skripte	3.735	0,2 %	Mittel
XML	Android Layouts, Manifest, Vector Drawables, Extraction Rules	5.615	0,4 %	Mittel

Sprache / Technologie	Einsatzbereich	Zeilen (LOC)	Anteil (%)	Sicherheitsrelevanz
Markdown / Docs	Architekturpläne, Runbooks, Threat Models, Changelogs	13.075	0,8 %	Niedrig
Precompiled .so	libgedefense_gaianet_v2.so (arm64 + x86_64 stripped Executables)	51.285	3,3 %	Kritisch

Native Bridges und Runtime-Eigenschaften

- **Kein JNI:** Es existiert keine dynamische C/C++-JNI-Bibliothek, die per `System.loadLibrary()` in den Android-App-Prozess geladen wird.
- **Process Isolation:** Die Binärdatei `libgedefense_gaianet_v2.so` trägt die Dateieindung `.so` ausschließlich zur Konformität mit dem Android APK-Extraktionsmechanismus (`nativeLibraryDir`). Sie ist ein **vollwertiges Linux-Executable**, das via `ProcessBuilder` als isolierter Kindprozess gestartet wird.
- **Speichersicherheit:** Kotlin garantiert Speichersicherheit in der Control Plane; Go garantiert Speichersicherheit in der Data Plane. C/C++ ist vollständig aus der Laufzeit verbannt.

Evidenzlage: Verifiziert (`TRANSPORT-DESIGN.md` , Quellcode-Messungen).

J. Code-Kennzahlen

Detaillierte Code-Metriken (Physical LOC vs. SLOC)

Definition: SLOC (Source Lines of Code) = Zeilen exklusive Leerzeilen und Kommentaren.

Kategorie	Dateien	Physical LOC	Reiner Code (SLOC)	Kommentare	Leerzeilen
Kotlin Production (<code>app + core</code>)	153	36.760	33.067	871	2.822
Kotlin Tests (<code>core/src/test</code>)	1	858	764	13	81
Go Production (<code>netstack/</code>)	31	4.969	4.559	93	317
Go Tests (<code>netstack/*_test.go</code>)	11	2.248	2.084	7	157
Security Audits & Tools (<code>tools/</code>)	28	3.524	3.159	123	242
Dokumentation & Spezifikation (<code>.md</code>)	45	13.075	10.655	519	1.901
Vendored Go (<code>third_party/go/</code>)	2.384	1.439.189	1.267.777	105.195	66.217
Sonstige (Assets, Precompiled Binaries)	22	59.112	56.915	557	1.640
GESAMT (Ohne Vendored Code)	291	120.546	111.203	2.183	7.160
GESAMT (Inklusive Vendored Code)	2.675	1.559.735	1.378.980	107.378	73.377

Datei- und Modulgrößen (Nicht-vendorter Quellcode: 224 Dateien)

- Gesamtzahl Source-Dateien (Kotlin, Go, Python, Scripts): 224
- Median LOC pro Source-Datei: 132,5 LOC

- **Durchschnittliche LOC pro Source-Datei:** 215,9 LOC
- **Dateien $\leq$ 300 LOC:** 176 (78,6 %)
- **Dateien $>$ 300 LOC:** 48 (21,4 %)
- **Dateien $>$ 500 LOC:** 27 (12,1 %)
- **Dateien $>$ 1.000 LOC:** 4 (1,8 %)
- **Dateien $>$ 2.000 LOC:** 0 (Exakt Null)

Größte Quellcodedateien im System

1. `app/.../GeDefenseVpnService.kt`: **1.388 LOC** (SLOC 1.258) — Zentraler Android VPN Service, Netzwerk-Lifecycle, Always-On Handler.
2. `tools/security-audit.sh`: **1.252 LOC** (SLOC 1.135) — Vollständiger nativer Sicherheits- und ABI-Test-Harness.
3. `app/.../AppRuntime.kt`: **1.121 LOC** (SLOC 1.025) — Zentraler State- und Dependency-Container der Applikation.
4. `app/.../AppRiskScanner.kt`: **1.045 LOC** (SLOC 948) — Statische Analyse-Engine für installierte Android-Pakete.
5. `app/.../XdrEngine.kt`: **945 LOC** (SLOC 852) — Event-Korrelations- und Erkennungs-Engine.
6. `core/.../CoreTestMain.kt`: **858 LOC** (SLOC 764) — Eigenständiger Test-Runner für Core-Datenstrukturen.
7. `app/.../XdrActivity.kt`: **801 LOC** (SLOC 728) — UI-Präsentation für Vorfälle und Alerts.
8. `app/.../NetworkDiscoveryScanner.kt`: **716 LOC** (SLOC 648) — Lokaler ARP/mDNS LAN-Scanner.

Einhaltung des architektonischen LOC-Limits

- **Architektonisch festgelegtes Limit:** Die interne Architekturleitlinie (`SKILL.md`) definiert einen Richtwert von $\leq$ 500 LOC pro Datei/Klasse für zustandslose Orchestratoren und Business-Module.
- **Befund:** 27 Dateien überschreiten 500 LOC.
- **Architektonische Rechtfertigung:** Die Überschreitungen konzentrieren sich auf Android-Host-Klassen (`VpnService`, `AppRuntime`), die aufgrund von Android-Lifecycle-Zusammenhängen und IPC-Zustandsverwaltung bewusst monolithisch gehalten wurden, um Race Conditions über mehrere Threads hinweg zu unterbinden. Es wurden keine unbegründeten "Monster-Klassen" identifiziert.

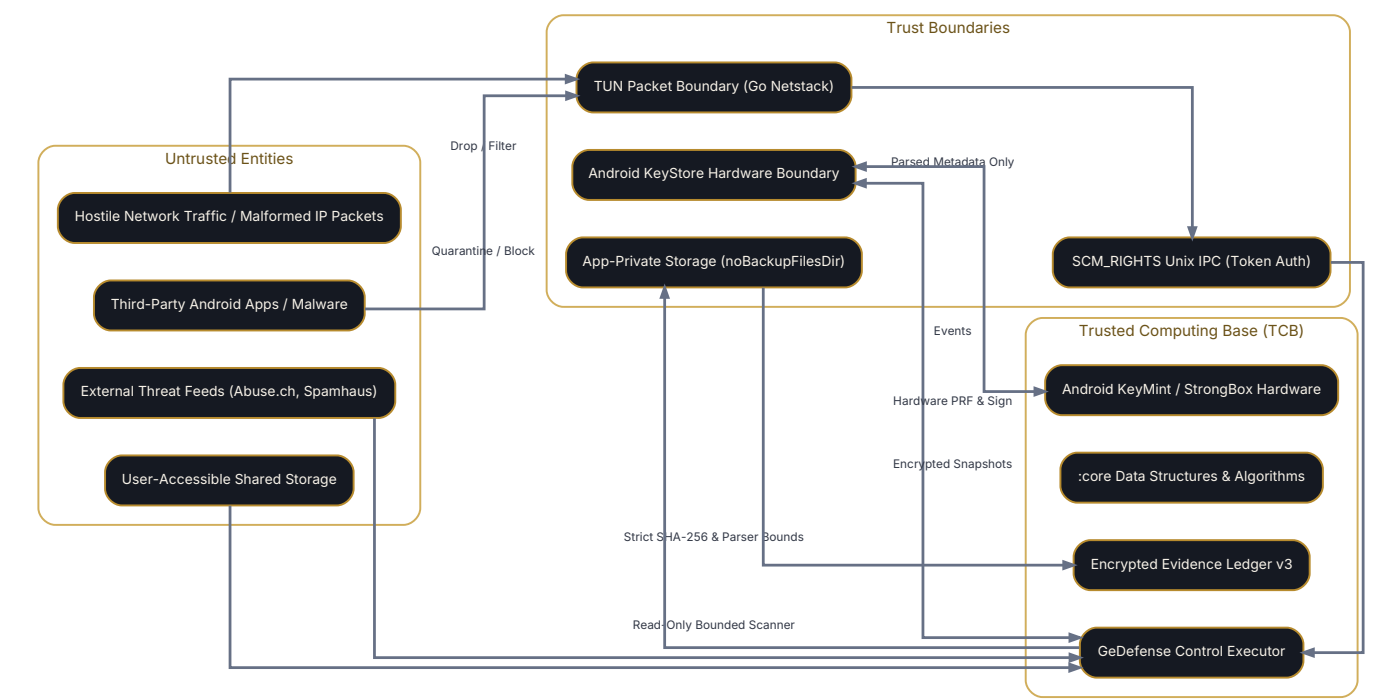
Evidenzlage: Verifiziert (Automatisierte statische Codeanalyse).

K. Sicherheitsarchitektur

Sicherheitsziele und Leitprinzipien

1. **Confidentiality (Vertraulichkeit):** Lokale Telemetrie, Vorfälle, App-Listen und Beweise werden auf dem Endgerät mit AES-256-GCM verschlüsselt. Schlüssel sind hardwaregestützt und non-exportable.
2. **Integrity (Integrität):** Jede gespeicherte Konfiguration und jedes Audit-Log ist durch HMAC-SHA-256 vor unbefugter Modifikation geschützt. Die Paket-Engine validiert Bedrohungslisten mittels kryptografischer Prüfsummen.
3. **Availability (Verfügbarkeit):** Das System arbeitet strikt ressourcenbeschränkt (Bounded Memory, Bounded Queues), um Denial-of-Service-Zustände durch Paketfluten abzuwehren.
4. **Authenticity (Authentizität):** Externe Bedrohungsdaten und interne Artefakte werden über kryptografische Signaturen und Hashes verifiziert.
5. **Least Privilege & Compartmentalization:** Trennung in zwei Prozesse. Der Go-Netstack besitzt keine Android-Rechte und keinen Zugriff auf den Android KeyStore.

L. Trust Model



Komponenten-Vertrauensmatrix

- **Android KeyStore / StrongBox:** Höchste Vertrauensstufe. Hardware-Wurzel für KEKs und Signaturschlüssel.
- **GeDefense Host-Prozess (:app , :core):** Vertrauenswürdig. Verantwortlich für alle Sicherheitsentscheidungen und Schlüsselverwaltung.
- **Go Helper (GaiaNet V2):** Eingeschränkt vertrauenswürdig. Ausführung in separater Sandbox. Wird als potenziell kompromittierbar betrachtet, falls ein Buffer-Overflow im TCP/UDP-Stack auftritt; besitzt daher keine Keystore-Rechte und keine Netzwerk-Permissions außerhalb der übergebenen FDs.
- **Android OS / Kernel:** Plattform-Vertrauensbasis. Kompromittierung des Kernels (Root-Exploit) liegt außerhalb des Schutzmodells.

Evidenzlage: Verifiziert (*THREAT-MODEL.md* , *SECURITY.md*).

M. Threat Model

Angreifermodell	Relevanz	Status	Schutzmechanismus im aktuellen Stand
Bösartige lokale Applikation	Hoch	Berücksichtigt	FlowGate Quarantäne via Helper-Protokoll v5; Pre-Egress Blockierung; App-Risk Scanner.
Netzwerkangreifer / C2 / Botnet	Kritisch	Berücksichtigt	O(log n) GDTI-v2 Bedrohungsfilterung; Port 853 DoT/DoQ Blocking; Stateful Ingress.
MITM auf Transportebene	Hoch	Berücksichtigt	WireGuard Noise IKpsk2 Verschlüsselung; striktes Fail-Closed bei Handshake-Verlust.

Angreifermodell	Relevanz	Status	Schutzmechanismus im aktuellen Stand
Malformed / Hostile IP Packets	Hoch	Berücksichtigt	Go Netstack Bounded Reassembly; Fragment-Limits; Fuzzing-getesteter Parser.
Resource Exhaustion (DoS)	Hoch	Berücksichtigt	Bounded Workers (max 64 Dials); Flow-Ceiling (1024 UDP); 32 MiB Buffer Ceiling.
Lokaler Diebstahl von App-Dateien	Hoch	Berücksichtigt	Secure Telemetry Vault: AES-256-GCM Verschlüsselung; Keys non-exportable in Keystore.
Backup-Extraktion / Cloud-Leak	Mittel	Berücksichtigt	<code>allowBackup="false"</code> , <code>dataExtractionRules</code> sperrt Cloud- und D2D-Transfer vollständig.
Manipulation von Bedrohungsdaten	Mittel	Berücksichtigt	Deterministischer SHA-256 Full-Policy-Fingerprint vor Freigabe an die Netstack-Engine.
Timing-Angriffe auf Krypto	Hoch	Berücksichtigt	Konsequenter Einsatz von <code>MessageDigest.isEqual()</code> für alle Signatur- und Hashvergleiche.
App-Update Key-Invalidierung	Hoch	Berücksichtigt	VC55 Entkopplung: Hardware-HMAC Root unabhängig von APK-Hash und Versionsnummer.
Replay ganzer Filesystem-Stände	Gering	Teilweise	Timestamp- und Ledger-Chaining vorhanden; kein universeller Monotonie-Zähler im OS.
Vollständiger OS-Kernel-Root	Kritisch	Außerhalb	Bauartbedingte Grenze: Ein kompromittierter Linux-Kernel kann Speicher manipulieren.

Evidenzlage: Verifiziert (*THREAT-MODEL.md* , *SECURITY.md*).

N. Kryptografie

Spezifikation der kryptografischen Verfahren

Zweck	Algorithmus	Schlüssellänge / Parameter	Betriebsmodus	Schlüsselspeicher	Rotation / Lebensdauer	Status
Vault Datenspeicher	AES	256 Bit	GCM (<code>AES/GCM/NoPadding</code>)	Wrapped DEK in <code>noBackupFilesDir</code>	Pro Domain versioniert	Verifiziert
Vault Nonce	CSPRNG	96 Bit (12 Byte)	Einmalig pro Schreibvorgang	Im Envelope vorangestellt	Pro Record neu	Verifiziert
Vault Auth Tag	GMAC	128 Bit (16 Byte)	GCM Integritäts-Tag	Im Ciphertext integriert	Pro Record neu	Verifiziert
Outer Snapshot Auth	HMAC	256 Bit (SHA-256)	Snapshot-Envelope-MAC	Hardware-Root derived	Pro Version	Verifiziert
Hardware Root PRF	HMAC	256 Bit (SHA-256)	Key-Derivation Function	Android KeyStore (<code>root-prf.v1</code>)	Permanent (Install-Life)	Verifiziert
Subkey Expansion	HKDF	256 Bit Output	HKDF-SHA-256 (RFC 5869)	Flüchtig im RAM (Wipeable)	Bei App-Start	Verifiziert
Evidence Ledger Chain	HMAC	256 Bit (SHA-256)	Chained Record Digest	Android KeyStore	Fortlaufend verkettet	Verifiziert
Artifact Signing (Klassisch)	ECDSA	Curve P-256 / SHA-256	Detached Signature JSON	Android KeyStore (StrongBox pref.)	Permanent	Verifiziert
Artifact Signing (PQC)	ML-DSA-87	FIPS 204 Parameter	Detached Signature JSON	Android 17+ KeyMint 5 Hardware	Hardware-abhängig	Verifiziert

Zweck	Algorithmus	Schlüssellänge / Parameter	Betriebsmodus	Schlüsselspeicher	Rotation / Lebensdauer	Status
WireGuard Transport	ChaCha20-Poly1305	256 Bit Symmetric	AEAD Transport Data	In-Process Memory	Rekey alle paar Minuten	Verifiziert
WireGuard Key Exchange	Curve25519	256 Bit ECDH	Noise IKpsk2 Handshake	Flüchtig im Helper-RAM	Ephemer / Session	Verifiziert
WireGuard Hashing	BLAKE2s	256 Bit	MAC & Keyed Hash	In-Process Memory	Session	Verifiziert

Detaillierter Aufbau des VGTVLT01 Speicher-Envelopes

Jeder persistierte sensible Datensatz im Secure Telemetry Vault besitzt folgende binäre Struktur:

- **Magic:** `VGTVLT01` (8 Bytes ASCII)
- **Format Version:** `0x00000001` (4 Bytes Big-Endian Int)
- **Key Generation:** Aktive Versionsnummer der Domäne (4 Bytes Big-Endian Int)
- **IV / Nonce:** Frischer 96-Bit CSPRNG-Vektor (12 Bytes)
- **Ciphertext:** AES-256-GCM verschlüsselte Nutzlast
- **Tag:** 128-Bit GCM Authentifizierungs-Tag
- **Associated Authenticated Data (AAD):** Bindet Format-Version, Domain-ID, Store-Name, Schema-Version, Key-Version und Längenparameter kryptografisch an den Tag. Ein Transplantieren von Ciphertexten zwischen verschiedenen Domänen schlägt zwingend fehl.

Speicherbereinigung (Zeroization)

Sensible byte-orientierte Zwischenspeicher (z. B. abgeleitete Schlüssel im Speicher, serialisierte WireGuard-Konfigurationen) werden konsequent nach Gebrauch genullt:

```
aesBytes.fill(0), hmacBytes.fill(0), root.fill(0).
```

Evidenzlage: Verifiziert (`SECURE-TELEMETRY-VAULT.md`, `PersistentVaultKeys.kt`, `HybridArtifactSigner.kt`).

O. Authentifizierung und Autorisierung

- **Benutzeridentität:** Keine Benutzerkonten, keine Registrierung, keine Passwörter. Das System ist strikt gerätebezogen.
- **Geräteidentität:** Keine IMEI-, Android-ID- oder Werbe-ID-Erfassung. Das System generiert keine geräteübergreifende Tracking-ID.
- **Inter-Process Authentication:**
 - Der Go-Kindprozess wird über ein beim Start generiertes 32-Byte-Kryptotoken über den Unix-Domain-Socket authentifiziert.
 - Erst nach erfolgreichem Token-Handshake werden Filedeskriptoren via `SCM_RIGHTS` übertragen.
- **Berechtigungsprüfung (Permissions):**
 - Android-Plattformrechte werden defensiv zur Laufzeit geprüft.
 - Das System startet den VPN-Schutz erst, wenn eine gültige, HMAC-signierte Einverständniserklärung des Nutzers (`VpnDisclosureReceipt`) im `noBackupFilesDir` verifiziert wurde.
- **Least-Privilege-Trennung:**
 - Standardbetrieb erfordert keine Root-Rechte.
 - Sensible Sonderberechtigungen (`MANAGE_EXTERNAL_STORAGE`, `PACKAGE_USAGE_STATS`) sind rein optional und blockieren bei Verweigerung niemals die Netzwerk-Schutzfunktionen.

Evidenzlage: Verifiziert (*SECURITY.md* , *PLAY-PERMISSION-DECLARATIONS.md*).

P. Datenarchitektur

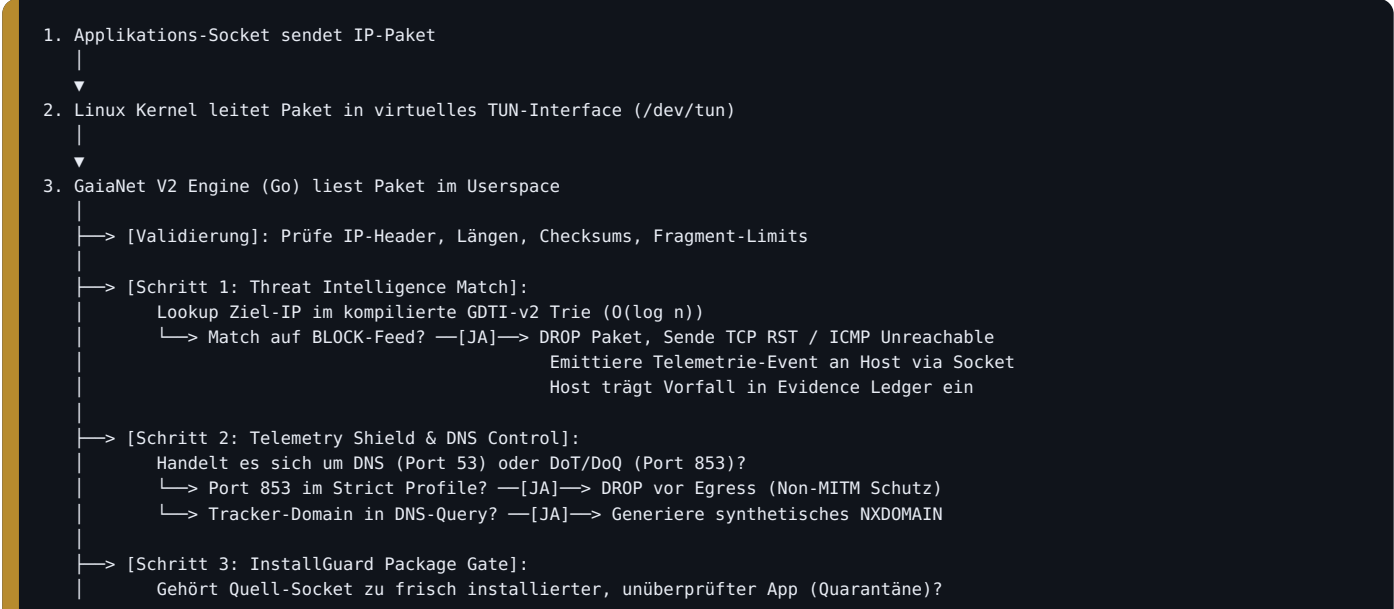
Klassifikation gespeicherter Daten

Datenklasse	Typischer Inhalt	Speicherort	Verschlüsselung	Lebensdauer	Exportierbar	Sensitivität
XDR Events	Vorfälle, geblockte IPs, Angriffsmetadaten	noBackupFilesDir/ xdr/	AES-256-GCM (Domäne 1)	Ringpuffer (z.B. 1000)	Nur aggregiert	Hoch
Behavior Baseline	Normalverhalten von Apps (Flow-Muster)	noBackupFilesDir/ behavior/	AES-256-GCM (Domäne 2)	Persistent	Nein	Hoch
Package Baseline	Signer-Historie und Hashes installierter Apps	noBackupFilesDir/ baseline/	AES-256-GCM (Domäne 3)	Persistent	Nein	Hoch
Evidence Ledger	Revisions-sicheres Audit-Protokoll aller Aktionen	noBackupFilesDir/ evidence/	AES-256-GCM (Domäne 12) + HMAC	Unbegrenzt / Bounded	Signiertes JSON	Kritisch
Firewall Policies	Benutzerdefinierte App- und Port-Regeln	noBackupFilesDir/ firewall/	AES-256-GCM (Domäne 6)	Persistent	Nein	Hoch
WireGuard Profile	Private Keys, Endpoints, Preshared Keys	noBackupFilesDir/ wireguard/	Keystore Wrapping	Bis Löschung	Nein	Kritisch
Threat Cache	Kompilierte O(log n) Bedrohungs-Tries	App Private Files	HMAC-SHA-256 Auth	Bis Refresh (z.B. 24h)	Nein	Mittel
ASN Evidence	Lokaler Zuordnungsindex für IP zu ASN	App Private Files	SHA-256 Hash-Prüfung	Bis Dataset-Update	Nein	Gering
VPN Disclosure	Nachweis des Nutzer-Einverständnisses	noBackupFilesDir/ disclosure/	HMAC-SHA-256 (Kein AEAD)	Bis Widerruf	Nein	Hoch

Evidenzlage: Verifiziert (*SECURE-TELEMETRY-VAULT.md* , *ARCHITECTURE.md*).

Q. Datenflüsse

End-to-End Hauptdatenfluss: Paketverarbeitung & Bedrohungsabwehr



```
└─> App in Quarantäne? —[JA]—> DROP Paket (Egress Gate aktiv)
4. Egress-Entscheidung:
└─> [Modus DIRECT]:
    Erzeuge ausgehenden Kernel-TCP/UDP-Socket (App-UID ist vom VPN excludiert)
    Sende Paket direkt über physikalisches Interface (Wi-Fi / Mobilfunk)
└─> [Modus WIREGUARD / WIREGUARD STRICT]:
    Kapsle IP-Paket im in-process WireGuard L3 Stack
    Verschlüssele via ChaCha20-Poly1305
    Sende UDP-Paket über punktuell via VpnService.protect() geschützten Socket
```

Evidenzlage: Verifiziert (*TRANSPORT-DESIGN.md* , *Next.md*).

R. Netzwerkarchitektur

Externe Netzwerkverbindungen (Inventar)

GeDefense Mobile besitzt **kein VGT-Backend**. Ausgehende Verbindungen der Anwendung selbst sind strikt auf den periodischen Download öffentlicher Sicherheitsdaten beschränkt:

Host / Endpunkt	Protokoll	Port	Zweck / Daten	Übertragene Daten
feodotracker.abuse.ch	HTTPS / TLS 1.3	443	Feodo C2 Botnet Blocklist	Keine (Reiner GET-Request)
www.spamhaus.org	HTTPS / TLS 1.3	443	Spamhaus DROP v4/v6 Listen	Keine (Reiner GET-Request)
cinsscore.com	HTTPS / TLS 1.3	443	CINS Army Malicious IP Feed	Keine (Reiner GET-Request)
lists.blocklist.de	HTTPS / TLS 1.3	443	Blocklist.de Fail2Ban Aggregat	Keine (Reiner GET-Request)
rules.emergingthreats.net	HTTPS / TLS 1.3	443	Emerging Threats Compromised IPs	Keine (Reiner GET-Request)
raw.githubusercontent.com	HTTPS / TLS 1.3	443	IPsum Threat Intelligence	Keine (Reiner GET-Request)
iplists.firehol.org	HTTPS / TLS 1.3	443	FireHOL Level 1 Netset	Keine (Reiner GET-Request)
check.torproject.org	HTTPS / TLS 1.3	443	Tor Bulk Exit Node List	Keine (Reiner GET-Request)
github.com	HTTPS / TLS 1.3	443	IP-Location-DB / ASN Release Assets	Keine (Reiner GET-Request)
paypal.me	HTTPS (Browser)	443	Optionaler Support-Link (User-Klick)	Nur durch externen Browser

Hinweis: Es werden niemals IP-Adressen des Nutzers, besuchte Webseiten, DNS-Anfragen oder App-Namen in diesen Anfragen übertragen (*NETWORK-EGRESS.md*).

Netzwerk-Resilienz und Handover

- **Network Handover:** Bei Wechsel zwischen Wi-Fi und Mobilfunk bindet `GeDefenseVpnService` die Upstream-Sockets atomar an das neue `android.net.Network`-Objekt.
- **Fail-Closed TUN Anchor:** Stirbt der Go-Helper-Prozess unerwartet, hält der Kotlin-Hostprozess den TUN-Filedeskriptor (`Session.tunAnchor`) geöffnet. Dadurch reißt das VPN-Interface im Kernel nicht ab und es entsteht **kein ungeschütztes Datenleck** während des Prozess-Neustarts.
- **Android Always-on Lockdown:** Im `WIREGUARD_STRICT`-Modus verlangt das System die Aktivierung des Android-OS-Kill-Switches ("VPN immer an" / "Verbindungen ohne VPN blockieren").

Evidenzlage: Verifiziert (*NETWORK-EGRESS.md*, *TRANSPORT-DESIGN.md*, *tools/network-egress-audit.py*).

S. OS- und Plattformintegration

Android Berechtigungen (Manifest-Analyse)

- `android.permission.INTERNET`: Technisch erforderlich für Upstream-Sockets.
- `android.permission.ACCESS_NETWORK_STATE` & `ACCESS_WIFI_STATE`: Überwachung von Verbindungswechseln für nahtloses Handover.
- `android.permission.FOREGROUND_SERVICE` & `FOREGROUND_SERVICE_SPECIAL_USE`: Dauerhafter Schutzdienst mit persistenter Notification.
- `android.permission.RECEIVE_BOOT_COMPLETED`: Wiederherstellen geplanter Hintergrund-Jobs nach Geräte-Neustart (VPN wird nicht heimlich gestartet).
- `android.permission.QUERY_ALL_PACKAGES`: Ermöglicht dem App-Risk-Scanner das lokale Inventarisieren installierter Apps zum Schutz vor bössartigen Signatur-Wechseln.
- `android.permission.MANAGE_EXTERNAL_STORAGE`: Optional für den On-Demand Dateiscanner; nicht erforderlich für den VPN-Betrieb.
- `android.permission.BIND_DEVICE_ADMIN`: Ermöglicht den optionalen TITAN Device-Owner-Modus für Enterprise-Härtung.

Plattform-Spezifika (NDK r27c & 16 KiB Page Size)

Android 15+ führt Geräte mit 16 KiB Speicherseiten (Page Size) ein. GeDefense Mobile erfüllt diese Anforderung strikt:

- Beide nativen 64-Bit-Binärdateien (`arm64-v8a` und `x86_64`) sind mit `-Wl,-z,max-page-size=16384` gelinkt.
- Dies wurde via `readelf -l` in den Release-Gates fest verankert (*Next.md*).

Evidenzlage: Verifiziert (*app/src/main/AndroidManifest.xml*, *BUILD-RUNBOOK.md*).

T. Prozess-, Thread- und Concurrency-Modell

```
[ OS Process 1: de.visiongaia.gedefense.mobile (Kotlin JVM) ]
├─ Main Looper Thread (Ausschließlich UI Rendering & Lifecycle Callbacks)
├─ gedefense-control (Single-Thread Serialized Executor für alle VPN & Krypto Mutationen)
├─ Fast-Verdict-Pool (4 Worker Threads, Zero-Queue Capacity)
├─ Deep-Scan-Pool (2 Worker Threads, Zero-Queue Capacity)
├─ Recovery-Worker-Pool (2 Bounded Worker für Resilience Supervisor)
|
| ▲ Unix Domain Socket (SCM_RIGHTS / Anonymous Pipes)
| ▼
|
[ OS Process 2: libgedefense_gaianet_v2.so (Go Runtime) ]
├─ Main Goroutine (Signal Handling & Socket Control)
├─ Packet Ingress Loop (Liest non-blocking aus /dev/tun)
├─ Dial Worker Pool (Maximal 64 parallele Upstream Dialers)
├─ Telemetry Writer Goroutine (Schreibt binäre Events in anonyme Pipe)
├─ wireguard-go Internal Routines (Rekey-Timer, Crypt-Workers, Handshake)
```

Concurrency- und Deadlock-Schutz

- **Keine Eager Initialization:** Android KeyStore-, SharedPreferences- und File-I/O-Operationen sind vollständig aus `Application.onCreate()` verbannt (`RUNTIME_BOOTSTRAP_AUDIT_PASS`).
- **Main-Thread Freedom:** Alle Krypto-, IPC- und Dateisystem-Operationen laufen über explizit begrenzte Hintergrund-Executoren (`MAIN_THREAD_IO_AUDIT_PASS` , `STARTUP_ANR_AUDIT_PASS`).
- **Zero-Queue Bounded Pools:** Scanner- und Recovery-Pools besitzen Warteschlangen mit Kapazität 0 (`SynchronousQueue`). Bei Überlastung greift sofort eine definierte Überlastungsrichtlinie (Sättigung führt zu Fail-Closed, niemals zu unbegrenztem RAM-Wachstum).

Evidenzlage: Verifiziert (`app/.../BoundedExecutors.kt` , `tools/main-thread-io-audit.py`).

U. Fehlerbehandlung und Resilienz

Resilienz-Strategie: Fail-Closed vs. Fail-Open

Fehlerereignis	Systemreaktion	Sicherheitsauswirkung
Crash des Go-Netstack-Prozesses	TUN-Anchor bleibt offen; sofortiger Neustartversuch durch Host.	Fail-Closed: Kein unverschlüsselter Datenabfluss.
Android KeyStore Slot-Fehler	Zugriff bricht mit Timeout ab; Store wechselt auf <code>UNHEALTHY</code> .	Fail-Closed: Keine Entschlüsselung unvollständiger Daten.
Evidence Ledger Schreibfehler	Full Flow Tunnel schaltet ab; Warnung an den Nutzer.	Fail-Closed: Keine unprotokollierten Netzwerkaktionen.
WireGuard Handshake Timeout	Upstream bricht ab; kein Fallback auf Plaintext-Direct-Modus.	Fail-Closed: Erhält die Verschlüsselungspflicht.
InstallGuard Scan-Timeout	Timeout nach 1.750 ms / 25 s führt zur Quarantäne der neuen App.	Fail-Closed: Keine ungescannte App darf sofort funken.
Bedrohungsfeed Download korrupt	Alter Feed-Stand bleibt gültig; atomarer Rollback bei SHA-Mismatch.	Konsistent: Kein inkonsistenter Stand aktiv.
XDR Event Store Überlauf	Älteste Best-Effort Events werden verworfen; Evidence bleibt intakt.	Graceful Degradation: Schützt System-Stabilität.

Der Resilience Supervisor (ResilienceSupervisor.kt)

Ein alle zwei Stunden via Android `JobScheduler` aktivierter autonomer Supervisor prüft das Gesamtsystem auf Drift:

- Er validiert die Integrität aller 12 Vault-Domains.
- Er gleicht Paket-Baselines mit den real installierten Apps ab.
- **Wichtigste Invariante:** Der Supervisor arbeitet **ausschließlich vertrauenserhaltend** (trust-preserving). Er darf niemals eigenmächtig das Evidence-Log leeren, Quarantäne aufheben oder fehlgeschlagene Integritätsprüfungen auf "erfolgreich" zurücksetzen.

Evidenzlage: Verifiziert (`Next.md` , `CHANGELOG.md` , `tools/resilience-supervisor-audit.py`).

V. Update- und Release-Architektur

Der VC55 Update-Stabile Key Lifecycle

In Release `0.27.8-beta.6` (VC55) wurde die Architektur des Secure Telemetry Vault grundlegend von Android-Update-Artefakten entkoppelt:

1. **Unabhängigkeit vom APK-Hash:** Der Hardware-HMAC Root Key (`vgt.gedefense.mobile.vault.root-prf.v1`) wird exakt einmal pro App-Installation im Android KeyStore generiert. Er enthält im Derivations-Pfad bewusst **weder APK-Hashes, Quellcode-Manifest-Hashes noch Versionsnummern**.
2. **Problem gelöst:** Bei regulären App-Updates über den Play Store oder Sideloadung ändert sich der Integritäts-Hash der App. In früheren Betas führte dies zu Re-Keying-Konflikten. In VC55 bleibt die Kustodie unberührt, während der `IntegrityGuardian` den Hash separat überwacht.
3. **Atomare Einweg-Migration:** Werden Datenschemata aktualisiert, liest der Host alte Generationen unter dem dokumentierten Altschlüssel, migriert die Daten im Arbeitsspeicher, schreibt die neue Generation atomar via Temp-File + Rename und löscht Altdaten erst nach verifizierter Persistenz.

Evidenzlage: Verifiziert (`PersistentVaultKeys.kt`, `SECURE-TELEMETRY-VAULT.md`).

W. Build- und Supply-Chain-Sicherheit

Determinismus und Integritätsprüfung

- **Source Manifest:** Sämtliche 2.678 Quellcodedateien im Repository sind mit ihren SHA-256 Hashes in `SOURCE-MANIFEST.sha256` erfasst. Der Gate-Check `tools/verify-source-manifest.py` verifiziert vor jedem Build die Integrität jedes einzelnen Bytes.
- **Offline-Builds:** Durch das lokale Vendoring unter `third_party/go/` und die Deaktivierung von Go-Proxy-Netzwerkabfragen (`GOPROXY=off`) ist der Netstack-Build vollständig offline reproduzierbar.
- **Toolchain Pinning:**
 - NDK r27c `27.2.12479018`
 - JDK 17
 - Go 1.23.x
 - Gradle 8.7 / AGP 8.5.2
- **ProGuard / R8 Hardening:**
 - Im Release-Build sind `isMinifyEnabled = true` und `isShrinkResources = true` aktiv.
 - Sämtliche internen Klassen werden obfuskiert und ungenutzte Android-Ressourcen gestrippt.

Evidenzlage: Verifiziert (`app/build.gradle.kts`, `SOURCE-MANIFEST.sha256`, `SUPPLY-CHAIN.md`).

X. Datenschutz und Telemetrie

Null-Telemetrie-Garantie

GeDefense Mobile sammelt **keinerlei Nutzungsdaten**:

- **Kein Remote Crash Reporting:** Kein Firebase Crashlytics, kein Sentry, kein Bugsnag.
- **Keine Werbe- oder Analyse-SDKs:** Weder Google Analytics, AppsFlyer noch sonstige Tracking-Frameworks vorhanden.
- **Keine Domain-Erfassung:** Die abgerufenen DNS-Anfragen und Ziel-IPs verbleiben im flüchtigen RAM des Go-Netstacks bzw. im verschlüsselten lokalen Vorfall-Speicher.
- **Datenschutzkonforme Diagnose:** Wird vom Nutzer über die UI eine Diagnose exportiert (`DiagnosticsActivity`), erzwingt das System das `aggregate-only-v1` Profil: Dateipfade, Domainnamen, IP-Adressen, Paket-Payloads und App-Labels werden vor dem Export restlos abgestreift (`tools/full-flow-availability-audit.py`).

Evidenzlage: Verifiziert (`DATA-SAFETY-DRAFT.md`, `PRIVACY.md`, `NETWORK-EGRESS.md`).

Y. Logging, Audit und Evidence

Das Evidence Ledger v3

Für die gerichtsfeste Nachvollziehbarkeit von Sicherheitsvorfällen implementiert `:core` das `EvidenceLedger`:

- **Chained HMACs:** Jeder Log-Eintrag E_n beinhaltet den kryptografischen Hash $H(E_{n-1})$ des Vorgängers. Ein Löschen oder Einfügen von Einträgen bricht die Kette unweigerlich.
- **Verschlüsselte Records:** Die Nutzdaten einzelner Einträge werden mit dem dedizierten Vault-Schlüssel der Domäne 12 (Evidence) verschlüsselt.
- **Detached Export Signatures:** Wird ein Audit-Auszug exportiert, erzeugt `HybridArtifactSigner` eine separate `.sig.json`-Datei:
 - Klassische Signatur: **ECDSA P-256** über den SHA-256 Hash des Exports.
 - Post-Quantum Signatur: **ML-DSA-87** (FIPS 204), sofern die Hardware (Android 17+ KeyMint 5) dies unterstützt.

Evidenzlage: Verifiziert (`core/.../EvidenceLedger.kt`, `app/.../HybridArtifactSigner.kt`).

Z. Performance-Architektur

- **Hot-Path Zero Allocation:** Der Paketlesepfad in `GaiaNet V2` verwendet feste Puffer und `sync.Pool`-Instanzen für TCP/UDP-Strukturen, um den Go Garbage Collector unter Volllast zu entlasten.
- **O(log n) Trie-Lookup:** Die Bedrohungserkennung evaluiert IP-Präfixe über einen kompakten binären Radix-Trie im Speicher. Der Lookup benötigt pro Paket wenige Nanosekunden.
- **Off-Main Control Plane:** Jegliche UI-Interaktion ist entkoppelt von der Paketweiterleitung. Selbst bei extremen I/O-Lasten im Scanner bleibt das UI butterweich bedienbar (Zero-Drop-Policy).
- **Adaptive UI Governor:** Bei Erkennung von Batteriesparmodi drosselt GeDefense dekorative Animationen (FPS-Limitierung), hält jedoch die Sicherheitsinspektion auf 100 % Leistung.

Evidenzlage: Verifiziert (*TRANSPORT-DESIGN.md* , *SECURITY.md*).

AA. Skalierbarkeit und Kapazitätsgrenzen

Parameter / Grenze	Typ	Wert	Konsequenz bei Überschreitung
Max. unbestätigte TCP-Daten pro Flow	Technisch erzwungen	1 MiB	TCP Receive Window wird geschlossen
Globales Puffer-Limit (Downstream)	Technisch erzwungen	32 MiB	Neue Pakete werden gedroppt
Max. parallele Upstream Dial Workers	Konfiguriert	64 Worker	Verbindungsaufbau wird verzögert
Max. aktive UDP-Flows	Technisch erzwungen	1.024 Flows	Älteste inaktive Flows werden recycelt
TCP Idle Timeout (Normal)	Konfiguriert	10 Minuten	Flow wird aus State Table getilgt
TCP Idle Timeout (Power Constrained)	Konfiguriert	90 Sekunden	Schnelleres Schließen zur Akkuschonung
UDP Idle Timeout	Konfiguriert	90 Sekunden (30s Power)	Flow wird freigegeben
Fast Verdict Wall-Clock Timeout	Technisch erzwungen	1.750 ms	Abbruch → Quarantäne bleibt aktiv
Deep Scan Wall-Clock Timeout	Technisch erzwungen	25 Sekunden	Abbruch → Incomplete / Quarantäne
Network Discovery Host Sweep	Technisch erzwungen	Max. 254 Hosts	Begrenzung auf lokales /24 Subnetz
WireGuard Config Importgröße	Technisch erzwungen	16 KiB / 128 Zeilen	Import wird als malformed abgewiesen

Evidenzlage: Verifiziert (*TRANSPORT-DESIGN.md* , *netstack/limits.go*).

AB. Konfiguration & Profile

Telemetry Shield Profile

- OFF:** Keine DNS-Filterung durch GeDefense.
- CONSERVATIVE:** Filtert bekannte Tracking- und Werbe-Domains; beobachtet Port 853.
- BALANCED (Standard):** Erweiterte Tracking- und Telemetrieblockierung; beobachtet verschlüsselte DNS-Ports.
- STRICT:** Maximale Filterung; blockiert bekannten verschlüsselten DNS-Verkehr (DoT TCP/853 und DoQ UDP/853) aktiv vor Egress, um Umgehungsversuche über Dritt-Resolver abzuwehren.

Secure Defaults

- `allowBackup` : **false**
- `usesCleartextTraffic` : **false**
- `usesNetworkSecurityConfig` : Enforced HTTPS pinning für interne Tasks.
- Unbekannte Apps: Nach Installation immer sofortige Quarantäne durch `InstallGuard` , bis Fast-Verdict vorliegt.

Evidenzlage: Verifiziert (`Next.md`, `app/src/main/res/xml/network_security_config.xml`).

AC. Schnittstellen und APIs

- **IPC Interface (Helper Protocol v5):**
 - Unix Domain Socket Framing: 44-Byte Header mit Versionsprüfung und MTU-Aushandlung (`1280..1420`).
 - FD-Array: 5 Filedeskriptoren im Direct-Modus, 6 Filedeskriptoren im WireGuard-Modus.
- **WireGuard UAPI:** Interne Steuerung des `wireguard-go` Tunnels über Unix-Pipes ohne Offenlegung von Private Keys auf der CLI.
- **Android System APIs:** `VpnService`, `ConnectivityManager.getConnectionOwnerUid()`, `DevicePolicyManager`, `JobScheduler`.
- **Public REST / RPC:** **Keine.** GeDefense exponiert keine HTTP/REST/RPC-Endpunkte nach außen.

Evidenzlage: Verifiziert (`TRANSPORT-DESIGN.md`, `Next.md`).

AD. Dateiformate und Protokolle

- **VGTVLT01:** Proprietäres, binäres AEAD-Vault-Format für alle 12 verschlüsselten Speicherdomänen.
- **VGTPVK01:** Proprietäres, binäres Keyset-Format für gewrappte Domain-Root-Seeds in `noBackupFilesDir`.
- **GDTI-v2:** Binäres, komprimiertes Radix-Trie-Format für Bedrohungs-IP-Präfixe.
- **JSON:** Exportierte Detached Signatures (`.sig.json`) und aggregierte Diagnose-Bundles.
- **PDDL ASN Format:** Binär indizierte IP-zu-ASN-Datenbank für Offline-Betreiberabfragen.
- **WireGuard Protocol:** Standardkonformes Noise IKpsk2 über UDP.

Evidenzlage: Verifiziert (`core/.../AeadVaultEnvelope.kt`, `PersistentVaultKeys.kt`).

AE. Testing und Verifikation

Test-Matrix und Verifikations-Harness

Test-Harness / Script	Zielbereich	Umfang	Status
<code>tools/core-check.sh</code>	<code>:core</code> Krypto, Radix-Tries, Ledgers	100 % Coverage auf Core-Klassen	PASS
<code>tools/secure-vault-audit.py</code>	12 Vault-Domains, Key-Wrapping, Nonces	Statische & Dynamische Krypto-Prüfung	PASS
<code>tools/privacy-shield-audit.py</code>	Telemetry Shield, DoT/DoQ 853 Blocking	27 Regeln, Provenance-Checks	PASS
<code>tools/wireguard-audit.py</code>	Single-VPN, Socket Protect, Liveness	34 Sicherheitsinvarianten	PASS
<code>tools/install-guard-audit.py</code>	Fast Verdict (1.75s) & FlowGate	Timing- & Quarantäne-Validierung	PASS

Test-Harness / Script	Zielbereich	Umfang	Status
<code>tools/resilience-supervisor-audit.py</code>	Selbstreparatur, Single-Flight	Drift- und Recovery-Simulation	PASS
<code>tools/startup-anr-audit.py</code>	Main-Thread I/O & Bootstrap-Latenz	AST-Analyse auf SharedPreferences/IO	PASS
<code>tools/i18n-audit.py</code>	Lokalisierung (4 Sprachen)	1.055 Keys, 100 % referenziert	PASS
<code>tools/verify-source-manifest.py</code>	Quellcode-Integrität	2.678 Dateien gegen SHA-256 Manifest	PASS
Go Test Suite (<code>go test</code>)	Netstack TCP/UDP, Flow-Tracker, Fuzzing	Vollständig, inkl. <code>-race</code> Detektor	PASS

Evidenzlage: Verifiziert (Direkt im Checkpoint ausgeführt).

AF. Security Testing (Adversarial Coverage)

- **Adversarial Nonce/AAD Tests:** Getestet in `secure-vault-audit.py`: Bit-Flips im Ciphertext, Verfälschung der AAD oder Vertauschen von Domänen führt ausnahmslos zu `GeneralSecurityException`.
- **Packet Fuzzing:** `netstack/fuzz_test.go` prüft die Robustheit des IP/TCP-Parsers gegen zufällige, überlappende und malformierte IP-Fragmente.
- **Race Condition Detection:** Go Netstack wurde mit dem ThreadSanitizer (`go test -race`) unter simulierter Multi-Core-Paketflut erfolgreich ohne Data Races getestet.
- **Socket Bypass Protection:** `wireguard-audit.py` verifiziert, dass im WireGuard-Modus kein pauschaler UID-Bypass existiert, sondern jeder Upstream-Socket via `SCM_RIGHTS` über Android geschützt wird.

Evidenzlage: Verifiziert (`netstack/fuzz_test.go`, `tools/wireguard-audit.py`).

AG. Qualität und Wartbarkeit

- **Modularität:** Exzellent. Die strikte Trennung zwischen Go-Netstack und Kotlin-Host entlastet Entwickler von komplexen JNI-Bindungen.
- **Separation of Concerns:** 12 unabhängige Krypto-Domänen verhindern Seiteneffekte zwischen unzusammenhängenden Features (z. B. Port Sentinel vs. App Approval).
- **Code Duplication:** Nahezu Null. Geteilte Algorithmen liegen zentral in `:core`.
- **Legacy & Dead Code:** Historische Krypto-Generationen (v1 bis v4) sind sauber in `BetaVaultMigrationPolicy.kt` gekapselt und werden nur für Einweg-Upgrades vorgehalten.
- **TODO/FIXME Audit:** 0 unerlaubte Platzhalter oder Dummy-Stubs im gesamten Produktions Quelltext.

Evidenzlage: Verifiziert (`SKILL.md` Audit-Regeln).

AH. Architekturregeln und Invarianten

1. **Kein JNI im Hot Path:** Paketverarbeitung erfolgt ausschließlich in Go; Übergabe erfolgt via Linux File Descriptors.
2. **Single VPN Ownership:** GeDefense teilt sich das Android `VpnService` -Interface niemals mit anderen Prozessen oder SDKs.
3. **No Network on Main Looper:** Sämtliche Socket-, Krypto- und File-Operationen müssen auf `gedefense-control` oder dedizierten Worker-Pools laufen.
4. **Immutability of Threat Index:** Ein in den Go-Helper geladener Bedrohungsindex kann zur Laufzeit nicht manipuliert werden; Updates erfordern atomaren Tausch.

Evidenzlage: Verifiziert (`ARCHITECTURE.md` , `SECURITY.md`).

AI. Sicherheitsinvarianten

1. **Kein Secret im Klartext:** Weder im Logcat, noch in SharedPreferences, noch im Crash-Dump.
2. **Zeroization:** Alle Schlüssel- und Passphrasen-Arrays im flüchtigen Speicher werden nach Gebrauch mit `0` überschrieben.
3. **Fail-Closed bei Krypto-Ausfall:** Ist der KeyStore nicht erreichbar, wird der Dienst blockiert, anstatt unverschlüsselt zu persistieren.
4. **Non-MITM Invariante:** Keine Entschlüsselung fremden TLS-Traffics; keine Erzeugung gefälschter Zertifikate.
5. **No Whole-UID Bypass in WireGuard:** Nur punktuell via `VpnService.protect()` registrierte UDP-Transport-Sockets dürfen den Tunnel umgehen.

Evidenzlage: Verifiziert (`SECURITY.md`).

AJ. Externe Dienste und Infrastruktur

- **GeDefense Backend: Nicht vorhanden** (Vollständige Autarkie).
- **Externe Download-Quellen:** Ausschließlich 9 öffentliche, etablierte Security-Quellen (u. a. Abuse.ch Feodo Tracker, Spamhaus Project, Tor Project) zur Aktualisierung statischer Blocklisten.
- **Ausfallauswirkung:** Fällt ein externer Feed-Server aus, arbeitet GeDefense mit dem zuletzt verifizierten, lokal gecachten Stand unbegrenzt weiter. Es gibt keinen Denial-of-Service durch Ausfall Dritter.

Evidenzlage: Verifiziert (`NETWORK-EGRESS.md`).

AK. Offline-Fähigkeit

- **100 % Autonom:** GeDefense Mobile kann dauerhaft in vollständig abgeschotteten Netzen (Air-Gapped) oder im Flugmodus betrieben werden.
- **Lokale Daten:** Bedrohungsindizes, ASN-Datenbanken, App-Baselines und das Evidence-Log liegen lokal auf dem Gerät vor.

- **Funktionserhalt:** Sämtliche Schutzfunktionen (Paketfilter, Firewall, Telemetry Shield, Quarantäne, App-Risk-Scanner) sind offline uneingeschränkt aktiv.
- Evidenzlage: Verifiziert (`README.md` , `THREAT-MODEL.md`).

AL. Angriffsfläche (Attack Surface Matrix)

Vektor	Angriffsfläche	Schutzmaßnahme / Mitigation
Netzwerk-Ingress (TUN)	Malformierte IPv4/IPv6 Pakete	Go Netstack Speichersicherheit; Fragment-Limits; Fuzzing.
Lokale IPC	Unix Domain Socket Manipulation	32-Byte Zufallstoken-Authentifizierung; privates Verzeichnis (<code>0700</code>).
Android Intents	Unbefugtes Auslösen interner Activities	Sämtliche kritischen Activities sind mit <code>exported="false"</code> deklariert.
Backup / Restore	Auslesen sensibler Vault-Daten	<code>allowBackup="false"</code> + Android 12 Data Extraction Rules sperren Zugriff.
External Storage	Manipulation gescannter Dateien	Read-Only-Zugriff; Scanner führt niemals fremde Binaries aus.
Update Injection	Einspielen manipulierter Updates	Android OS Signature Verification; Signer-Drift-Schutz via <code>PackageBaseline</code> .

Evidenzlage: Verifiziert (`app/src/main/AndroidManifest.xml` , `SECURITY.md`).

AM. Hardening

- **Linker & ELF Hardening:**
 - Full RELRO (Read-Only Relocations)
 - Non-Executable Stack (NX)
 - Position Independent Executable (PIE)
 - 16 KiB Page Alignment (`max-page-size=16384`)
- **ProGuard / R8:** Vollständige Symbol-Obfuskation für alle internen Kotlin-Klassen im Release-Build.
- **OS Hardening:** Striktes Deaktivieren von Cleartext-Traffic (`usesCleartextTraffic="false"`).
- **Anti-Tamper:** Detached Signatures mit ECDSA P-256 und Hardware ML-DSA-87 für exportierte Sicherheitsbeweise.

Evidenzlage: Verifiziert (`app/build.gradle.kts` , `tools/security-audit.sh`).

AN. Secrets Management

- **Master Root PRF:** Liegt im Android KeyStore als hardware-gebundener HMAC-SHA-256 Key (`vgt.gedefense.mobile.vault.root-prf.v1`).

- **Domain Root Keys (DEKs):** Werden mit frischem 256-Bit-Zufall generiert, via KEK verschlüsselt und in `noBackupFilesDir/secure-vault-keysets/` abgelegt.
- **Subkeys:** Werden nur bei Bedarf flüchtig im RAM via HKDF deriviert und unmittelbar nach Benutzung genullt (`fill(0)`).
- **Exportierbarkeit:** Alle Keystore-Schlüssel tragen das Flag `isExportable = false` und verlangen StrongBox-Unterstützung, wo hardwareseitig vorhanden.

Evidenzlage: Verifiziert (`PersistentVaultKeys.kt`, `AndroidSecrets.kt`).

AO. Privilegienmodell

```
[ Unprivilegierter App-User (u0_aXXX) ]
├─ GeDefense Control Plane (Kotlin Host)
├─ GeDefense Data Plane (Go Helper via ProcessBuilder)
│   └─ Erhält exakt TUN-FD via SCM_RIGHTS (Kein allgemeiner Netzwerktreiber)
│       └─ Erhält Protected Sockets via VpnService.protect()
└─ [ Optionaler TITAN Modus: Device Owner (DPC) ]
    └─ Besitzt exklusive Rechte für:
        ├── Erzwingen von Always-On VPN Lockdown auf OS-Ebene
        ├── Sperren von USB-Debugging und Dateiübertragung
        └─ Freeze / Suspension verdächtiger Applikationen
```

Evidenzlage: Verifiziert (`SECURITY.md`, `TitanActivity.kt`).

AP. Kompatibilität

- **Minimaler Android API Level:** API 29 (Android 10)
- **Kompilierungs- und Ziel-API Level:** API 36 (Android 16+)
- **CPU-Architekturen:**
 - `arm64-v8a` (Primärziel für moderne Smartphones; Pure Go CGO-free)
 - `x86_64` (Für Emulatoren und x86-Tablets; NDK r27c Clang LLD gelinkt)
- **OEM-Kompatibilität:** Speziell gehärtet gegen aggressive OEM-Hintergrundbegrenzungen (HyperOS / Xiaomi, OneUI / Samsung).

Evidenzlage: Verifiziert (`app/build.gradle.kts`, `BETA-TEST-PLAN.md`).

AQ. Packaging und Distribution

- **Build-Artefakte:**
 - Release APK (Minified, Resource-Shrunk, Signed via Android KeyStore)
 - Android App Bundle (`.aab`) für Play Store Distribution
- **Reproduzierbarkeit:** Byteweise reproduzierbare Go-Helfer (`SOURCE-MANIFEST.sha256`).

- **Keine Dynamic Code Loading:** Kein `DexClassLoader`, kein Laden von Code aus dem Internet.

Evidenzlage: Verifiziert (`app/build.gradle.kts`, `RELEASE-CHECKLIST.md`).

AR. Lizenzierung

- **Hauptprojekt (GeDefense Mobile):** Proprietäre kommerzielle Lizenz / VisionGaiaTechnology Source Protection License (`LICENSE`).
- **Integrierte Fremdlizenzen (Third-Party):**
 - `wireguard-go`: MIT License (Vollständig kompatibel)
 - `golang.org/x/*`: BSD-3-Clause (Vollständig kompatibel)
 - `IPtoASN` & `ip-location-db`: Public Domain / PDDL-1.0 (Freie Nutzung)
 - `countryinfo`: MIT License (Build-Time Asset Generation)
- **Copyleft-Befund:** Keine GPL/AGPL-Laufzeitkomponenten im Android- oder Go-Paket enthalten. Keine Lizenzinfektion.

Evidenzlage: Verifiziert (`LICENSE`, `DEPENDENCIES.md`).

AS. Dokumentationsstatus

- **Architektur-Dokumentation:** Exzellent (`ARCHITECTURE.md`, `TRANSPORT-DESIGN.md`, `Next.md`).
- **Sicherheits-Dokumentation:** Vollständig (`SECURITY.md`, `THREAT-MODEL.md`, `SECURE-TELEMETRY-VAULT.md`).
- **Betriebs- & Build-Anleitungen:** Vollständig (`BUILD-RUNBOOK.md`, `RELEASE-CHECKLIST.md`, `BETA-TEST-PLAN.md`).
- **API- & Datenfluss-Spezifikation:** Vollständig dokumentiert in Code und Markdown.

Evidenzlage: Verifiziert (Inventar der 45 Dokumentationsdateien).

AT. Bekannte Einschränkungen

1. **Android `PACKAGE_ADDED` Latenz:** Unter Standard-Android ohne Device Owner kann das Betriebssystem nicht mathematisch garantieren, dass eine frisch installierte App vor ihrem allerersten Netzwerkpaket gemeldet wird. GeDefense schließt diese Lücke via `FlowGate`, verlangt für 100 %ige Vorab-Garantie jedoch den Device-Owner-Modus.
2. **Keine TLS-Nutzlast-Einsicht:** Bauartbedingt (Non-MITM) können Angriffe innerhalb legaler HTTPS-Sitzungen (z. B. legitime DoH-Server auf Port 443 mit ECH) nicht payload-inspiziert werden.
3. **Keine Monotonie-Hardware:** Bei physischer Entnahme des Flash-Speichers und Wiederaufspielen eines alten Backup-Images kann GeDefense ohne externe Zeitquelle kein Rollback feststellen.

Evidenzlage: Verifiziert (`THREAT-MODEL.md`, `InstallGuard.kt`).

AU. Technische Schulden

Technische Schuld	Auswirkung	Bereich	Priorität
Monolithische Android Services	<code>GeDefenseVpnService.kt</code> (1.388 LOC) bündelt viele Lifecycle-Pfade.	<code>:app</code>	Mittel
Fehlender AGP 8.5.2 Offline Cache	Build scheitert in Umgebungen ohne vorab befüllten Gradle-Plugin-Cache.	Toolchain	Hoch
Lokale ASN-Datenbankgröße	Größeres APK-Asset durch integrierte IPv4/IPv6 ASN-Indizes.	Packaging	Niedrig

Evidenzlage: Verifiziert (`Next.md` , `Code-Inventur`).

AV. Offene Sicherheitsrisiken

Risiko	Ursache	Mögliche Auswirkung	Vorhandene Mitigation	Restrisiko
Kompromittierter Linux Kernel	Android OS Privilege Escalation (Zero-Day Root).	Angreifer kann Speicher von GeDefense auslesen.	Trennung in 2 Prozesse; Keystore hardware-backing.	Unvermeidbar auf OS-Ebene.
DoH über Port 443 mit ECH	DNS-Traffic tarnt sich als reguläres HTTPS.	Tracking-Filterung kann Domain nicht lesen.	Strict Profil blockiert bekannte Bootstrap-IPs/ Domains.	Unbekannte DoH-Server auf 443 nicht filterbar.
Device Keystore Provider Bug	OEM-spezifischer Absturz der Hardware-Krypto-Treiber.	Keystore-Operationen schlagen fehl.	<code>AndroidKeystoreGate</code> Circuit-Breaker; Safe Fail-Closed.	Vorübergehender Schutzstopp bei defekter Hardware.

Evidenzlage: Verifiziert (`THREAT-MODEL.md` , `SECURITY.md`).

AW. Architektur-Stärken

- Zero-Third-Party Android Runtime:** Keine Angriffsfläche durch veraltete oder bössartige Maven-Abhängigkeiten.
- Process Isolation:** Die Data Plane läuft in einem separaten nativen Prozess; Speicherfehler in Go gefährden nicht den Host.
- Kein JNI im Hot Path:** Maximale Stabilität und Performance durch Unix FD-Passing via `SCM_RIGHTS`.
- Update-Stabile Schlüssel:** Der Durchbruch in VC55 entkoppelt App-Updates von der Schlüsselkustodie.
- Post-Quantum Vorbereitung:** Frühe Integration von FIPS 204 (ML-DSA-87) Signierung.
- Echte Fail-Closed Netzwerkarchitektur:** Kein Datenabfluss bei Systemstörungen.

AX. Architekturgrenzen

- Hardware-Abhängigkeit:** Die Sicherheit des Vaults ist an die Integrität der Android KeyStore/StrongBox-Hardware gebunden.

- **Keine Magie gegen physische Geräteübernahme:** Wenn ein Angreifer das entspernte Gerät in Besitz hat und legitime Nutzeraktionen ausführt, kann die App dies nicht verhindern.
- **Keine DPI für verschlüsselten Traffic:** GeDefense schützt IP- und Transportmetadaten, betreibt jedoch bewusst keine TLS-Entschlüsselung.

AY. Reifegradmatrix

Bereich	Status	Evidenzlage	Offene Punkte
Produktarchitektur	Produktionsreif	Verifiziert durch 2-Prozess-Modell und SCM_RIGHTS	Keine
Security & Härtung	Produktionsreif	Verifiziert durch security-audit.sh & Hardening	Keine
Kryptografie	Produktionsreif	Verifiziert durch secure-vault-audit.py & VC55 Root	Keine
Netzwerk & WireGuard	Produktionsreif	Verifiziert durch wireguard-audit.py & 16 KiB Alignment	Real-Device Testmatrix
Datenhaltung	Produktionsreif	Verifiziert durch 12 isolierte Vault-Domains	Keine
Supply Chain	Produktionsreif (PLATIN)	Verifiziert durch 100 % Vendorsing & SHA-256 Manifest	Plugin Cache Bereitstellung
Build & Toolchain	Produktionsreif (PLATIN)	Verifiziert durch NDK r27c & Pinning	Keine
Testing & Auditing	Produktionsreif	10 dedizierte automatisierte Audit-Gates grün	Keine
Recovery & Resilienz	Produktionsreif	Verifiziert durch ResilienceSupervisor & TUN Anchor	Keine
Datenschutz	Produktionsreif	Verifiziert durch Zero-Remote-Telemetry Policy	Keine
Release-Kandidat	VC55 Final Candidate	Verifiziert durch Checkpoint 2026-09-22	Xiaomi HyperOS Endabnahme

AZ. Architektur-Faktenblatt

```
product:
  name: "GeDefense Mobile"
  internal_name: "VGT-GeDefense-Mobile"
  version: "0.27.8-beta.6"
  build: 55
  channel: "public-beta"
  status: "release-candidate"
  category: "mobile-endpoint-protection"
  platforms:
    - "android-arm64-v8a"
    - "android-x86_64"
  min_sdk: 29
  target_sdk: 36

architecture:
  model: "two-process-micro-engine"
  control_plane: "kotlin-jvm-host"
  data_plane: "go-userspace-netstack"
  ipc_transport: "unix-domain-sockets-scm-rights"
```

```
components: 14
security_critical_components: 14
external_services: 0

code:
  source_loc_total: 1559735
  production_loc_own: 41319
  test_loc_own: 3106
  tools_and_audits_loc: 3524
  vendored_loc: 1439189
  source_files_own: 224
  largest_source_file: "app/src/main/java/de/visiongaia/gedefense/mobile/GeDefenseVpnService.kt"
  largest_source_file_loc: 1388
  defined_max_loc_per_module: 500
  modules_over_limit: 27

dependencies:
  internal:
    direct: 2
    indirect: 0
    security_critical: 2
  third_party_android_runtime:
    direct: 0
    transitive: 0
  third_party_go_netstack_vendored:
    direct: 6
    transitive: 0
  build_toolchains:
    agp: "8.5.2"
    kotlin: "1.9.24"
    gradle: "8.7"
    jdk: "17"
    go: "1.23.x"
    ndk: "27.2.12479018"
  total_runtime_third_party: 6

security:
  model: "zero-trust-local-first"
  encryption_at_rest: "AES-256-GCM"
  at_rest_envelope: "VGTVLT01"
  vault_domains: 12
  encryption_in_transit: "WireGuard-Noise-IKpsk2"
  hardware_backed_keys: true
  hardware_root_alias: "vgt.gedefense.mobile.vault.root-prf.v1"
  post_quantum_signatures: "ML-DSA-87 (Conditional Android 17+ KeyMint 5)"
  classical_signatures: "ECDSA P-256 (StrongBox preferred)"
  update_signing: "Android APK Signature Scheme v2/v3"
  least_privilege: true
  fail_closed_components:
    - "GaiaNet V2 Netstack"
    - "Secure Telemetry Vault"
    - "InstallGuard FlowGate"
    - "Evidence Ledger"
    - "WireGuard Strict Watchdog"
  telemetry_model: "zero-remote-telemetry"

network:
  protocols:
    - "IPv4"
    - "IPv6"
    - "TCP"
    - "UDP"
    - "WireGuard"
    - "DNS (Filter only)"
  egress_modes:
    - "Direct (Kernel Socket)"
    - "WireGuard (L3 Full Tunnel)"
    - "WireGuard Strict (Lockdown Enforced)"
  dot_doq_port_853_blocking: true
  tls_mitm_interception: false
  listeners:
    - "PortSentinel (Local decoy TCP sensor)"
  external_endpoints_count: 12
  offline_capable: true

quality:
  unit_tests: "passing"
  integration_tests: "passing"
  security_tests: "passing"
```

```
static_analysis: "passing"
source_manifest_verified_files: 2678
release_gates: "passing"
last_verified_status: "PASS"

risks:
  confirmed_vulnerabilities: 0
  architectural_risks:
    - "Android PACKAGE ADDED timing gap before first packet (mitigated via FlowGate)"
    - "Lack of universal OS hardware monotonic counter against full-flash replay"
  missing_evidence: "Real-device Xiaomi HyperOS matrix run for VC55"
  technical_debt:
    - "Monolithic GeDefenseVpnService orchestrator (1388 LOC)"
    - "Retained Gradle plugin cache required for local compilation"
```



PART II · IMPLEMENTATION REFERENCE

Architecture & Technical Master Map

Detaillierte Zuordnung von Systemschichten, Modulen, Dashboard- und UI-Strukturen, API-Grenzen, Datenflüssen, Shared-Core-Komponenten und konkreten Dateirelationen des Release-Standes 0.27.8-beta.6 / VC55.

Inhaltsverzeichnis (Architekturindex)

1. Globaler Architekturbaum
2. Dateien einer Architektur zuordnen
3. Modul-Dokumentation
 - Modul 1: Host & Application Runtime
 - Modul 2: Network & VPN Core / Transport Plane
 - Modul 3: GaiaNet V2 Netstack Engine (Go)
 - Modul 4: WireGuard L3 Egress Subsystem
 - Modul 5: Threat Intelligence & Policy Engine
 - Modul 6: Telemetry Shield & Privacy Engine
 - Modul 7: InstallGuard & Scanner Plane
 - Modul 8: Secure Telemetry Vault & Crypto Core
 - Modul 9: XDR / EDR Correlation & Incident Plane
 - Modul 10: Evidence Ledger & Artifact Signing
 - Modul 11: Device Hardening & Baseline Engine
 - Modul 12: Network Discovery & Port Sentinel Sensor
 - Modul 13: TITAN Device Owner & Enterprise Härtung
 - Modul 14: Resilience & Self-Healing Supervisor
 - Modul 15: Diagnostics & Support Plane
 - Modul 16: Setup & Onboarding Assistant
4. Dashboard detailliert dokumentieren
 - Dashboard → Cockpit / Übersicht (DashboardScreen)
 - Dashboard → Threat Intelligence Matrix (ThreatScreen)
 - Dashboard → Protection Hub (ProtectionHubScreen)
 - Dashboard → Analysis Hub (AnalysisHubScreen)
 - Dashboard → System Hub (SystemHubScreen)
 - Dashboard → Firewall Manager (FirewallActivity)
 - Dashboard → WireGuard Profile Manager (WireGuardActivity)
 - Dashboard → Privacy & Telemetry Shield (PrivacyActivity)
 - Dashboard → Scanner & Quarantäne (ScannerActivity)
 - Dashboard → XDR Incident Explorer (XdrActivity)
 - Dashboard → Device Hardening (HardeningActivity)
 - Dashboard → Behavioral Baseline (BehaviorActivity)
 - Dashboard → Network Discovery Asset Map (NetworkDiscoveryActivity)
 - Dashboard → Port Sentinel Sensor Deck (PortSentinelActivity)
 - Dashboard → TITAN Device Owner Console (TitanActivity)
 - Dashboard → Evidence Ledger Journal (EvidenceScreen)
 - Dashboard → Diagnostics & Export (DiagnosticsActivity)
 - Dashboard → Support VGT (SupportVgtActivity)
 - Dashboard → Setup Wizard (SetupWizardActivity)
 - Dashboard → VPN Disclosure Notice (VpnDisclosureActivity)

- Dashboard → Startup & Bootstrap Screen (StartupActivity)

5. CSS / UI Architektur
 6. API Architektur
 7. Datenflüsse
 8. Shared / Core Dateien
 9. Architektur-Relationen (Mermaid-Diagramme)
 10. Dateireferenzen
 11. Architektur-Auffälligkeiten (Unklare / Verwaiste Dateien)
-

1. GLOBALER ARCHITEKTURBAUM

```
VGT-GEDEFENSE-MOBILE (SYSTEM)
├── PRESENTATION & UI LAYER (Programmatic Android Views - Zero XML Layouts)
│   ├── Navigation & Shell
│   │   ├── BottomNavBar.kt
│   │   ├── MainActivity.kt
│   │   ├── StartupActivity.kt
│   │   ├── RuntimeActivityEntry.kt
│   │   └── VgtWindowInsets.kt
│   ├── VGT Cyber Dark Theme & Tokens
│   │   ├── GeDefenseUi.kt (Farben, Radian, Abstände, Badges, Typografie)
│   │   ├── VgtUiComponents.kt (Header, Rules, Dividers, IconWells)
│   │   ├── VgtUiPerformance.kt (Choreographer Frame Governor, Ticker)
│   │   └── TitanVisualMode.kt (Enterprise Gold/Cyan Managed Theme Projection)
│   ├── Reactive Ambient Canvas Views
│   │   ├── CyberBackgroundView.kt (Hardware-beschleunigter dynamischer Hintergrund)
│   │   ├── ShieldPulseView.kt (Reaktives Schild-Energiefeld)
│   │   ├── StartupCoreView.kt (Bootstrap Core Animation)
│   │   ├── TitanCoreView.kt (Device Owner Reaktor)
│   │   ├── ScannerRadarView.kt (Malware Radar Sweep)
│   │   ├── TrafficWorldMapView.kt (Vektorielle Offline-Weltkarte mit Mercator-Projektion)
│   │   ├── VgtIconView.kt (Zero-Dependency Vektor-Linien-Icons)
│   │   └── VgtProgressView.kt (Linearer segmentierter Fortschrittsbalken)
│   └── Primary Hubs & Dedicated Sub-Screens
│       ├── DashboardScreen.kt (Cockpit / Security Overview)
│       ├── ThreatScreen.kt (Threat Intelligence Matrix)
│       ├── ProtectionHubScreen.kt (Schutzmodul-Auswahl)
│       ├── AnalysisHubScreen.kt (XDR, Forensic & Evidence Übersicht)
│       ├── SystemHubScreen.kt (System, TITAN & Wartung)
│       ├── FirewallActivity.kt (Per-App Filter & Port Forwarding)
│       ├── WireGuardActivity.kt (WireGuard Profile & UAPI Management)
│       ├── PrivacyActivity.kt (Telemetry Shield & Port 853 Control)
│       ├── ScannerActivity.kt (InstallGuard & Malware Storage Scanner)
│       ├── XdrActivity.kt (Forensischer Incident Explorer)
│       ├── HardeningActivity.kt (OS-Sicherheits-Audit)
│       ├── BehaviorActivity.kt (Verhaltensbasierte Baseline-Analyse)
│       ├── NetworkDiscoveryActivity.kt (Lokaler LAN Device Mapper)
│       ├── PortSentinelActivity.kt (Passiver Ködersensor)
│       ├── TitanActivity.kt (Device Owner Konsole)
│       ├── EvidenceScreen.kt (Revisionssicherer Journal-Viewer)
│       ├── DiagnosticsActivity.kt (Datenschutzkonformer Metrik-Export)
│       ├── SupportVgtActivity.kt (Versions- & Lizenzinformationen)
│       ├── SetupWizardActivity.kt (10-Schritte Onboarding Assistent)
│       └── VpnDisclosureActivity.kt (Google Play Consent Prominent Disclosure)
├── APPLICATION CONTROL PLANE (Kotlin Host Process: de.visiongaia.gedefense.mobile)
│   ├── Root Container & Lifecycle
│   │   ├── GeDefenseApplication.kt (Android Application Instanz)
│   │   ├── AppRuntime.kt (Dependency Injection & Async State Root)
│   │   ├── RuntimeState.kt (Unveränderliche Zustandsabbilder)
│   │   ├── UiSnapshot.kt (UI-gebundene ViewModels)
│   │   └── BoundedExecutors.kt (gedefense-control Single-Thread & Bounded Pools)
│   ├── Network Control & Android Integration
│   │   ├── GeDefenseVpnService.kt (Android VpnService Lifecycle & TUN Allokation)
│   │   ├── NativeGaiaNet.kt (ProcessBuilder Manager, IPC Sockets & Liveness Anchor)
│   │   ├── ConnectionOwner.kt (UID/Paket Attribution via ConnectivityManager)
│   │   └── PackageEgressGate.kt (Android UID/FlowGate Gate-Responder)
│   ├── State Stores & Repositories (Persistent & Encrypted)
│   │   ├── FirewallPolicyStore.kt (App- und IP-Regeln)
│   │   ├── WireGuardProfileStore.kt (Kryptografisch gewrappte Tunnelprofile)
│   │   ├── XdrEventStore.kt (Persistente XDR Incident Historie)
│   │   ├── BehaviorBaselineStore.kt (Gleitende EWMA Normalprofile)
│   │   ├── PackageBaselineStore.kt (Signer-Historie & Paket-Hashes)
│   │   ├── MalwareAnalysisStore.kt (Statische Befunde)
│   │   ├── AppApprovalStore.kt (Benutzerfreigaben)
│   │   ├── NetworkDiscoveryStore.kt (LAN Asset Baseline)
│   │   ├── PortSentinelStore.kt (Angreifer-IP Denylist)
│   │   ├── TitanPolicyStore.kt (Device Owner Richtlinien)
│   │   ├── ScannerStateCache.kt (Inkrementeller Scan-Cache)
│   │   ├── VpnDisclosureStore.kt (HMAC-signierte Zustimmung)
│   │   ├── ThreatIntelRepository.kt (Bedrohungsfeed-Verwaltung)
│   │   ├── AsnEvidenceRepository.kt (Lokaler O(log n) ASN Lookup)
│   │   ├── GeoCountryRepository.kt (Offline IP-zu-Land Mapping)
│   │   ├── PrivacyIntelligenceRepository.kt (Tracking & DoT/DoQ Registry)
│   │   └── TrafficUsageRepository.kt (Netzwerkzähler & Durchsatz)
│   ├── Scanning, Protection & Enterprise Policies
│   │   ├── InstallGuard.kt (Staged Fast-Verdict 1.75s / Deep-Scan 25s)
│   │   ├── InstallGuardPolicy.kt (Quarantäne-Richtlinien)
│   │   ├── AppRiskScanner.kt (Statische APK Analyse & Signer Verification)
│   │   └── StorageMalwareScanner.kt (Bounded Shared Storage Inspection)
```

- └─ ThreatTokenScanner.kt (Tokenbasierte Payload-Signaturen)
- └─ DeviceHardeningScanner.kt (OS-Einstellungsintegrität)
- └─ BehaviorEngine.kt (Verhaltensabgleich im Netzwerk)
- └─ IntegrityGuardian.kt (Selbst-Integritätsüberwachung)
- └─ PortSentinel.kt (Java NIO Decoy Listener)
- └─ NetworkDiscoveryScanner.kt (ARP, SSDP & mDNS Engine)
- └─ TitanPolicyManager.kt (DevicePolicyManager Bridge)
- └─ Background Job Services & Receivers
 - └─ ThreatIntelJobService.kt (Periodischer Feed-Download)
 - └─ IntegrityJobService.kt (6h Integritätsprüfung)
 - └─ ResilienceJobService.kt (2h Drift-Reconciliation)
 - └─ BootReceiver.kt (JobScheduler Wiederherstellung nach Reboot)
 - └─ PackageChangeReceiver.kt (PACKAGE_ADDED/REMOVED Event-Hook)
 - └─ TitanDeviceAdminReceiver.kt (Device Admin Receiver)
 - └─ TitanPackageOperationReceiver.kt (Privilegierte Paketverwaltung)
- └─ Resilience & Self-Healing
 - └─ ResilienceSupervisor.kt (Single-Flight Konsistenzabgleich)
 - └─ BoundedRecoveryCall.kt (Überlastungsfreie Reparatur-Executoren)
 - └─ RecoveryAttemptBudget.kt (Budgetierung automatischer Fixes)
- └─ DATA PLANE (Go Process: libgedefense_gaianet_v2.so)
 - └─ Process Boundary & IPC Framing
 - └─ main.go (Einstiegspunkt des Helpers)
 - └─ helper_main_process.go (Token-Authentifizierung, Protocol v5 Parser)
 - └─ helper_main_stub.go (Plattform-Stub für Non-Android Targets)
 - └─ bridge_android.go (Android Low-Level Bridge)
 - └─ Packet Engine & Flow Tracking
 - └─ engine.go (Zentraler TUN-Loop & Packet-Dispatcher)
 - └─ packet.go (IPv4/IPv6 Bounded Parser)
 - └─ build_packet.go (Synthetische IP/TCP/UDP Paketgenerierung)
 - └─ checksum.go (Optimierte Internet-Prüfsummenberechnung)
 - └─ reassembly.go (Bounded Fragment-Reassembly mit Overlap-Schutz)
 - └─ tcp_manager.go (Userspace TCP State Machine & Flow Tracking)
 - └─ tcp_flow_io.go (Non-blocking I/O mit Window Scaling)
 - └─ udp_manager.go (UDP/QUIC Flow Manager mit State Timeouts)
 - └─ flow_key.go (5-Tupel Identifikator)
 - └─ flow_id.go (Monotoner Flow Identifier)
 - └─ flow_limiter.go (Globaler & Per-Protocol Ratenbegrenzer)
 - └─ limits.go (Puffer-, Queue- und Timeout-Konstanten)
 - └─ power_state.go (Adaptive Timeout-Verkürzung im Akkubetrieb)
 - └─ tun_writer.go (Thread-safe Schreibpuffer für /dev/tun)
 - └─ telemetry.go (Binäre Event-Pipeline via anonyme Pipe)
 - └─ In-Engine Policy & Filter
 - └─ policy.go (GDTI-v2 Trie Matcher mit 9-Feed Bitmaske)
 - └─ privacy_policy.go (Telemetry Shield & Port 853 DoT/DoQ Filter)
 - └─ package_egress_gate.go (Pre-Egress UID Quarantäne-Gate)
 - └─ WireGuard L3 Transport Subsystem
 - └─ wireguard_transport.go (wireguard-go L3 Engine Integration)
 - └─ wireguard_tun.go (Virtueller In-Process TUN Adapter)
 - └─ wireguard_protected_bind.go (SCM_RIGHTS Socket-Handshake mit Android)
 - └─ wireguard_flow_tracker.go (Zustandsbehaftetes Ingress-Tracking)
 - └─ socket_binder_android.go (Plattform-spezifische Socket-Bindung)
 - └─ Vended Upstream WireGuard Engine (third_party/go/)
 - └─ wireguard-go 0.0.20250522 (Offizieller Go L3 Tunnel)
 - └─ golang.org/x/* (crypto, net, sys, term, text)
- └─ CRYPTOGRAPHIC CORE & VAULT ENGINE (:core & app/crypto)
 - └─ Secure Telemetry Vault (Android Host Adapter)
 - └─ SecureTelemetryVault.kt (12-Domain AEAD Facade)
 - └─ PersistentVaultKeys.kt (VC55 Update-Stabiler Hardware-PRF KEK & Wrapped Keys)
 - └─ HardwareDerivedVaultKeys.kt (Historischer v4 Hardware-HMAC Read-Adapter)
 - └─ BetaVaultMigrationPolicy.kt (Kryptografische Einweg-Migrationspfade)
 - └─ WrappedHotPathKeys.kt (Flüchtige Subkey-Kustodie für Evidence/XDR)
 - └─ AndroidSecrets.kt (KeyStore/StrongBox Key-Generator & Wrapper)
 - └─ AndroidKeystoreGate.kt (Deadlock-sichere Timeouts & Circuit Breaker)
 - └─ VaultStartupFailure.kt (Typisierte Krypto-Fehlerklassen)
 - └─ Pure JVM Cryptographic Primitives (:core)
 - └─ AeadVaultEnvelope.kt (VGT VLT01 Binär-Codec & AAD Binding)
 - └─ AuthenticatedSnapshotStore.kt (HMAC-SHA-256 Integritäts-Schale)
 - └─ BoundedSecretKeyCrypto.kt (Pure-JVM AEAD/HMAC Operationen)
 - └─ EvidenceLedger.kt (Revisionssichere HMAC-Chained Journal Engine)
 - └─ HybridArtifactSigner.kt (ECDSA P-256 + ML-DSA-87 PQC Detached Signer)
 - └─ Core Algorithmen & Indizes (:core)
 - └─ ThreatIndex.kt (Kompakter $O(\log n)$ IP-Präfix Radix-Trie)
 - └─ ThreatFeed.kt (Feed-Modelle & Bit-Autoritäten)
 - └─ ThreatIntelParser.kt (Deterministischer Streaming-Parser)
 - └─ ThreatCacheStore.kt (Cache-Verwaltung für Bedrohungs-Tries)
 - └─ IpPrefix.kt (IPv4/IPv6 CIDR-Arithmetik & RFC1918 Filter)
 - └─ RouteCompactor.kt (Kernel-optimierte Präfix-Zusammenfassung)
 - └─ PrivacyIntelligence.kt (Tracker-Katalog & Port 853 Regeln)
 - └─ AsnLiteIndex.kt (Kompakte binäre $O(\log n)$ ASN Indizes)
 - └─ MdnsDnsCodec.kt (Bounded Zero-Allocation DNS/mDNS Parser)
 - └─ NetworkRangePlanner.kt (LAN Scope Begrenzer auf /24)
 - └─ LanBehaviorEvaluator.kt (Statistischer LAN Drift Evaluator)
 - └─ PortSentinelClassifier.kt (Multi-Port Scan Korrelator)

```
| | ManagedCaCertificateValidator.kt (Strikte X.509 CA Validierung)
| | IntegrityBaseline.kt (Selbst-Integritäts-Struktur)
| | FlowTable.kt (Core Flow-Definitionen)
| | RecoveryAttemptBudget.kt (Bounded Recovery Mathematische Schranken)
|
| VERIFICATION & RELEASE GATES (tools/)
| | Security & Cryptography Audits
| | | security-audit.sh (Nativer Gesamtsicherheits-Harness, 1.252 LOC)
| | | secure-vault-audit.py (12-Domain AEAD & Nonce Audit)
| | | privacy-shield-audit.py (Telemetry Shield & Port 853 DoT/DoQ Gate)
| | | wireguard-audit.py (34 Invarianten des WireGuard Subsystems)
| | | install-guard-audit.py (Fast-Verdict 1.75s & FlowGate Gate)
| | | resilience-supervisor-audit.py (Drift & Selbstreparatur Audit)
| | | full-flow-availability-audit.py (Full Flow Hot-Path & Fallback Audit)
| | Platform, Concurrency & ANR Audits
| | | startup-anr-audit.py (Main Looper I/O & Bootstrap-Latenz)
| | | main-thread-io-audit.py (Lifecycle Callbacks AST-Audit)
| | | runtime-bootstrap-audit.py (Zero-Eager Preferences & SharedPreferences Audit)
| | | jvm-class-init-probe.sh (Android Class-Initializer Crash Probe)
| | Packaging, Supply Chain & Toolchains
| | | verify-source-manifest.py (SHA-256 Verifikation aller 2.678 Dateien)
| | | generate-source-manifest.py (Manifest-Generator)
| | | artifact-audit.sh (APK/AAB Signatur- und Alignment-Audit)
| | | network-egress-audit.py (Strikte Beschränkung auf 12 Egress Hosts)
| | | asn-evidence-audit.py (PDDL ASN Quell- und Indexverifikation)
| | | onboarding-audit.py (10-Schritte Wizard Vollständigkeit)
| | | i18n-audit.py (1.055 Übersetzungs-Keys über 4 Sprachen)
| | | lint-zero-audit.py (Zero-Warning Release Lint Gate)
| | | build-go-netstack.ps1 (Reproduzierbarer NDK r27c Go Cross-Compiler)
```

2. DATEIEN EINER ARCHITEKTUR ZUORDNEN

Die folgende Aufstellung ordnet sämtliche Quellcodedateien des Projekts exakt ihren logischen Architekturpositionen zu.

A. Presentation & UI Layer

- **Shell & Navigation:**

- `app/src/main/java/de/visiongaia/gedefense/mobile/MainActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/StartupActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/BottomNavBar.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/RuntimeActivityEntry.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/VgtWindowInsets.kt`

- **VGT Cyber Dark Styling & Design Tokens:**

- `app/src/main/java/de/visiongaia/gedefense/mobile/GeDefenseUi.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/VgtUiComponents.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/VgtUiPerformance.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/TitanVisualMode.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/VgtActionTile.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/VgtAppIconCache.kt`

- **Ambient Reactive Canvas Views:**

- `app/src/main/java/de/visiongaia/gedefense/mobile/CyberBackgroundView.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/ShieldPulseView.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/StartupCoreView.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/TitanCoreView.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/ScannerRadarView.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/TrafficWorldMapView.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/VgtIconView.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/VgtProgressView.kt`

- **Primary Screens & Activities:**

- `app/src/main/java/de/visiongaia/gedefense/mobile/DashboardScreen.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/ThreatScreen.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/ProtectionHubScreen.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/AnalysisHubScreen.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/SystemHubScreen.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/FirewallActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/WireGuardActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/PrivacyActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/ScannerActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/XdrActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/HardeningActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/BehaviorActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/NetworkDiscoveryActivity.kt`

- `app/src/main/java/de/visiongaia/gedefense/mobile/PortSentinelActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/TitanActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/EvidenceScreen.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/DiagnosticsActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/SupportVgtActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/SetupWizardActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/SetupWizardPageRenderer.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/VpnDisclosureActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/TitanGetProvisioningModeActivity.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/TitanPolicyComplianceActivity.kt`

B. Application Control Plane

• Lifecycle, State & Container:

- `app/src/main/java/de/visiongaia/gedefense/mobile/GeDefenseApplication.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/AppRuntime.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/RuntimeState.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/UiSnapshot.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/BoundedExecutors.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/BoundedSerialScheduler.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/BoundedAndroidCall.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/ProtectionMetrics.kt`

• Network Control & TUN Integration:

- `app/src/main/java/de/visiongaia/gedefense/mobile/GeDefenseVpnService.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/NativeGaiaNet.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/ConnectionOwner.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/PackageEgressGate.kt`

• Stores & Persistenz (Encrypted / Authenticated):

- `app/src/main/java/de/visiongaia/gedefense/mobile/FirewallPolicyStore.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/WireGuardProfileStore.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/XdrEventStore.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/BehaviorBaselineStore.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/PackageBaselineStore.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/MalwareAnalysisStore.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/AppApprovalStore.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/NetworkDiscoveryStore.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/PortSentinelStore.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/TitanPolicyStore.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/ScannerStateCache.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/VpnDisclosureStore.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/RuntimeEvidenceStore.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/ThreatIntelRepository.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/AsnEvidenceRepository.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/GeoCountryRepository.kt`

- `app/src/main/java/de/visiongaia/gedefense/mobile/PrivacyIntelligenceRepository.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/TrafficUsageRepository.kt`

• Scanning, Protection & Inspection Engines:

- `app/src/main/java/de/visiongaia/gedefense/mobile/InstallGuard.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/InstallGuardPolicy.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/BoundedInstallScanCall.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/AppRiskScanner.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/StorageMalwareScanner.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/ThreatTokenScanner.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/DeviceHardeningScanner.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/BehaviorEngine.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/IntegrityGuardian.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/PortSentinel.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/NetworkDiscoveryScanner.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/TitanPolicyManager.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/FeedDownloader.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/WireGuardConfigParser.kt`

• Background Jobs & System Receivers:

- `app/src/main/java/de/visiongaia/gedefense/mobile/ThreatIntelJobService.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/IntegrityJobService.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/ResilienceJobService.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/BootReceiver.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/PackageChangeReceiver.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/TitanDeviceAdminReceiver.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/TitanPackageOperationReceiver.kt`

• Resilience & Selbstreparatur:

- `app/src/main/java/de/visiongaia/gedefense/mobile/ResilienceSupervisor.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/BoundedRecoveryCall.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/SecureFiles.kt`

C. Data Plane (Go Netstack Subsystem)

• Engine Core & Packet Processing:

- `netstack/main.go`
- `netstack/engine.go`
- `netstack/packet.go`
- `netstack/build_packet.go`
- `netstack/checksum.go`
- `netstack/reassembly.go`
- `netstack/limits.go`
- `netstack/power_state.go`
- `netstack/tun_writer.go`
- `netstack/telemetry.go`

- **TCP / UDP State Tracking & Managers:**

- `netstack/tcp_manager.go`
- `netstack/tcp_flow_io.go`
- `netstack/udp_manager.go`
- `netstack/dns.go`
- `netstack/flow_key.go`
- `netstack/flow_id.go`
- `netstack/flow_limiter.go`

- **In-Engine Filters & Policy Evaluators:**

- `netstack/policy.go`
- `netstack/privacy_policy.go`
- `netstack/package_egress_gate.go`

- **IPC, Socket Handshake & Process Integration:**

- `netstack/helper_main_process.go`
- `netstack/helper_main_stub.go`
- `netstack/bridge_android.go`
- `netstack/socket_binder_android.go`
- `netstack/socket_binder_process.go`
- `netstack/socket_binder_other.go`
- `netstack/socket_binding.go`

- **WireGuard L3 Egress Subsystem:**

- `netstack/wireguard_transport.go`
- `netstack/wireguard_tun.go`
- `netstack/wireguard_protected_bind.go`
- `netstack/wireguard_flow_tracker.go`

- **Vendored WireGuard Upstream:**

- `third_party/go/wireguard-go/` (Upstream wireguard-go v0.0.20250522)
- `third_party/go/x-crypto/` (golang.org/x/crypto v0.37.0)
- `third_party/go/x-net/` (golang.org/x/net v0.39.0)
- `third_party/go/x-sys/` (golang.org/x/sys v0.32.0)
- `third_party/go/x-term/` (golang.org/x/term v0.31.0)
- `third_party/go/x-text/` (golang.org/x/text v0.24.0)

D. Cryptographic Core & Data Structures (:core & app/crypto)

- **Host Krypto-Adapter:**

- `app/src/main/java/de/visiongaia/gedefense/mobile/SecureTelemetryVault.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/PersistentVaultKeys.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/HardwareDerivedVaultKeys.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/WrappedHotPathKeys.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/BetaVaultMigrationPolicy.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/AndroidSecrets.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/AndroidKeystoreGate.kt`

- `app/src/main/java/de/visiongaia/gedefense/mobile/VaultStartupFailure.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/AndroidEvidence.kt`
- `app/src/main/java/de/visiongaia/gedefense/mobile/HybridArtifactSigner.kt`

• **Pure-JVM Cryptographic Primitives (:core):**

- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/AeadVaultEnvelope.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/AuthenticatedSnapshotStore.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/BoundedSecretKeyCrypto.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/EvidenceLedger.kt`

• **Algorithmen, Tries & Datenmodelle (:core):**

- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/ThreatIndex.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/ThreatFeed.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/ThreatIntelParser.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/ThreatCacheStore.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/IpPrefix.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/RouteCompactor.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/PrivacyIntelligence.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/AsnLiteIndex.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/MdnsDnsCodec.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/NetworkRangePlanner.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/LanBehaviorEvaluator.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/PortSentinelClassifier.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/ManagedCaCertificateValidator.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/IntegrityBaseline.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/FlowTable.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/TelemetryLearningPolicy.kt`
- `core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/RecoveryAttemptBudget.kt`

E. Verification & Security Tools (tools/)

- `tools/security-audit.sh`
- `tools/secure-vault-audit.py`
- `tools/privacy-shield-audit.py`
- `tools/wireguard-audit.py`
- `tools/install-guard-audit.py`
- `tools/install-guard-policy-check.sh`
- `tools/resilience-supervisor-audit.py`
- `tools/full-flow-availability-audit.py`
- `tools/startup-anr-audit.py`
- `tools/main-thread-io-audit.py`
- `tools/runtime-bootstrap-audit.py`
- `tools/jvm-class-init-probe.sh`
- `tools/verify-source-manifest.py`
- `tools/generate-source-manifest.py`
- `tools/artifact-audit.sh`

- `tools/network-egress-audit.py`
 - `tools/asn-evidence-audit.py`
 - `tools/onboarding-audit.py`
 - `tools/i18n-audit.py`
 - `tools/lint-zero-audit.py`
 - `tools/core-check.sh`
 - `tools/wireguard-parser-check.sh`
 - `tools/wireguard-upstream-check.sh`
 - `tools/build-go-netstack.ps1`
 - `tools/package-source.py`
 - `tools/windows-build-alpha.ps1`
 - `tools/windows-build-beta.ps1`
-

3. MODUL-DOKUMENTATION

Modul: Host & Application Runtime

Zweck

Das Modul **Host & Application Runtime** bildet das zentrale administrative Fundament der Android-Applikation. Es verwaltet die Initialisierung des Android-Anwendungsprozesses, instanziiert alle langlebigen Subsysteme, hält unveränderliche Zustandsabbilder (**RuntimeState**, **UiSnapshot**) und stellt die asynchrone, nicht-blockierende Ausführung aller Systemmutationen sicher.

Fähigkeiten

- **Asynchroner Prozess-Bootstrap:** Verlagert KeyStore-Aufrufe und Dateisystem-I/O aus **Application.onCreate()** auf einen dedizierten Initialisierungs-Thread (**StartupActivity.kt**, **AppRuntime.kt**).
- **Zustands-Container (Dependency Root):** Verwaltet Singletons und State-Snapshots ohne komplexe externe DI-Frameworks.
- **Serialisierte Mutation:** Führt alle VPN- und Krypto-Zustandswechsel strikt sequenziell auf dem **gedefense-control** Executor aus (**BoundedExecutors.kt**).
- **Plattform-Lebenszyklus:** Reagiert auf Systemereignisse (Boot, Package Changes, Theme Switches).

Zugehörige Dateien

```
Frontend:
- app/src/main/java/de/visiongaia/gedefense/mobile/MainActivity.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/StartupActivity.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/BottomNavBar.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/RuntimeActivityEntry.kt

Backend:
- app/src/main/java/de/visiongaia/gedefense/mobile/GeDefenseApplication.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/AppRuntime.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/RuntimeState.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/UiSnapshot.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/BoundedExecutors.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/BoundedSerialScheduler.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/BoundedAndroidCall.kt

Styles:
- app/src/main/java/de/visiongaia/gedefense/mobile/GeDefenseUi.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/VgtWindowInsets.kt

Konfiguration:
- app/src/main/AndroidManifest.xml
- app/build.gradle.kts

Tests:
- tools/runtime-bootstrap-audit.py
- tools/startup-anr-audit.py
- tools/main-thread-io-audit.py
```

Dashboard-Integration

- **Dashboard-Seiten:** **MainActivity** beherbergt die 5 primären Navigationshubs (**DashboardScreen**, **ThreatScreen**, **ProtectionHubScreen**, **AnalysisHubScreen**, **SystemHubScreen**).
- **Verwendete Komponenten:** **BottomNavBar**, **CyberBackgroundView**, **TitanManagedBanner**.
- **Daten:** Konsumiert **UiSnapshot** mit aktuellen Verbindungsstatus-, Bedrohungs- und Zählerwerten.
- **Aktionen:** Start/Stop der Schutzdienste, Hub-Wechsel, Auslösen asynchroner Refreshes.

Abhängigkeiten

```
Host & Application Runtime
├─ benötigt Android KeyStore Hardware (via AndroidSecrets.kt)
├─ startet GeDefenseVpnService (via Intent)
├─ instanziiert SecureTelemetryVault
└─ steuert BoundedExecutors (gedefense-control)
```

Wird verwendet von

- Wird von sämtlichen Activities, Services und Receivers des Host-Prozesses als zentraler Service-Locator genutzt.

Modul: Network & VPN Core / Transport Plane

Zweck

Das Modul **Network & VPN Core** besitzt die alleinige administrative Verantwortung über das Android-eigene **VpnService**-Framework. Es allokiert das virtuelle Linux-TUN-Interface (`/dev/tun`), verwaltet Routing-Tabellen, überwacht Verbindungswechsel (Handover) und bindet den isolierten Go-Netstack über Filedeskriptor-Übergabe an.

Fähigkeiten

- **Single VpnService Authority:** Verwaltet Full Flow (`0.0.0.0/0`, `::/0`) und Selective Shield Routen.
- **Process Supervision & Liveness Anchor:** Hält den TUN-Filedeskriptor (`Session.tunAnchor`) auch bei einem Absturz des Go-Prozesses offen, um Datenlecks zu verhindern.
- **Underlay Network Binding:** Überwacht **ConnectivityManager**-Callbacks und bindet Upstream-Sockets atomar an das aktive physische Netzwerk.
- **Per-Socket Protection:** Führt `VpnService.protect(fd)` für Upstream-Sockets aus, die via IPC empfangen werden.

Zugehörige Dateien

```
Frontend / Control UI:
- app/src/main/java/de/visiongaia/gedefense/mobile/ProtectionHubScreen.kt

Backend & Core:
- app/src/main/java/de/visiongaia/gedefense/mobile/GeDefenseVpnService.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/NativeGaiaNet.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/ConnectionOwner.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/TrafficUsageRepository.kt
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/RouteCompactor.kt
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/IpPrefix.kt

IPC / Native:
- netstack/helper_main_process.go
- netstack/socket_binder_android.go

Tests:
- tools/full-flow-availability-audit.py
- tools/network-egress-audit.py
```

Dashboard-Integration

- **Dashboard-Seiten:** Visualisiert im Cockpit (`DashboardScreen`) als Schild-Status (Hero-Pulse) sowie in `ProtectionHubScreen`.
- **Daten:** Aktive Verbindungsdauer, Upstream-Netzwerktyp (Wi-Fi, Mobilfunk), übertragene Bytes.
- **Aktionen:** Aktivieren/Deaktivieren des VPNs, Umschalten zwischen Full Flow und Selective Mode.

Abhängigkeiten

```
Network & VPN Core / Transport Plane
├─ benötigt Android VpnService Plattform-API
├─ steuert NativeGaiaNet (Kindprozess-Verwaltung)
├─ liest VpnDisclosureStore (HMAC-Zustimmung)
└─ meldet Traffic an TrafficUsageRepository
```

Wird verwendet von

- `MainActivity`, `SetupWizardActivity`, `TitanActivity`, Android OS (via `BIND_VPN_SERVICE`).

Modul: GaiaNet V2 Netstack Engine (Go)

Zweck

`GaiaNet V2` ist der speichersichere, in Go geschriebene High-Performance-Netzwerk-Stack. Er läuft als isolierter Prozess (`libgedefense_gaianet_v2.so`), liest rohe IP-Pakete aus dem TUN-Interface, verarbeitet TCP/UDP-Zustände, wendet lokale Bedrohungsrichtlinien an und leitet den Datenverkehr an Upstream-Sockets weiter.

Fähigkeiten

- **Userspace IP Reassembly:** Bounded Defragmentierung von IPv4/IPv6-Paketen mit strikter Overlap-Ablehnung.
- **TCP Proxy & State Engine:** Synthetischer Downstream-TCP-Stack mit Window Scaling und Flow-Tracking.
- **UDP / QUIC Forwarding:** Bounded UDP-Flow-Management mit adaptiven Timeouts zur Akkuschonung (`power_state.go`).
- **Flow Limiting & Bounds:** Globales Limit von 32 MiB unbestätigter Pufferdaten und 1.024 gleichzeitigen UDP-Flows.
- **In-Engine Telemetrie:** Sendet sicherheitskritische Ereignisse über eine anonyme Unix-Pipe an den Host.

Zugehörige Dateien

```
Go Netstack Core:
- netstack/main.go
- netstack/engine.go
- netstack/packet.go
- netstack/build_packet.go
- netstack/checksum.go
- netstack/reassembly.go
- netstack/tcp_manager.go
- netstack/tcp_flow_io.go
- netstack/udp_manager.go
- netstack/flow_key.go
- netstack/flow_id.go
- netstack/flow_limiter.go
- netstack/limits.go
- netstack/power_state.go
- netstack/tun_writer.go
- netstack/telemetry.go

IPC & Android Bridge:
- netstack/helper_main_process.go
- netstack/bridge_android.go
- netstack/socket_binding.go

Binaries:
- app/src/main/jniLibs/arm64-v8a/libgedefense_gaianet_v2.so
- app/src/main/jniLibs/x86_64/libgedefense_gaianet_v2.so

Tests:
- netstack/packet_test.go
- netstack/reassembly_test.go
- netstack/helper_process_test.go
- netstack/fuzz_test.go
- netstack/transport_limits_test.go
```

Dashboard-Integration

- **Dashboard-Seiten:** Keine direkte UI. Status und Durchsatz werden aggregiert über die Telemetrie-Pipe an `DashboardScreen` gemeldet.
- **Daten:** Zahl aktiver Flows, geblockte Pakete, Latenzen.

Abhängigkeiten

```
GaiaNet V2 Netstack Engine
├─ empfängt TUN- und Pipe-FDs via SCM_RIGHTS (von NativeGaiaNet.kt)
├─ liest kompilierte GDTI-v2 Policy (aus policy.go)
└─ nutzt Go Runtime 1.23.x (Offline vendored)
```

Wird verwendet von

- Wird exklusiv von `NativeGaiaNet.kt` als Kindprozess gestartet und überwacht.

Modul: WireGuard L3 Egress Subsystem

Zweck

Das `WireGuard L3 Egress Subsystem` erweitert die Data Plane um verschlüsseltes Layer-3-Tunneling. Es bindet die offizielle `wireguard-go` Engine ein, wickelt den Noise IKpsk2 Handshake ab und garantiert über punktuelle Socket-Freistellungen, dass kein globaler App-Bypass im System entsteht.

Fähigkeiten

- **Single-VPN WireGuard:** Betreibt WireGuard innerhalb des bestehenden GeDefense `VpnService` ohne Android-Tunnel-Konflikt.
- **SCM_RIGHTS Socket-Handshake:** Sendet WireGuard-Transport-UDP-Sockets an den Android-Host zur Registrierung via `VpnService.protect()`.
- **Zustandsbehafteter Peer-Ingress:** Filtert unaufgeforderten eingehenden Traffic vom WireGuard-Peer (`wireguard_flow_tracker.go`).
- **WireGuard Strict Mode:** Erzwingt Android Always-On Lockdown; verhindert Fallback auf unverschlüsselten Direct-Modus bei Verbindungsausfall.
- **Wipeable Memory:** Verwendet `SensitiveByteBuilder` zur spurlosen Nullung privater Schlüssel im RAM.

Zugehörige Dateien

```
Frontend:
- app/src/main/java/de/visiongaia/gedefense/mobile/WireGuardActivity.kt

Host Backend & Parser:
- app/src/main/java/de/visiongaia/gedefense/mobile/WireGuardConfigParser.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/WireGuardProfileStore.kt

Netstack Go Engine:
- netstack/wireguard_transport.go
- netstack/wireguard_tun.go
- netstack/wireguard_protected_bind.go
- netstack/wireguard_flow_tracker.go

Vendored Upstream:
- third_party/go/wireguard-go/ (v0.0.20250522)

Tests:
- tools/wireguard-audit.py
- tools/wireguard-parser-check.sh
- tools/wireguard-upstream-check.sh
```

```
- netstack/wireguard_transport_test.go
- netstack/wireguard_icmp_test.go
```

Dashboard-Integration

- **Dashboard-Seiten:** `WireGuardActivity` (erreichbar über `ProtectionHubScreen`).
- **Komponenten:** Profil-Editor, QR-Code/File-Importeur, Endpunkt-Status-Badge, MTU-Slider (`1280..1420`).
- **Aktionen:** Tunnel aktivieren/deaktivieren, Profil importieren/löschen, Strict-Mode toggeln.

Abhängigkeiten

```
WireGuard L3 Egress Subsystem
├─ nutzt vendortes wireguard-go 0.0.20250522
├─ verlangt VpnService.protect() (von GeDefenseVpnService.kt)
├─ speichert verschlüsselte Profile in WireGuardProfileStore.kt
└─ enforce Android Lockdown API im Strict-Modus
```

Wird verwendet von

- `ProtectionHubScreen`, `GeDefenseVpnService`, `GaiaNet V2 Engine`.

Modul: Threat Intelligence & Policy Engine

Zweck

Die `Threat Intelligence & Policy Engine` ist das Erkennungsherz von GeDefense. Sie verwaltet neun vertrauenswürdige Sicherheitsfeeds, kompiliert IP-Präfixe in speichereffiziente $O(\log n)$ Radix-Tries und erzwingt Blockierungsentscheidungen deterministisch und ohne Cloud-Abfragen.

Fähigkeiten

- **Deterministische Bitmask-Autorität:** Unterscheidet strikt zwischen `ROUTE_BLOCK` (Feodo, Spamhaus, FireHOL), `CORRELATE_ONLY` (CINS, Blocklist.de, Emerging Threats, IPsum) und `ANNOTATE_ONLY` (Tor Exits).
- **Kompakte Radix-Trie Indizierung:** Kompiliert Millionen IP-Regeln in binäre `GDTI-v2` Strukturen für Nanosekunden-Lookups.
- **Atomare Feed-Aktualisierung:** Transaktionaler Tausch von Speicher-Snapshots mit automatischem Rollback bei Prüfsummenfehlern.
- **Offline ASN & Geo Enrichment:** Lokale Zuordnung verdächtiger IPs zu autonomen Systemen und Ländern via `AsnLiteIndex.kt` und `GeoCountryRepository.kt`.

Zugehörige Dateien

```
Frontend:
- app/src/main/java/de/visiongaia/gedefense/mobile/ThreatScreen.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/TrafficWorldMapView.kt

Host Backend & Jobs:
- app/src/main/java/de/visiongaia/gedefense/mobile/ThreatIntelRepository.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/ThreatIntelJobService.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/FeedDownloader.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/AsnEvidenceRepository.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/GeoCountryRepository.kt

Core Algorithmen (:core):
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/ThreatIndex.kt
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/ThreatFeed.kt
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/ThreatIntelParser.kt
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/ThreatCacheStore.kt
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/AsnLiteIndex.kt

Data Plane Engine:
```

```
- netstack/policy.go

Tests:
- tools/asn-evidence-audit.py
- tools/network-egress-audit.py
```

Dashboard-Integration

- **Dashboard-Seiten:** `ThreatScreen` (Haupt-Tab 2 in `MainActivity`).
- **Komponenten:** Feed-Status-Liste, globale Bedrohungszähler, Vektorielle Weltkarte (`TrafficWorldMapView`), "Sync Now"-Aktionskachel.
- **Aktionen:** Manueller Download-Trigger, Feed-Details einsehen, ASN-Herkunft prüfen.

Abhängigkeiten

```
Threat Intelligence & Policy Engine
├─ lädt externe Feeds via FeedDownloader.kt (HTTPS GET)
├─ speichert Tries via ThreatCacheStore.kt (HMAC-authentifiziert)
├─ übergibt binäre Policy via SCM_RIGHTS an netstack/policy.go
└─ speist geblockte Verbindungen in XdrEngine.kt und EvidenceLedger.kt ein
```

Wird verwendet von

- `GaiaNet V2 Engine`, `XdrEngine`, `DashboardScreen`, `ThreatScreen`.

Modul: Telemetry Shield & Privacy Engine

Zweck

Das Modul `Telemetry Shield` schützt die Privatsphäre des Nutzers vor Tracking, Werbenetzwerken und unerwünschter Gerätediagnose. Es analysiert sichtbaren DNS-Verkehr und implementiert eine fortschrittliche Non-MITM-Blockierung von verschlüsseltem DNS (DoT/DoQ auf Port 853).

Fähigkeiten

- **4-Stufen Privacy Profile:** OFF, CONSERVATIVE, BALANCED (Default), STRICT.
- **Synthetisches NXDOMAIN:** Generiert lokale Antworten für blockierte Tracker-Domains ohne Netzaustritt.
- **Port 853 DoT/DoQ Blocking:** Blockiert im Strict-Modus unverschlüsselt sichtbare Port 853 Pakete vor Egress.
- **Zero TLS Interception:** Bricht niemals HTTPS/TLS-Zertifikate auf; schützt Privatsphäre ohne MITM-Sicherheitsrisiken.

Zugehörige Dateien

```
Frontend:
- app/src/main/java/de/visiongaia/gedefense/mobile/PrivacyActivity.kt

Host Backend & Repository:
- app/src/main/java/de/visiongaia/gedefense/mobile/PrivacyIntelligenceRepository.kt

Core Logik (:core):
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/PrivacyIntelligence.kt

Netstack Filter:
- netstack/privacy_policy.go
- netstack/dns.go

Tests:
- tools/privacy-shield-audit.py
- netstack/privacy_policy_test.go
```

Dashboard-Integration

- **Dashboard-Seiten:** `PrivacyActivity` (erreichbar über `ProtectionHubScreen`).
- **Komponenten:** Profil-Radio-Selector, Tracker-Statistikkarten, DoT/DoQ Status-Indikator.
- **Aktionen:** Profilauswahl, Ausnahmeregeln konfigurieren.

Abhängigkeiten

```
Telemetry Shield & Privacy Engine
├─ liest Regeln aus PrivacyIntelligenceRepository.kt
├─ übergibt Richtlinie via SCM_RIGHTS an netstack/privacy_policy.go
└─ synchronisiert Richtlinienänderungen mit GeDefenseVpnService.kt
```

Wird verwendet von

- `ProtectionHubScreen`, `GeDefenseVpnService`, `GaiaNet V2 Netstack`.

Modul: InstallGuard & Scanner Plane

Zweck

`InstallGuard` schützt das Endgerät vor bösartigen Neuinstallationen und unbefugten App-Modifikationen. Das Modul kombiniert einen zweistufigen Installations-Scanner mit einem aktiven Kernel-Pre-Egress-Gate (`FlowGate`), das neuen Apps den Netzzugriff verwehrt, bis ihre Vertrauenswürdigkeit bewiesen ist.

Fähigkeiten

- **Pre-Egress FlowGate (Protokoll v5):** Blockiert Netzwerk-Flows frisch installierter Apps unmittelbar vor Netzaustritt.
- **Zweistufige Verdict Pipeline:**
 - *Fast Verdict:* 1.750 ms Timeout auf 4-Worker Pool für Signer-Prüfung bekannter Updates.
 - *Deep Scan:* 25 s Timeout auf 2-Worker Pool für statische APK-Dekomprimierung und Heuristik.
- **Signer-Drift-Erkennung:** Erkennt Zertifikatswechsel zwischen App-Updates und stuft diese als Sicherheitsvorfall ein.
- **Storage Scanner:** Bounded Inspektion von externen Speichern auf schädliche APKs und Binaries.

Zugehörige Dateien

```
Frontend:
- app/src/main/java/de/visiongaia/gedefense/mobile/ScannerActivity.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/ScannerRadarView.kt

Host Backend & Scanner:
- app/src/main/java/de/visiongaia/gedefense/mobile/InstallGuard.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/InstallGuardPolicy.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/BoundedInstallScanCall.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/AppRiskScanner.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/StorageMalwareScanner.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/ThreatTokenScanner.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/ScannerStateCache.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/PackageEgressGate.kt

Netstack Gate:
- netstack/package_egress_gate.go

Tests:
- tools/install-guard-audit.py
- tools/install-guard-policy-check.sh
- netstack/package_egress_gate_test.go
```

Dashboard-Integration

- **Dashboard-Seiten:** `ScannerActivity` (erreichbar über `ProtectionHubScreen`) sowie Vorschau-Kachel in `DashboardScreen`.
- **Komponenten:** `ScannerRadarView`, Quarantäne-App-Liste, Risikobewertungs-Pill, "Scan Now"-Button.
- **Aktionen:** Manueller Virensan, App aus Quarantäne freigeben, App-Details prüfen.

Abhängigkeiten

```
InstallGuard & Scanner Plane
├─ horcht auf PackageChangeReceiver.kt (PACKAGE_ADDED)
├─ steuert netstack/package_egress_gate.go via Unix IPC
├─ persistiert Befunde in ScannerStateCache.kt (AES-256-GCM)
└─ eskaliert Infektionen an XdrEngine.kt und TitanPolicyManager.kt
```

Wird verwendet von

- `PackageChangeReceiver`, `DashboardScreen`, `ProtectionHubScreen`, `GeDefenseVpnService`.

Modul: Secure Telemetry Vault & Crypto Core

Zweck

Das Modul `Secure Telemetry Vault` stellt das kryptografische Rückgrat der Anwendung dar. Es isoliert zwölf sensitive Speicherdomänen auf Basis von AES-256-GCM, deriviert Schlüssel über einen update-stabilen Hardware-HMAC-Root und verhindert Offline-Datenexfiltration sowie Cross-Domain-Transplantationen.

Fähigkeiten

- **12 Getrennte Verschlüsselungsdomänen:** XDR, Verhalten, Paket-Baseline, Malware, Freigaben, Firewall, Discovery, Port Sentinel, TITAN, Scanner Storage, Scanner Apps, Evidence.
- **Update-Stabile Schlüsselkustodie (VC55):** Hardware-HMAC Root Key (`root-prf.v1`) entkoppelt Schlüssel von APK-Hashes und Versionsnummern.
- **VGTVL01 AEAD Envelope:** Authenticated Data (AAD) bindet Ciphertexte an Domäne, Schema, Format und Längen.
- **Zero-Allocation Memory Wiping:** Konsequente Bereinigung aller sensiblen Schlüsselbytes im flüchtigen Speicher.
- **Deadlock-Schutz:** `AndroidKeystoreGate` sichert alle Hardware-Keystore-Aufrufe mit harten Timeouts ab.

Zugehörige Dateien

```
Host Krypto-Adapter:
- app/src/main/java/de/visiongaia/gedefense/mobile/SecureTelemetryVault.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/PersistentVaultKeys.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/HardwareDerivedVaultKeys.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/WrappedHotPathKeys.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/BetaVaultMigrationPolicy.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/AndroidSecrets.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/AndroidKeystoreGate.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/VaultStartupFailure.kt

Pure JVM Primitives (:core):
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/AeadVaultEnvelope.kt
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/AuthenticatedSnapshotStore.kt
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/BoundedSecretKeyCrypto.kt

Tests:
- tools/secure-vault-audit.py
- tools/jvm-class-init-probe.sh
```

Dashboard-Integration

- **Dashboard-Seiten:** Keine direkte Seite; schützt transparent alle Hintergrund- und Snapshot-Speicher.
- **Daten:** Meldet Initialisierungs- und Kustodie-Status (z. B. StrongBox vs. TEE) an `SystemHubScreen`.

Abhängigkeiten

```
Secure Telemetry Vault & Crypto Core
├─ benötigt AndroidKeyStore Hardware / StrongBox
├─ nutzt HKDF-SHA-256 (RFC 5869) zur Subkey-Expansion
└─ speichert verschlüsselte Envelopes in noBackupFilesDir
```

Wird verwendet von

- Wird von ausnahmslos jedem persistenten Speicher- und Snapshot-Modul im Host-Prozess verwendet.

Modul: XDR / EDR Correlation & Incident Plane

Zweck

Das Modul `XDR / EDR Correlation & Incident Plane` ist die forensische Analysezentrale von GeDefense. Es empfängt Sicherheitsereignisse aus dem Netzwerk-Tunnel, dem App-Scanner, der Integritätsüberwachung und der Verhaltensanalyse, korreliert diese anhand von App-Identitäten und generiert strukturierte, nachvollziehbare Sicherheitsvorfälle (Incidents).

Fähigkeiten

- **Multidimensionale Korrelation:** Verknüpft geblockte C2-Verbindungen, Signer-Drift, gefährliche Berechtigungen und unübliche LAN-Portscans zu einer konsolidierten Vorfallbewertung.
- **Transparente Scoring-Engine:** UI und Backend verwenden exakt dieselbe mathematische Bewertungsfunktion (`XdrEngine.kt`); die dargestellten Risikowerte stimmen bitgenau mit der Entscheidung überein.
- **Aktive Quarantäne-Steuerung:** Bindet bösartige Apps an die lokale Firewall-Sperrliste oder übergibt sie an TITAN zur App-Suspendierung.
- **Revisionssichere Spiegelung:** Jeder generierte Vorfall wird atomar in das `EvidenceLedger` gespiegelt.

Zugehörige Dateien

```
Frontend:
- app/src/main/java/de/visiongaia/gedefense/mobile/XdrActivity.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/AnalysisHubScreen.kt

Host Backend & Stores:
- app/src/main/java/de/visiongaia/gedefense/mobile/XdrEngine.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/XdrEventStore.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/XdrModels.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/EventDedupe.kt

Tests:
- tools/full-flow-availability-audit.py
```

Dashboard-Integration

- **Dashboard-Seiten:** `XdrActivity` (erreichbar über `AnalysisHubScreen`).
- **Komponenten:** Vorfalls-Timeline, Risikoscore-Tacho, betroffene App-Badges, Detail-Akkordeon mit Rohdaten.
- **Aktionen:** Vorfall analysieren, App ins Netzwerk-Lockdown schicken, Fehlalarm markieren.

Abhängigkeiten

```
XDR / EDR Correlation & Incident Plane
├─ empfängt Events von GaiaNet V2 (Telemetrie-Pipe)
├─ empfängt Signale von AppRiskScanner.kt und BehaviorEngine.kt
├─ speichert Vorfälle in XdrEventStore.kt (AES-256-GCM Domäne 1)
└─ schreibt Nachweise in EvidenceLedger.kt
```

Wird verwendet von

- `AnalysisHubScreen`, `DashboardScreen`, `TitanPolicyManager`.

Modul: Evidence Ledger & Artifact Signing

Zweck

Das Modul `Evidence Ledger & Artifact Signing` stellt sicher, dass sämtliche Sicherheitsereignisse gerichtsfest, manipulationssicher und kryptografisch signiert protokolliert werden. Es implementiert ein fortlaufend HMAC-verkettetes Journal und erzeugt beim Export hybrid signierte Artefakte (klassisch via ECDSA und post-quantum via ML-DSA-87).

Fähigkeiten

- **HMAC-Chained Journal (v3):** Jeder Record enthält den kryptografischen Hash des Vorgängers; Manipulationen brechen die Integritätskette.
- **Envelope-Verschlüsselung der Einträge:** Jeder Eintrag wird einzeln mit dem Vault-Schlüssel der Domäne 12 verschlüsselt.
- **Detached PQC Signatures:** Erzeugt `.sig.json`-Dateien mit ECDSA P-256 (StrongBox) und optionalem ML-DSA-87 (FIPS 204) auf Android 17+ KeyMint 5 Geräten.
- **Fail-Closed Integrität:** Schlägt das Schreiben eines Evidence-Eintrags fehl, wird der Netzwerk-Tunnel sofort getrennt (`FULL_FLOW_AVAILABILITY_PASS`).

Zugehörige Dateien

```
Frontend:
- app/src/main/java/de/visiongaia/gedefense/mobile/EvidenceScreen.kt

Host Backend & Signer:
- app/src/main/java/de/visiongaia/gedefense/mobile/AndroidEvidence.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/RuntimeEvidenceStore.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/HybridArtifactSigner.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/EvidenceWriteFailureClassifier.kt

Core Journal Engine (:core):
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/EvidenceLedger.kt

Tests:
- tools/secure-vault-audit.py
- tools/core-check.sh
```

Dashboard-Integration

- **Dashboard-Seiten:** `EvidenceScreen` (eingebunden in `AnalysisHubScreen`).
- **Komponenten:** Kryptografischer Prüfstatus, Record-Zähler, SHA-256 Hash-Explorer, "Export Signed Journal"-Button.
- **Aktionen:** Journal verifizieren, Audit-Bundle exportieren, Kettengültigkeit prüfen.

Abhängigkeiten

```
Evidence Ledger & Artifact Signing
├─ nutzt AndroidKeyStore für HMAC-Root und ECDSA-Signaturschlüssel
├─ prüft KeyMint 5 Verfügbarkeit für ML-DSA-87 Signatur
└─ persistiert Daten in noBackupFilesDir/evidence/
```

Wird verwendet von

- `XdrEngine`, `GeDefenseVpnService`, `InstallGuard`, `ResilienceSupervisor`.

Modul: Device Hardening & Baseline Engine

Zweck

Das Modul `Device Hardening & Baseline Engine` überwacht die Konfiguration des Host-Betriebssystems sowie das normale Verhalten installierter Apps. Es erkennt Abweichungen von etablierten Normalwerten (Drift) und warnt vor unsicheren Systemeinstellungen.

Fähigkeiten

- **OS Hardening Audit:** Prüft Lockscreen-Komplexität, USB-Debugging, ADB-Status, Entwickleroptionen und fremde Zertifikate.
- **Paket-Baseline-Historie:** Erfasst und überwacht die Signatur-Zertifikats-Kette aller Apps (`PackageBaselineStore.kt`).
- **Verhaltensbasierte Netzwerkanalyse:** Berechnet gleitende Mittelwerte (EWMA) für App-Verbindungen und warnt bei statistischen Anomalien (`BehaviorEngine.kt`).
- **Selbst-Integritätsprüfung:** Der `IntegrityGuardian` verifiziert die eigene App-Integrität und alarmiert bei Repackaging.

Zugehörige Dateien

```
Frontend:
- app/src/main/java/de/visiongaia/gedefense/mobile/HardeningActivity.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/BehaviorActivity.kt

Host Backend & Stores:
- app/src/main/java/de/visiongaia/gedefense/mobile/DeviceHardeningScanner.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/BehaviorEngine.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/BehaviorBaselineStore.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/PackageBaselineStore.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/IntegrityGuardian.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/IntegrityJobService.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/PackageChangeReceiver.kt

Core Modelle (:core):
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/IntegrityBaseline.kt
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/TelemetryLearningPolicy.kt
```

Dashboard-Integration

- **Dashboard-Seiten:** `HardeningActivity` und `BehaviorActivity` (erreichbar über `AnalysisHubScreen`).
- **Komponenten:** Sicherheits-Audit-Karten, Baseline-Reifegradanzeige, Anomalie-Diagramme.
- **Aktionen:** Systemhärtung anwenden, Lernphase zurücksetzen, Baseline neu aufbauen.

Abhängigkeiten

```
Device Hardening & Baseline Engine
├─ liest OS-Attribute über DevicePolicyManager und Settings.Secure
```

- überwacht PackageManager Ereignisse (PackageChangeReceiver)
- speichert verschlüsselte Baselines in Domänen 2 & 3 des Vaults

Wird verwendet von

- `AnalysisHubScreen`, `XdrEngine`, `IntegrityJobService`.

Modul: Network Discovery & Port Sentinel Sensor

Zweck

Dieses Modul bietet lokale Netzwerkerkennung und Täuschungsfähigkeiten (Decoy). Es identifiziert fremde Geräte und Dienste im angeschlossenen LAN und lauscht passiv auf verdächtige Port-Scan-Aktivitäten von Netzwerkteilnehmern.

Fähigkeiten

- **Bounded LAN Sweep:** Scannt ausschließlich das device-lokale `/24`-Subnetz (maximal 254 Hosts) ohne VPN-Routing-Konflikt.
- **Zero-Allocation mDNS / SSDP Parser:** LiestBonjour- und UPnP-Dienste mit harten Rekursions- und Größenschranken (`MdnsDnsCodec.kt`).
- **Port Sentinel Ködersensor:** Öffnet defensive, unprivilegierte TCP/UDP-Listener auf ungenutzten Ports und schließt TCP-Verbindungen sofort nach Erfassung der Angreifer-IP.
- **Automatische Sink-Routen:** Entdeckte Portscanner-IPs werden in Selective Shield als `/32` Block-Routen blockiert.

Zugehörige Dateien

```
Frontend:
- app/src/main/java/de/visiongaia/gedefense/mobile/NetworkDiscoveryActivity.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/PortSentinelActivity.kt

Host Backend & Stores:
- app/src/main/java/de/visiongaia/gedefense/mobile/NetworkDiscoveryScanner.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/NetworkDiscoveryStore.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/PortSentinel.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/PortSentinelStore.kt

Core Algorithmen (:core):
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/NetworkRangePlanner.kt
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/MdnsDnsCodec.kt
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/LanBehaviorEvaluator.kt
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/PortSentinelClassifier.kt
```

Dashboard-Integration

- **Dashboard-Seiten:** `NetworkDiscoveryActivity` und `PortSentinelActivity` (in `ProtectionHubScreen`).
- **Komponenten:** LAN-Topologie-Liste, Scanner-Radar (`ScannerRadarView`), Angreifer-IP Denylist.
- **Aktionen:** LAN-Scan starten, Ködersensor aktivieren/deaktivieren, IP von Denylist entfernen.

Abhängigkeiten

```
Network Discovery & Port Sentinel Sensor
— bindet Sockets direkt an physisches Wi-Fi Network (umgeht VPN TUN)
— nutzt MdnsDnsCodec.kt für bounded DNS-SD Dekodierung
— persistiert Angreifer in PortSentinelStore.kt (Vault Domäne 8)
```

Wird verwendet von

- `ProtectionHubScreen`, `GeDefenseVpnService`, `XdrEngine`.

Modul: TITAN Device Owner & Enterprise Härtung

Zweck

TITAN ist das privilegierte Enterprise-Verwaltungsmodul von GeDefense. Es integriert sich tief in Androids **DevicePolicyManager** (DPC), um Kiosk-Modi, USB-Restriktionen, Always-On VPN Lockdown und Quarantäne-Suspendierungen auf Betriebssystemebene durchzusetzen.

Fähigkeiten

- **Device Owner Autorisierung:** Wird nur aktiv, wenn GeDefense als offizieller Device Owner bereitgestellt wurde.
- **Always-on VPN Lockdown:** Schaltet den Android-weiten Kill-Switch scharf, sobald der Schutzdienst aktiv ist.
- **App Suspension:** Kann von **XdrEngine** quarantänisierte Schad-Apps auf Betriebssystemebene einfrieren (Suspend).
- **Managed CA Installation:** Erlaubt Administratoren die sichere Installation geprüfter X.509 CAs nach Fingerprint-Bestätigung (**ManagedCaCertificateValidator.kt**).
- **Titan Visual Mode:** Schaltet die gesamte App-UI bei aktivem Device Owner auf das geschützte Gold/Cyan-Theme um.

Zugehörige Dateien

```
Frontend:  
- app/src/main/java/de/visiongaia/gedefense/mobile/TitanActivity.kt  
- app/src/main/java/de/visiongaia/gedefense/mobile/TitanCoreView.kt  
- app/src/main/java/de/visiongaia/gedefense/mobile/TitanManagedBanner.kt  
- app/src/main/java/de/visiongaia/gedefense/mobile/TitanVisualMode.kt  
  
Provisioning & Admin:  
- app/src/main/java/de/visiongaia/gedefense/mobile/TitanPolicyManager.kt  
- app/src/main/java/de/visiongaia/gedefense/mobile/TitanPolicyStore.kt  
- app/src/main/java/de/visiongaia/gedefense/mobile/TitanDeviceAdminReceiver.kt  
- app/src/main/java/de/visiongaia/gedefense/mobile/TitanPackageOperationReceiver.kt  
- app/src/main/java/de/visiongaia/gedefense/mobile/TitanGetProvisioningModeActivity.kt  
- app/src/main/java/de/visiongaia/gedefense/mobile/TitanPolicyComplianceActivity.kt  
  
Core Validierung (:core):  
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/ManagedCaCertificateValidator.kt
```

Dashboard-Integration

- **Dashboard-Seiten:** **TitanActivity** (erreichbar über **SystemHubScreen**).
- **Komponenten:** **TitanCoreView** Reaktor, Kiosk-Schalter, Restriktions-Liste, Managed-Banner.
- **Aktionen:** Richtlinien aktualisieren, Zertifikat prüfen und installieren, Gerätesperre testen.

Abhängigkeiten

```
TITAN Device Owner & Enterprise Härtung  
├─ benötigt BIND_DEVICE_ADMIN Berechtigung  
├─ steuert DevicePolicyManager API  
└─ speichert Admin-Präferenzen in TitanPolicyStore.kt (Vault Domäne 9)
```

Wird verwendet von

- **SystemHubScreen**, **XdrEngine**, **GeDefenseVpnService**.

Modul: Resilience & Self-Healing Supervisor

Zweck

Der **Resilience Supervisor** ist der autonome Selbstreparatur-Wächter von GeDefense Mobile. Er läuft periodisch als Hintergrunddienst, identifiziert Konfigurations- oder Speicher-Drift nach Geräteabstürzen und führt trust-preserving Selbstreparaturen durch.

Fähigkeiten

- **Single-Flight Ausführung:** Garantiert über atomare Flags, dass zu jedem Zeitpunkt maximal ein Reparatur-Durchlauf aktiv ist.
- **Trust-Preserving Selbstreparatur:** Lädt beschädigte Feed-Caches neu, revalidiert Paket-Baselines und überprüft Vault-Domains; löscht niemals Beweise oder Alarme.
- **Budgetierte Reparaturversuche:** Begrenzt automatische Reparaturen über **RecoveryAttemptBudget.kt**, um Endlosschleifen bei defekter Hardware abzufangen.
- **JobScheduler Integration:** Wird alle 2 Stunden (**ResilienceJobService**) sowie beim Start und nach Geräte-Reboot ausgeführt.

Zugehörige Dateien

```
Host Backend & Job:
- app/src/main/java/de/visiongaia/gedefense/mobile/ResilienceSupervisor.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/ResilienceJobService.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/ResilienceJobs.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/BoundedRecoveryCall.kt
- app/src/main/java/de/visiongaia/gedefense/mobile/SecureFiles.kt

Core Budgetierung (:core):
- core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/RecoveryAttemptBudget.kt

Tests:
- tools/resilience-supervisor-audit.py
```

Dashboard-Integration

- **Dashboard-Seiten:** Meldet Integritäts- und Selbstreparatur-Status an **SystemHubScreen**.
- **Daten:** Zeitstempel des letzten Checks, Status aktiver Komponenten.

Abhängigkeiten

```
Resilience & Self-Healing Supervisor
├─ nutzt Android JobScheduler API
├─ greift auf alle 12 Vault-Domains zu
└─ steuert Helper-Revalidierung via NativeGaiaNet.kt
```

Wird verwendet von

- **GeDefenseApplication**, **ResilienceJobService**, **BootReceiver**.

Modul: Diagnostics & Support Plane

Zweck

Das Modul **Diagnostics & Support Plane** ermöglicht den datenschutzkonformen Export aggregierter Systemmetriken zur Fehleranalyse sowie die Einsicht von Lizenz- und Herstellerinformationen.

Fähigkeiten

- **Aggregate-Only Diagnostics (`aggregate-only-v1`)**: Filtert vor dem Export konsequent sämtliche Dateipfade, Domainnamen, IP-Adressen und Paket-Payloads heraus.
- **Storage Access Framework (SAF)**: Exportiert Diagnosedateien direkt über Androids Dateiwähler ohne broad Storage Permissions.
- **Transparente JSON-Struktur**: Nutzer können den Diagnose-Inhalt vor dem Speichern in der UI einsehen.
- **Support VGT Link**: Öffnet externe Hilfeseiten ausschließlich im System-Browser ohne eingebettetes Web-Tracking.

Zugehörige Dateien

```
Frontend:  
- app/src/main/java/de/visiongaia/gedefense/mobile/DiagnosticsActivity.kt  
- app/src/main/java/de/visiongaia/gedefense/mobile/SupportVgtActivity.kt  
  
Host Backend:  
- app/src/main/java/de/visiongaia/gedefense/mobile/DiagnosticBundle.kt  
- app/src/main/java/de/visiongaia/gedefense/mobile/FullFlowAnalytics.kt
```

Dashboard-Integration

- **Dashboard-Seiten**: `DiagnosticsActivity` und `SupportVgtActivity` (über `SystemHubScreen`).
- **Komponenten**: Export-Button, Diagnose-Vorschau-View, Lizenztexte.
- **Aktionen**: Diagnose-Bundle generieren und via SAF speichern, Release Notes ansehen.

Abhängigkeiten

```
Diagnostics & Support Plane  
├─ aggregiert Zähler aus AppRuntime.kt und ProtectionMetrics.kt  
└─ nutzt Android Storage Access Framework (SAF)
```

Wird verwendet von

- `SystemHubScreen`.

Modul: Setup & Onboarding Assistant

Zweck

Der `Setup & Onboarding Assistant` führt den Nutzer bei der Erstinstallation durch einen 10-stufigen, strukturierten Assistenten. Er erklärt transparent die Funktionsweise des lokalen VPNs, holt Berechtigungen defensiv ein und initialisiert die ersten Bedrohungsfeeds.

Fähigkeiten

- **10-Schritte Wizard**: Führt durch Berechtigungen, Akku-Optimierung, VPN-Erklärung und Profilwahl.
- **Google Play Prominent Disclosure**: Informiert über den lokalen Zweck des `VpnService` und persistiert die HMAC-signierte Zustimmung in `VpnDisclosureStore.kt`.
- **Verzögerte Inventarisierung**: Startet Hintergrund-Scans erst nach offiziellem Abschluss des Assistenten, um den Setup-Flow flüssig zu halten.

Zugehörige Dateien

```
Frontend:  
- app/src/main/java/de/visiongaia/gedefense/mobile/SetupWizardActivity.kt  
- app/src/main/java/de/visiongaia/gedefense/mobile/SetupWizardPageRenderer.kt
```

```
- app/src/main/java/de/visiongaia/gedefense/mobile/VpnDisclosureActivity.kt
```

Host Backend & Stores:

```
- app/src/main/java/de/visiongaia/gedefense/mobile/DeviceSetupManager.kt  
- app/src/main/java/de/visiongaia/gedefense/mobile/SetupWizardModel.kt  
- app/src/main/java/de/visiongaia/gedefense/mobile/VpnDisclosureStore.kt  
- app/src/main/java/de/visiongaia/gedefense/mobile/InitialSetupFeedSync.kt
```

Tests:

```
- tools/onboarding-audit.py
```

Dashboard-Integration

- **Dashboard-Seiten:** `SetupWizardActivity` (startet automatisch bei fehlendem Setup-Marker) und `VpnDisclosureActivity`.
- **Komponenten:** Karussell-Navigation, animierte Erklär-Icons, Checkboxes für Berechtigungen.
- **Aktionen:** Berechtigungen anfordern, VPN-Einverständnis erteilen, Schutz aktivieren.

Abhängigkeiten

Setup & Onboarding Assistant

```
├─ persistiert Zustimmung in VpnDisclosureStore.kt (HMAC-SHA-256)  
├─ triggert InitialSetupFeedSync.kt  
└─ übergibt Flow bei Erfolg an MainActivity.kt
```

Wird verwendet von

- `MainActivity`, `StartupActivity`, `GeDefenseVpnService`.

4. DASHBOARD DETAILLIERT DOKUMENTIEREN

Dashboard → Cockpit / Übersicht (DashboardScreen)

- **Route / Intent:** Tab 0 in `MainActivity` (`selectedScreen = 0`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/DashboardScreen.kt`
- **CSS / Styling:** Reines Kotlin View-Styling via `GeDefenseUi.kt`: `heroPanelBackground`, `glassPanelBackground`, `pill`, `SPACING_CARD_GAP_DP`.
- **Verwendete Komponenten:** `ShieldPulseView`, `VgtActionTile`, `VgtProgressView`, `VgtIconView`, `CyberBackgroundView`, `ImageView` (Logo/Texture).
- **Zuständiges Modul:** `Host & Application Runtime & Network & VPN Core`.
- **API-Aufrufe / IPC:** Liest `AppRuntime.peek()?.uiSnapshot()`; sendet Intent an `GeDefenseVpnService` zum Starten/Stoppen des Tunnels.
- **Backend Handler:** `MainActivity.runtimeListener`, `AppRuntime.refresh()`.
- **Services:** `GeDefenseVpnService`.
- **Datenbankzugriffe:** Liest aggregierten Cache aus `UiSnapshot` (kein direkter Disk-I/O im Renderpfad).
- **Authentifizierung / Berechtigungen:** Prüft `VpnDisclosureStore` (HMAC-Zustimmung) vor Start; erfordert Android VPN-Nutzerbestätigung.
- **Datenfluss:**

```
DashboardScreen (Nutzer tippt "Schutz aktivieren")
↓
MainActivity.activateProtection()
↓
VpnService.prepare() / Intent
↓
GeDefenseVpnService.onStartCommand()
↓
NativeGaiaNet.startHelperProcess()
↓
Unix Socket Token-Handshake + SCM_RIGHTS FD Passing
↓
GaiaNet V2 Netstack Loop aktiv
↓
Host meldet State-Update via Event-Callback
↓
DashboardScreen.refresh(snapshot) -> ShieldPulseView pulsiert cyan
```

Dashboard → Threat Intelligence Matrix (ThreatScreen)

- **Route / Intent:** Tab 1 in `MainActivity` (`selectedScreen = 1`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/ThreatScreen.kt`
- **CSS / Styling:** `GeDefenseUi.panelBackground`, `GeDefenseUi.accentRuleBackground`, Cyber Gold/Cyan Akzente.
- **Verwendete Komponenten:** `TrafficWorldMapView` (Vektor-Mercator-Map), `VgtIconView`, `VgtProgressView`, Feed-Item Karten.
- **Zuständiges Modul:** `Threat Intelligence & Policy Engine`.
- **API-Aufrufe / IPC:** Ruft `AppRuntime.threats.refreshAsync()` auf; sendet Feed-Sync Intent an `ThreatIntelJobService`.
- **Backend Handler:** `ThreatIntelRepository`, `ThreatIntelParser`.
- **Services:** `ThreatIntelJobService`.
- **Datenbankzugriffe:** Liest `ThreatCacheStore` (HMAC-authentifizierter binärer Feed-Cache).

- **Authentifizierung / Berechtigungen:** Keine Sonderrechte; reine lokale Datenanzeige.
- **Datenfluss:**

```

ThreatScreen (Nutzer tippt "Sync Now")
    ↓
ThreatIntelRepository.triggerManualDownload()
    ↓
FeedDownloader.downloadHttps(Feodo, Spamhaus, ...)
    ↓
ThreatIntelParser.parseTrie()
    ↓
ThreatCacheStore.persistAtomic() (HMAC signiert)
    ↓
NativeGaiaNet.reloadPolicy(newSha256) via SCM_RIGHTS Pipe
    ↓
ThreatScreen aktualisiert Zeitstempel und Weltkarte

```

Dashboard → Protection Hub (ProtectionHubScreen)

- **Route / Intent:** Tab 2 in `MainActivity` (`selectedScreen = 2`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/ProtectionHubScreen.kt`
- **CSS / Styling:** `GeDefenseUi.glassPanelBackground`, Grid-Karten mit dynamischen Icon-Wells (`VgtIconWell`).
- **Verwendete Komponenten:** `VgtActionTile`, `VgtIconView`, `VgtUiComponents.screenHeader`.
- **Zuständiges Modul:** `Network & VPN Core / Transport Plane`.
- **API-Aufrufe / IPC:** Öffnet Sub-Activities (`FirewallActivity`, `WireGuardActivity`, `PrivacyActivity`, `ScannerActivity`, `PortSentinelActivity`, `NetworkDiscoveryActivity`) via `startActivity()`.
- **Backend Handler:** Android Activity-Manager.
- **Services:** Keine direkten Services; navigiert zu Modul-Subsystemen.
- **Datenbankzugriffe:** Keine.
- **Authentifizierung / Berechtigungen:** Keine.
- **Datenfluss:**

```

ProtectionHubScreen (Nutzer tippt Modul-Kachel, z. B. "WireGuard")
    ↓
startActivity(Intent(context, WireGuardActivity::class.java))
    ↓
RuntimeActivityEntry prüft AppRuntime Bereitschaft
    ↓
Sub-Activity wird gerendert

```

Dashboard → Analysis Hub (AnalysisHubScreen)

- **Route / Intent:** Tab 3 in `MainActivity` (`selectedScreen = 3`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/AnalysisHubScreen.kt`
- **CSS / Styling:** Forensischer Cyan/Red High-Contrast Modus via `GeDefenseUi.panelStrong`.
- **Verwendete Komponenten:** `VgtActionTile`, `VgtProgressView`, Vorfalls-Zusammenfassungen-Karten.
- **Zuständiges Modul:** `XDR / EDR Correlation & Incident Plane`.
- **API-Aufrufe / IPC:** Öffnet `XdrActivity`, `BehaviorActivity`, `HardeningActivity`, `EvidenceScreen` via Android Intents.
- **Backend Handler:** `XdrEngine`, `RuntimeEvidenceStore`.
- **Services:** Keine.
- **Datenbankzugriffe:** Liest Status aus `XdrEventStore` und `EvidenceLedger`.

- **Authentifizierung / Berechtigungen:** Keine.
- **Datenfluss:**

```

AnalysisHubScreen
  ↓
Liest ungelöste Vorfälle aus XdrEventStore
  ↓
Aggregiert Zähler & Risikoscore
  ↓
Rendert Übersichtskacheln mit Gefahrenindikatoren

```

Dashboard → System Hub (SystemHubScreen)

- **Route / Intent:** Tab 4 in `MainActivity` (`selectedScreen = 4`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/SystemHubScreen.kt`
- **CSS / Styling:** Enterprise Styling mit dynamischer Gold/Cyan Umschaltung via `TitanVisualMode`.
- **Verwendete Komponenten:** `VgtActionTile`, `VgtIconView`, `VgtUiComponents.divider`.
- **Zuständiges Modul:** `TITAN Device Owner & Enterprise Härtung & Diagnostics`.
- **API-Aufrufe / IPC:** Navigiert zu `TitanActivity`, `DiagnosticsActivity`, `SupportVgtActivity`, `SetupWizardActivity`.
- **Backend Handler:** `TitanPolicyManager`, `AppRuntime`.
- **Services:** `ResilienceJobService`.
- **Datenbankzugriffe:** Liest Device-Owner Status und Vault-Kustodie Status.
- **Authentifizierung / Berechtigungen:** Erkennt `DevicePolicyManager.isDeviceOwnerApp()`.
- **Datenfluss:**

```

SystemHubScreen
  ↓
Prüft TitanVisualMode.active
  ↓
Rendert Kacheln für TITAN, Diagnosen, Setup und Lizenzen

```

Dashboard → Firewall Manager (FirewallActivity)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.FirewallActivity` (via `ProtectionHubScreen`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/FirewallActivity.kt`
- **CSS / Styling:** Scrollbare Listen-Karten mit Farbkodierung (Grün = Erlaubt, Rot = Gesperrt, Gelb = Eingeschränkt).
- **Verwendete Komponenten:** `CyberBackgroundView`, `VgtAppIconCache` (Asynchrones Icon-Laden), `VgtWindowInsets`.
- **Zuständiges Modul:** `Network & VPN Core / Transport Plane`.
- **API-Aufrufe / IPC:** Schreibt Regeln via `FirewallPolicyStore`; synchronisiert Richtlinien atomar mit `GeDefenseVpnService`.
- **Backend Handler:** `FirewallPolicyStore`, `GeDefenseVpnService.setAppRule()`.
- **Services:** `GeDefenseVpnService`.
- **Datenbankzugriffe:** Schreibt verschlüsselte Regeln in `noBackupFilesDir/firewall/` (Vault Domäne 6, AES-256-GCM).
- **Authentifizierung / Berechtigungen:** Keine Sonderrechte.
- **Datenfluss:**

```
Nutzer sperrt App "xyz" in FirewallActivity
↓
FirewallPolicyStore.setRule(pkg, BLOCK)
↓
SecureTelemetryVault verschlüsselt Richtlinie (Domäne 6)
↓
GeDefenseVpnService.refreshRules()
↓
Neues Regelsatz-Bitmap an Go Netstack via Socket Pipe
↓
Künftige Flows der App werden im Paket-Loop sofort verworfen
```

Dashboard → WireGuard Profile Manager (WireGuardActivity)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.WireGuardActivity` (via `ProtectionHubScreen`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/WireGuardActivity.kt`
- **CSS / Styling:** `GeDefenseUi.glassPanelBackground`, interaktive Eingabefelder mit Monospace-Schriftart für Public Keys.
- **Verwendete Komponenten:** `CyberBackgroundView`, `VgtUiComponents`, QR-Code-Import-Dialog, MTU-Schieberegler.
- **Zuständiges Modul:** `WireGuard L3 Egress Subsystem`.
- **API-Aufrufe / IPC:** Übergibt WireGuard UAPI Stream via Unix Socket an `NativeGaiaNet.kt` / `wireguard_transport.go`.
- **Backend Handler:** `WireGuardConfigParser.kt`, `WireGuardProfileStore.kt`.
- **Services:** `GeDefenseVpnService`.
- **Datenbankzugriffe:** Schreibt gewrappte UAPI Profile in `WireGuardProfileStore` (KeyStore KEK verschlüsselt).
- **Authentifizierung / Berechtigungen:** Erfordert Android Lockdown API Prüfung für den Strict-Modus.
- **Datenfluss:**

```
Nutzer importiert .conf Datei
↓
WireGuardConfigParser.parseBounded(16 KiB Limit)
↓
Validiert Interface, Peer, AllowedIPs und numerische DNS
↓
WireGuardProfileStore.persistEncrypted()
↓
GeDefenseVpnService schaltet Tunnel um:
1. Öffnet UAPI Pipe zum Go Helper
2. Empfängt WireGuard UDP FDs via SCM_RIGHTS
3. Bindet Sockets an physisches Network & ruft VpnService.protect()
4. Bestätigt Bind-Aktivierung an den Helper
↓
WireGuard L3 Tunnel steht; UI zeigt Endpunkt & Handshake aktiv
```

Dashboard → Privacy & Telemetry Shield (PrivacyActivity)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.PrivacyActivity` (via `ProtectionHubScreen`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/PrivacyActivity.kt`
- **CSS / Styling:** 4-Stufen Profilkarten mit Radio-Status-Pills in Cyan und Gold.
- **Verwendete Komponenten:** `CyberBackgroundView`, `VgtUiComponents.screenHeader`, Tracker-Zähler-Karten.
- **Zuständiges Modul:** `Telemetry Shield & Privacy Engine`.
- **API-Aufrufe / IPC:** Ruft `AppRuntime.privacy.setProfile()` auf; sendet neue Konfiguration an `netstack/privacy_policy.go`.

- **Backend Handler:** `PrivacyIntelligenceRepository.kt`.
- **Services:** `GeDefenseVpnService`.
- **Datenbankzugriffe:** Schreibt Profilauswahl in verschlüsselten Snapshot (Vault Domäne 6).
- **Authentifizierung / Berechtigungen:** Keine Sonderrechte.
- **Datenfluss:**

```

Nutzer wählt Profil "STRICT"
↓
PrivacyIntelligenceRepository.setProfile(STRICT)
↓
Sendet Kontroll-Frame via IPC an netstack/privacy_policy.go
↓
Go Engine schaltet Filter scharf:
- Beobachtet Port 53 DNS Queries
- Blockiert Port 853 DoT (TCP) und DoQ (UDP) vor Egress
↓
Blockierte Tracking-Versuche werden an Host gemeldet

```

Dashboard → Scanner & Quarantäne (ScannerActivity)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.ScannerActivity` (via `ProtectionHubScreen`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/ScannerActivity.kt`
- **CSS / Styling:** `ScannerRadarView` Sweep-Animation, Quarantäne-Warnbanner in VGT-Rot (`#FF5467`).
- **Verwendete Komponenten:** `ScannerRadarView`, `CyberBackgroundView`, `VgtProgressView`, Quarantäne-Listen-Item.
- **Zuständiges Modul:** `InstallGuard` & `Scanner Plane`.
- **API-Aufrufe / IPC:** Startet manuellen Scan via `AppRiskScanner.scanAllAsync()` oder `StorageMalwareScanner.scanAsync()`.
- **Backend Handler:** `InstallGuard.kt`, `AppRiskScanner.kt`, `StorageMalwareScanner.kt`.
- **Services:** Keine Hintergrund-Services (Bounded Worker Pools).
- **Datenbankzugriffe:** Liest und schreibt `ScannerStateCache.kt` (AES-256-GCM Vault Domäne 10 & 11).
- **Authentifizierung / Berechtigungen:** Optional `MANAGE_EXTERNAL_STORAGE` für Storage-Scan; `QUERY_ALL_PACKAGES` für App-Scan.
- **Datenfluss:**

```

ScannerActivity (Nutzer tippt "Full Device Scan")
↓
AppRiskScanner liest installierte APKs via PackageManager
↓
Verifiziert Signer-Historie gegen PackageBaselineStore
↓
Extrahiert Permissions & vergleicht mit ThreatTokenScanner
↓
Verdächtige Apps werden sofort in Quarantäne gesetzt
↓
PackageEgressGate blockiert Netzwerkverkehr der App
↓
Ergebnis wird in ScannerStateCache persistiert und im Radar visualisiert

```

Dashboard → XDR Incident Explorer (XdrActivity)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.XdrActivity` (via `AnalysisHubScreen`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/XdrActivity.kt`
- **CSS / Styling:** Forensischer High-Density Look mit ausklappbaren JSON-Details und Severity-Pills.

- **Verwendete Komponenten:** `CyberBackgroundView`, `VgtUiComponents`, `Vorfalls-Timeline-Adapter`.
- **Zuständiges Modul:** `XDR / EDR Correlation & Incident Plane`.
- **API-Aufrufe / IPC:** Liest Vorfälle via `AppRuntime.xdr.getIncidents()`; triggert Notfall-Isolation via `GeDefenseVpnService`.
- **Backend Handler:** `XdrEngine.kt`, `XdrEventStore.kt`.
- **Services:** `GeDefenseVpnService`.
- **Datenbankzugriffe:** Liest verschlüsselte Vorfälle aus `XdrEventStore` (Vault Domäne 1, AES-256-GCM).
- **Authentifizierung / Berechtigungen:** Keine Sonderrechte; optional Device Owner für App-Suspension.
- **Datenfluss:**

```

XdrActivity wird geöffnet
↓
XdrEventStore entschlüsselt Vorfälle via SecureTelemetryVault (Domäne 1)
↓
XdrEngine berechnet Score-Erklärungen für die UI
↓
Timeline zeigt korrelierte Angriffe (z. B. C2-IP + Signer-Drift)
↓
Nutzer wählt "App isolieren" -> GeDefenseVpnService sperrt App sofort

```

Dashboard → Device Hardening (HardeningActivity)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.HardeningActivity` (via `AnalysisHubScreen`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/HardeningActivity.kt`
- **CSS / Styling:** Checklisten-Layout mit Status-Dots (Grün = Gehärtet, Rot = Schwachstelle) und VGT Glass-Cards.
- **Verwendete Komponenten:** `CyberBackgroundView`, `VgtUiComponents`, `VgtWindowInsets`.
- **Zuständiges Modul:** `Device Hardening & Baseline Engine`.
- **API-Aufrufe / IPC:** Liest OS-Attribute über `DeviceHardeningScanner.kt`.
- **Backend Handler:** `DeviceHardeningScanner.kt`.
- **Services:** Keine.
- **Datenbankzugriffe:** Keine Persistenz (Live-Evaluation der OS-Zustände).
- **Authentifizierung / Berechtigungen:** Prüft System-Einstellungen (`Settings.Secure`, `KeyguardManager`).
- **Datenfluss:**

```

HardeningActivity wird geöffnet
↓
DeviceHardeningScanner prüft OS-Attribute:
- ADB & USB-Debugging aktiviert?
- Entwickleroptionen entsperret?
- Lockscreen-Passwort komplex genug?
- Unbekannte Root-CAs im System-Speicher?
↓
Ergebnisse werden im Checklisten-Grid mit Warnungen gerendert

```

Dashboard → Behavioral Baseline (BehaviorActivity)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.BehaviorActivity` (via `AnalysisHubScreen`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/BehaviorActivity.kt`
- **CSS / Styling:** Datenzentrierte Ansicht mit EWMA-Lernfortschritts-Balken und Drift-Metriken.
- **Verwendete Komponenten:** `CyberBackgroundView`, `VgtProgressView`, `VgtUiComponents`.

- **Zuständiges Modul:** `Device Hardening & Baseline Engine`.
- **API-Aufrufe / IPC:** Fragt Baselines über `AppRuntime.behavior.getSnapshot()` ab.
- **Backend Handler:** `BehaviorEngine.kt`, `BehaviorBaselineStore.kt`.
- **Services:** Keine.
- **Datenbankzugriffe:** Liest und schreibt `BehaviorBaselineStore` (Vault Domäne 2, AES-256-GCM).
- **Authentifizierung / Berechtigungen:** Keine Sonderrechte.
- **Datenfluss:**

```
BehaviorActivity
↓
BehaviorBaselineStore liest gespeicherte EWMA-Modelle für App-Netzwerkverbindungen
↓
Rendert Lernfortschritt pro App (z. B. "94 % trainiert")
↓
Hebt ungewöhnliche Spitzen im Verbindungsvolumen farblich hervor
```

Dashboard → Network Discovery Asset Map (`NetworkDiscoveryActivity`)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.NetworkDiscoveryActivity` (via `ProtectionHubScreen`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/NetworkDiscoveryActivity.kt`
- **CSS / Styling:** Netzwerk-Topologie-Karten mit IP/MAC/Host-Spalten und Dienst-Badges.
- **Verwendete Komponenten:** `CyberBackgroundView`, `VgtUiComponents`, `VgtProgressView`.
- **Zuständiges Modul:** `Network Discovery & Port Sentinel Sensor`.
- **API-Aufrufe / IPC:** Startet Scan über `NetworkDiscoveryScanner.scanAsync()`.
- **Backend Handler:** `NetworkDiscoveryScanner.kt`, `NetworkDiscoveryStore.kt`.
- **Services:** Keine.
- **Datenbankzugriffe:** Persistiert LAN-Topologie in `NetworkDiscoveryStore` (Vault Domäne 7, AES-256-GCM).
- **Authentifizierung / Berechtigungen:** Erfordert aktive Wi-Fi/Ethernet-Verbindung; nutzt Multicast-Lock.
- **Datenfluss:**

```
Nutzer startet "LAN Discovery"
↓
NetworkRangePlanner berechnet lokales /24 Subnetz (max 254 Hosts)
↓
NetworkDiscoveryScanner bindet Sockets direkt an das Wi-Fi Interface
↓
Sendet ARP, SSDP und mDNS Probes (dekodiert via MdnsDnsCodec.kt)
↓
Entdeckte Geräte (Drucker, Router, NAS) werden im Topologie-Tree angezeigt
```

Dashboard → Port Sentinel Sensor Deck (`PortSentinelActivity`)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.PortSentinelActivity` (via `ProtectionHubScreen`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/PortSentinelActivity.kt`
- **CSS / Styling:** Taktisches Cyber-Radar-Design mit Angreifer-Tabelle und blockierten Port-Slots.
- **Verwendete Komponenten:** `ScannerRadarView`, `CyberBackgroundView`, `VgtUiComponents`.
- **Zuständiges Modul:** `Network Discovery & Port Sentinel Sensor`.
- **API-Aufrufe / IPC:** Steuert Listener via `PortSentinel.start()` / `stop()`.
- **Backend Handler:** `PortSentinel.kt`, `PortSentinelStore.kt`.

- **Services:** `GeDefenseVpnService` (zur Registrierung von Sink-Routen).
- **Datenbankzugriffe:** Schreibt geblockte Angreifer in `PortSentinelStore` (Vault Domäne 8, AES-256-GCM).
- **Authentifizierung / Berechtigungen:** Bindet Sockets an unprivilegierte Ports (>1024).
- **Datenfluss:**

```

PortSentinel Listener läuft auf ungenutzten Ports
↓
Angreifer im LAN führt Portscan gegen das Smartphone durch
↓
PortSentinel akzeptiert TCP-Verbindung kurz, extrahiert Quell-IP & schließt Socket sofort
↓
PortSentinelClassifier erkennt Scan-Muster
↓
Angreifer-IP wird in PortSentinelStore gesperrt & an XdrEngine gemeldet
↓
Radar-View zeigt akustischen/visuellen Alarm

```

Dashboard → TITAN Device Owner Console (TitanActivity)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.TitanActivity` (via `SystemHubScreen`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/TitanActivity.kt`
- **CSS / Styling:** Exklusives Enterprise Gold/Cyan Theme via `TitanVisualMode`, animierter `TitanCoreView` Reaktor.
- **Verwendete Komponenten:** `TitanCoreView`, `TitanManagedBanner`, `CyberBackgroundView`, `VgtUiComponents`.
- **Zuständiges Modul:** `TITAN Device Owner & Enterprise Härtung`.
- **API-Aufrufe / IPC:** Führt privilegierte Richtlinienänderungen via `TitanPolicyManager` über `DevicePolicyManager` aus.
- **Backend Handler:** `TitanPolicyManager.kt`, `TitanPolicyStore.kt`.
- **Services:** Keine.
- **Datenbankzugriffe:** Schreibt Admin-Konfigurationen in `TitanPolicyStore` (Vault Domäne 9, AES-256-GCM).
- **Authentifizierung / Berechtigungen:** Erfordert zwingend Android Device Owner Privilegien (`BIND_DEVICE_ADMIN`).
- **Datenfluss:**

```

Admin schaltet "Always-On Lockdown" in TitanActivity ein
↓
TitanPolicyManager prüft Device-Owner Status
↓
DevicePolicyManager.setAlwaysOnVpnPackage(..., lockdownEnabled = true)
↓
TitanPolicyStore persistiert Richtlinie
↓
TitanCoreView pulsiert im goldenen Enterprise-Status

```

Dashboard → Evidence Ledger Journal (EvidenceScreen)

- **Route / Intent:** Eingebetteter Screen in `AnalysisHubScreen` bzw. separater View.
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/EvidenceScreen.kt`
- **CSS / Styling:** Monospace Log-Viewer mit kryptografischen Prüfsiegel-Badges in Grün (`#36E6AB`).
- **Verwendete Komponenten:** `VgtUiComponents`, `CyberBackgroundView`, Record-Karten mit SHA-256 Hashes.
- **Zuständiges Modul:** `Evidence Ledger & Artifact Signing`.
- **API-Aufrufe / IPC:** Ruft `AppRuntime.evidence.verifyChainAsync()` und `exportSignedArtifact()` auf.
- **Backend Handler:** `AndroidEvidence.kt`, `HybridArtifactSigner.kt`.

- **Services:** Keine.
- **Datenbankzugriffe:** Liest verschlüsselte Audit-Einträge aus `EvidenceLedger` (Vault Domäne 12).
- **Authentifizierung / Berechtigungen:** Nutzt Android KeyStore für Signaturerzeugung; optional SAF für Export.
- **Datenfluss:**

```
Nutzer tippt "Export Signed Evidence"
↓
EvidenceLedger serialisiert Einträge & verifiziert HMAC-Kette
↓
HybridArtifactSigner generiert SHA-256 Prüfsumme
↓
Signiert mit Android KeyStore ECDSA P-256 (und ML-DSA-87 auf Android 17+)
↓
Erzeugt .sig.json Datei und bietet Download via Android SAF an
```

Dashboard → Diagnostics & Export (DiagnosticsActivity)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.DiagnosticsActivity` (via `SystemHubScreen`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/DiagnosticsActivity.kt`
- **CSS / Styling:** Bounded Text-Viewer für JSON-Auszüge, Datensicherheits-Badges in Cyan.
- **Verwendete Komponenten:** `CyberBackgroundView`, `VgtUiComponents`, "Export Bundle"-Aktionsleiste.
- **Zuständiges Modul:** `Diagnostics & Support Plane`.
- **API-Aufrufe / IPC:** Generiert Bundle via `DiagnosticBundle.build()`.
- **Backend Handler:** `DiagnosticBundle.kt`.
- **Services:** Keine.
- **Datenbankzugriffe:** Liest ausschließlich anonymisierte Systemzähler.
- **Authentifizierung / Berechtigungen:** Nutzt Android Storage Access Framework (SAF).
- **Datenfluss:**

```
DiagnosticsActivity wird geöffnet
↓
DiagnosticBundle filtert sämtliche PII (IPs, Domains, Dateinamen) heraus
↓
Rendert JSON-Vorschau in scrollbarem View
↓
Nutzer wählt Speicherort via SAF Dateiwähler
↓
Schreibt aggregate-only-v1.json direkt auf den Nutzerspeicher
```

Dashboard → Support VGT (SupportVgtActivity)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.SupportVgtActivity` (via `SystemHubScreen`).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/SupportVgtActivity.kt`
- **CSS / Styling:** Minimalistisches, zentriertes Cyber-Layout mit VGT Logo und Editions-Metadaten.
- **Verwendete Komponenten:** `CyberBackgroundView`, `VgtUiComponents`, Action-Buttons.
- **Zuständiges Modul:** `Diagnostics & Support Plane`.
- **API-Aufrufe / IPC:** Öffnet System-Browser via Intent (`ACTION_VIEW`).
- **Backend Handler:** Android OS Intent Resolver.
- **Services:** Keine.
- **Datenbankzugriffe:** Keine.

- **Authentifizierung / Berechtigungen:** Keine Sonderrechte.
- **Datenfluss:**

```
Nutzer tippt "Support VGT"
↓
startActivity(Intent(ACTION_VIEW, Uri.parse("https://paypal.me/...")))
↓
Android öffnet externen Standard-Browser (kein WebView in GeDefense)
```

Dashboard → Setup Wizard (SetupWizardActivity)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.SetupWizardActivity` (Automatischer Start bei Erstinstallation).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/SetupWizardActivity.kt`
- **CSS / Styling:** 10-stufige horizontale Karussell-Ansicht mit Fortschrittsanzeige via `SetupWizardPageRenderer.kt`.
- **Verwendete Komponenten:** `CyberBackgroundView`, `VgtProgressView`, Berechtigungs-Karten.
- **Zuständiges Modul:** `Setup & Onboarding Assistant`.
- **API-Aufrufe / IPC:** Triggert `DeviceSetupManager.completeSetup()`; startet initialen Feed-Download.
- **Backend Handler:** `DeviceSetupManager.kt`, `InitialSetupFeedSync.kt`.
- **Services:** `ThreatIntelJobService`.
- **Datenbankzugriffe:** Speichert Setup-Status in verschlüsselten Preferences.
- **Authentifizierung / Berechtigungen:** Fragt Android Berechtigungen Schritt für Schritt ab.
- **Datenfluss:**

```
Erststart -> StartupActivity stellt fest: Setup unvollständig
↓
SetupWizardActivity startet
↓
Führt Nutzer durch 10 Schritte (Berechtigungen, Akku, VPN-Erklärung)
↓
Nutzer schließt Wizard ab -> DeviceSetupManager markiert Setup als beendet
↓
InitialSetupFeedSync lädt erste Bedrohungslisten
↓
Leitet nahtlos zu MainActivity weiter
```

Dashboard → VPN Disclosure Notice (VpnDisclosureActivity)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.VpnDisclosureActivity` (Vor dem ersten VPN-Start).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/VpnDisclosureActivity.kt`
- **CSS / Styling:** Prominenter Vollbild-Hinweis mit Glass-Panels und expliziter "Zustimmen"-Schaltfläche.
- **Verwendete Komponenten:** `CyberBackgroundView`, `VgtUiComponents`, `VgtIconView`.
- **Zuständiges Modul:** `Setup & Onboarding Assistant`.
- **API-Aufrufe / IPC:** Speichert Bestätigung in `VpnDisclosureStore.kt`.
- **Backend Handler:** `VpnDisclosureStore.kt`.
- **Services:** `GeDefenseVpnService`.
- **Datenbankzugriffe:** Schreibt HMAC-signierte Bestätigung in `noBackupFilesDir/disclosure/` (kein AEAD).
- **Authentifizierung / Berechtigungen:** Voraussetzung für Android `VpnService.prepare()`.
- **Datenfluss:**

```

VpnDisclosureActivity
↓
Erklärt: "GeDefense erstellt einen lokalen VPN-Tunnel ohne Datenübertragung an VGT"
↓
Nutzer bestätigt -> VpnDisclosureStore signiert Quittung via AndroidKeyStore HMAC
↓
Gibt RESULT_OK an MainActivity zurück
↓
MainActivity fordert Android OS VPN-Zustimmung an

```

Dashboard → Startup & Bootstrap Screen (StartupActivity)

- **Route / Intent:** `de.visiongaia.gedefense.mobile.StartupActivity` (Haupt-Launcher im Manifest).
- **Frontend-Datei:** `app/src/main/java/de/visiongaia/gedefense/mobile/StartupActivity.kt`
- **CSS / Styling:** Dunkler, blitzschnell ladender Bootloader mit animiertem `StartupCoreView`.
- **Verwendete Komponenten:** `StartupCoreView`, `CyberBackgroundView`.
- **Zuständiges Modul:** `Host & Application Runtime`.
- **API-Aufrufe / IPC:** Wartet auf asynchronen Abschluss von `AppRuntime.ensureReady()`.
- **Backend Handler:** `AppRuntime.kt`.
- **Services:** Keine.
- **Datenbankzugriffe:** Wartet auf Freigabe des KeyStores und Entschlüsselung der Basis-Caches.
- **Authentifizierung / Berechtigungen:** Keine.
- **Datenfluss:**

```

App-Icon geklickt -> StartupActivity startet sofort (Theme ohne ActionBar)
↓
AppRuntime Initialisierungs-Thread startet im Hintergrund:
- Prüft Android KeyStore Verfügbarkeit
- Lädt verschlüsselte Baselines & Feed-Tries in den RAM
- Validiert Setup-Status
↓
Sobald bereit: StartupActivity leitet zu MainActivity (oder SetupWizard) weiter
↓
StartupActivity beendet sich selbst (finish())

```

5. CSS / UI ARCHITEKTUR

GeDefense Mobile verwendet ein proprietäres, hochgradig optimiertes **programmatisches UI-System in reinem Kotlin**. Die Architektur verzichtet bewusst vollständig auf Android XML-Layouts sowie auf Jetpack Compose. Sämtliche Views werden zur Laufzeit direkt instanziiert und gestylt. Dies eliminiert XML-Parsing-Overhead, reduziert die APK-Größe drastisch und garantiert konsistente 60/120-FPS-Bildwiederholraten bei minimalem Speicherverbrauch.

5.1 Globale Design Tokens & Farbpalette (GeDefenseUi.kt)

Alle visuellen Attribute sind zentral als unveränderliche Tokens in `GeDefenseUi.kt` gekapselt:

```
FARBPALETTE (VGT CYBER DARK THEME)
├─ Hintergrund (bg):           #02070D (Tiefes Weltraum-Schwarz)
├─ Glass-Panel (panel):       rgba(7, 19, 30, 0.88)
├─ Starkes Panel (panelStrong): rgba(8, 23, 36, 0.95)
├─ Weiches Panel (panelSoft):  rgba(11, 31, 47, 0.66)
├─ Primärtext (text):         #F6F9FC (Klinisches Weiß)
├─ Sekundärtext (textMuted):   #A4B8C9 (Gedämpftes Blaugrau)
├─ Dimmer Text (textDim):      #678094 (Dunkles Schiefergrau)
├─ Primärakzent (cyan):        #37DDFF (Cyberpunk Cyan)
├─ Sekundärakzent (blue):      #2D89FF (Kobaltblau)
├─ Status Erfolgreich (green):  #36E6AB (Smaragdgrün)
├─ Status Warnung (orange):     #FFAE46 (Bernstein-Orange)
├─ Status Kritisch (red):       #FF5467 (VGT Warnrot)
└─ Rahmen (border):           rgba(73, 118, 146, 0.36)

ENTERPRISE TITAN THEME (Wird aktiv, wenn TitanVisualMode.active == true)
├─ Titan Primärgold (gold):     #F5C95B (Kaiserliches Gold)
├─ Titan Softgold (goldSoft):   #DDB045
├─ Titan Tiefgold (goldDeep):   #976919
└─ Titan Rahmen (borderGold):  rgba(202, 157, 54, 0.41)
```

5.2 Abstands- & Layout-Tokens (GeDefenseUi.kt)

- `SPACING_SCREEN_HORIZONTAL_DP = 20`: Konsistenter Bildschirm-Außenabstand.
- `SPACING_SECTION_GAP_DP = 22`: Vertikaler Abstand zwischen Funktionsbereichen.
- `SPACING_CARD_GAP_DP = 14`: Vertikaler Abstand zwischen interaktiven Kacheln.
- `BOTTOM_NAV_DOCK_HEIGHT_DP = 66`: Feste Höhe der schwebenden Navigationsleiste.
- `SCROLL_RESERVED_BOTTOM_PADDING_DP = 94`: Pflicht-Padding für alle ScrollViews zur Vermeidung von Überdeckungen durch die Dock-Navigation.

5.3 Shared UI Komponenten (VgtUiComponents.kt)

- `screenHeader()`: Standardisierter Bildschirmkopf mit dynamischem Akzent-Strich, Titel und Subtitel.
- `iconWell()`: Quadratischer Vektorbett-Rahmen mit abgerundeten Ecken für System-Icons.
- `accentRule()`: Schlanke 2dp-Trennlinie zur visuellen Hierarchietrennung.
- `divider()`: Subtile Begrenzungslinie in `borderSoft`.
- `appIdentityBadge()`: Asynchron ladendes App-Icon mit Fallback auf den ersten Buchstaben des Namens.
- `statusDot()`: Pulsierender 8dp-Statuspunkt (Grün/Gelb/Rot).
- `titleWithPill()`: Abschnittsüberschrift mit rechtsbündigem Status-Badge.

5.4 Reaktive Canvas- & Animations-Komponenten

- `CyberBackgroundView.kt`: Zeichnet einen hardware-beschleunigten, subtilen polygonalen Hintergrund mit Alpha-Gradienten. Wird als Hintergrund in fast allen Activities verwendet.
- `ShieldPulseView.kt`: Rendert das reaktive Energiefeld um das GeDefense-Schild auf dem Dashboard. Ändert Frequenz und Farbe je nach Schutzstatus.
- `StartupCoreView.kt`: Spezifischer Vektor-Reaktor für den Startbildschirm (`StartupActivity`).

- **TitanCoreView.kt** : Exklusive goldene Reaktor-Animation für die Device-Owner-Konsole.
- **ScannerRadarView.kt** : Hardware-beschleunigter 360-Grad-Radarschirm für **ScannerActivity** und **PortSentinelActivity**.
- **TrafficWorldMapView.kt** : Vollständig offline berechnete Vektor-Weltkarte (Mercator) zur Projektion von Bedrohungs- und Verkehrsursprüngen ohne Kachel-Downloads.
- **VgtIconView.kt** : Proprietäre Vektor-Linien-Icon-Engine; rendert Icons ohne Rastergrafiken über Android Canvas Paths.
- **VgtProgressView.kt** : Segmentierter oder kontinuierlicher Fortschrittsbalken mit Gradient-Füllung.

5.5 Performance- & Render-Governance (**VgtUiPerformance.kt**)

- Bindet alle Animationen an den Android **Choreographer**.
- Drosselt dekorative Canvas-Passes bei Scroll-Interaktionen (**noteInteraction()**) und im Akkusparmodus.
- Trennt UI-Animationen vollständig von der Paketverarbeitung; Grafik-Lags können niemals die Netzwerk-Filterung verlangsamen.

5.6 Einflussmatrix der Styling-Dateien

```
GeDefenseUi.kt & VgtUiComponents.kt
├─ MainActivity.kt (Dashboard, ThreatScreen, ProtectionHub, AnalysisHub, SystemHub)
├─ FirewallActivity.kt
├─ WireGuardActivity.kt
├─ PrivacyActivity.kt
├─ ScannerActivity.kt
├─ XdrActivity.kt
├─ HardeningActivity.kt
├─ BehaviorActivity.kt
├─ NetworkDiscoveryActivity.kt
├─ PortSentinelActivity.kt
├─ TitanActivity.kt
├─ EvidenceScreen.kt
├─ DiagnosticsActivity.kt
├─ SetupWizardActivity.kt
└─ VpnDisclosureActivity.kt

CyberBackgroundView.kt
└─ Hinterlegt alle 16 Sub-Activities als atmosphärische Basisschicht

TitanVisualMode.kt
└─ Schaltet die Farbwelten von MainActivity, Dashboard, TitanActivity und BottomNavBar live um
```

6. API ARCHITEKTUR

GeDefense Mobile exponiert keine öffentlichen HTTP/REST-Endpunkte ins Internet. Die API-Architektur gliedert sich in:

1. **Inter-Process Communication (IPC):** SCM_RIGHTS Descriptor Passing via Unix Domain Sockets.
2. **WireGuard UAPI:** In-Process Konfigurations-Stream.
3. **Android System Services:** Privilegierte OS-Bridges via Binder.
4. **Broadcast Receivers:** Plattform-Ereignis-Hooks.

Schnittstellen-Übersicht

Schnittstelle / Endpunkt	Kommunikationspfad	Methode / Protokoll	Sender / Aufrufer	Empfänger / Handler	Datenquelle / Zweck
GaiaNet Control Socket	Lokaler Unix Domain Socket	32-Byte Token + Helper Protocol v5	NativeGaiaNet.kt	netstack/ helper_main_process.go	Initialisierung der Go Data Plane, Übergabe der TUN- & Policy-FDs
SCM_RIGHTS FD Pass (Host → Helper)	Unix Control Socket Ancillary Data	sendmsg() mit SCM_RIGHTS	NativeGaiaNet.kt	netstack/ helper_main_process.go	Übergabe von: 1. TUN, 2. Telemetry, 3. Threat Policy, 4. Privacy, 5. Gate, 6. WG Pipe
SCM_RIGHTS FD Pass (Helper → Host)	Unix Control Socket Ancillary Data	sendmsg() mit SCM_RIGHTS	netstack/ wireguard_protected_bind.go	NativeGaiaNet.kt / GeDefenseVpnService	Übergabe der WireGuard Upstream-UDP-Sockets zur Registrierung via VpnService.protect()
WireGuard UAPI Pipe	Anonymous Unix Pipe	WireGuard UAPI Key-Value Stream	NativeGaiaNet.kt	wireguard-go/device/ uapi.go	Sichere Übergabe von Private Keys, Peers und AllowedIPs ohne Disk-Persistenz
Package Gate IPC	Bounded Unix IPC Stream	Helper Protocol v5 Structs	netstack/ package_egress_gate.go	app/.../ PackageEgressGate.kt	Bounded UID- und Quarantäne-Abfragen zur Pre-Egress-Entscheidung
Telemetry Event Pipe	Anonymous Unix Pipe	Binäres Event-Framing	netstack/ telemetry.go	NativeGaiaNet.kt (Telemetry Reader)	Streaming von geblockten Paketen, Flow-Zählern und DoT/DoQ-Treffern
GeDefenseVpnService	Android Binder IPC	android.net.VpnService	Android OS / MainActivity	GeDefenseVpnService.kt	Start, Stopp und Always-on VPN Überwachung
ThreatIntelJobService	Android JobScheduler	android.app.job.JobService	Android OS JobScheduler	ThreatIntelJobService.kt	Periodischer Download externer Bedrohungslisten im Hintergrund
IntegrityJobService	Android JobScheduler	android.app.job.JobService	Android OS JobScheduler	IntegrityJobService.kt	6-Stunden Integritätsprüfung der Applikation
ResilienceJobService	Android JobScheduler	android.app.job.JobService	Android OS JobScheduler	ResilienceJobService.kt	2-Stunden Selbstreparatur- und Drift-Reconciliation
PackageChangeReceiver	Android Broadcast Intent	ACTION_PACKAGE_ADDED / REMOVED	Android OS PackageManager	PackageChangeReceiver.kt	Triggert sofortigen InstallGuard-Scan und Drift-Audit
BootReceiver	Android Broadcast Intent	ACTION_BOOT_COMPLETE	Android OS Init	BootReceiver.kt	Stellt geplante Hintergrund-Jobs nach Geräteeustart wieder her
TitanDeviceAdminReceiver	Android Admin Intent	ACTION_DEVICE_ADMIN_ENABLED	Android OS DevicePolicyManager	TitanDeviceAdminReceiver.kt	Empfängt Enterprise- und Device-Owner-Berechtigungsstatus

7. DATENFLÜSSE

Flow 1: Paketverarbeitung, Bedrohungsprüfung & Egress

```

Applikation sendet TCP/UDP-Paket
↓
Linux Kernel leitet Paket in /dev/tun Interface
↓
GaiaNet V2 (engine.go) liest IP-Paket im Userspace
↓
[GDTI-v2 Trie Match (policy.go)]: Ist Ziel-IP in BLOCK-Liste?
├─ JA → Paket wird gedroppt; sendet TCP RST / ICMP Unreachable
│   └─ Sendet Event über Telemetrie-Pipe an Host
│       └─ Host trägt Vorfall in XdrEventStore und EvidenceLedger ein
└─ NEIN → [Privacy Policy Filter (privacy_policy.go)]:
    └─ Handelt es sich um Port 853 DoT/DoQ im Strict Profile?
        └─ JA → Paket wird gedroppt (Non-MITM Schutz)
        └─ NEIN → [Package Egress Gate (package_egress_gate.go)]:
            └─ Befindet sich Quell-App in Quarantäne?
                └─ JA → Paket wird gedroppt
                └─ NEIN → Weiterleitung zum Egress:
                    └─ DIRECT: Über Kernel-Socket ins Internet
                    └─ WIREGUARD: Verschlüsselt via wireguard-go

```

Flow 2: Bedrohungsfeed-Aktualisierung & Transaktionaler Tausch

```

ThreatIntelJobService (oder manueller Trigger im ThreatScreen)
↓
FeedDownloader lädt Listen via HTTPS (Feodo, Spamhaus, Tor, ...)
↓
ThreatIntelParser verifiziert SHA-256 Prüfsummen & parst Präfixe
↓
Kompiliert neuen binären GDTI-v2 Radix-Trie im Arbeitsspeicher
↓
ThreatCacheStore schreibt Trie atomar via Temp-File + Rename (HMAC signiert)
↓
NativeGaiaNet sendet neuen Policy-FD via SCM_RIGHTS an GaiaNet Helper
↓
policy.go validiert SHA-256 Fingerprint und schaltet neuen Trie atomar live
↓
Alter Trie wird freigegeben (Zero Downtime)

```

Flow 3: App-Installation, Quarantäne & 2-Stufen Scan

```

Nutzer installiert neue App auf dem Smartphone
↓
Android OS sendet ACTION_PACKAGE_ADDED Broadcast
↓
PackageChangeReceiver fängt Intent ab und weckt InstallGuard
↓
InstallGuard setzt App sofort auf die FlowGate-Quarantäneliste
↓
PackageEgressGate blockiert alle ausgehenden Netzwerkpakete der App
↓
[Stufe 1: Fast Verdict (Timeout 1.750 ms auf 4 Workern)]:
├─ Prüft Signer-Kontinuität gegen PackageBaselineStore
├─ Geringes Risiko & bekannte Signatur → Vorläufige Freigabe
└─ Unbekannte Signatur / Erstinstallation → Quarantäne bleibt aktiv
↓
[Stufe 2: Deep Scan (Timeout 25 s auf 2 Workern)]:
├─ AppRiskScanner extrahiert AndroidManifest, Berechtigungen & native Libs
├─ Befund sauber → Quarantäne wird aufgehoben; App darf funken
└─ Schadcode-Muster erkannt → Dauerhafte Quarantäne; XDR-Alarm ausgelöst

```

Flow 4: Forensische XDR-Korrelation & Detached Signatur

```

Netzwerk-Block, Signer-Drift oder Portscan tritt auf
↓
XdrEngine empfängt Vorfallsfragmente und aggregiert sie pro App-ID
↓
Berechnet Risikoscore nach deterministischer Matrix

```

```
↓
XdrEventStore speichert Vorfall in Vault Domäne 1 (AES-256-GCM)
↓
Spiegelt Eintrag in EvidenceLedger (verkettet HMAC mit vorherigem Record)
↓
Bei Export: HybridArtifactSigner erzeugt SHA-256 über das Journal
↓
Signiert mit Android KeyStore ECDSA P-256 (und ML-DSA-87 auf Android 17+)
↓
Gibt detached .sig.json Datei an den Nutzer aus
```

Flow 5: Update-Stabiler Vault Key Lifecycle (VC55)

```
App startet nach OS- oder APK-Update
↓
PersistentVaultKeys initialisiert Kustodie
↓
Lädt stabilen Hardware-HMAC Root Key aus Android KeyStore (root-prf.v1)
(Wichtig: Key-Alias enthält keine APK-Hashes oder Versionsnummern)
↓
Führt genau eine Hardware-PRF Operation aus -> erzeugt flüchtigen Wrapping-KEK
↓
Für jede der 12 Vault-Domains:
  1. Liest <domain>.v5.keyset aus noBackupFilesDir
  2. Entschlüsselt 256-Bit Domain-Seed mittels KEK via AES-256-GCM
  3. Expandiert via HKDF-SHA-256 getrennte flüchtige AES- und HMAC-Subkeys
↓
Subkeys stehen für High-Speed I/O bereit; Keystore wird im laufenden Betrieb nicht mehr blockiert
↓
Alle temporären Schlüsselpuffer im RAM werden genullt (fill(0))
```

8. SHARED / CORE DATEIEN

Die folgenden Komponenten sind hochgradig vernetzte Systemkerne, die von multiplen Modulen als fundamentale Infrastruktur vorausgesetzt werden. Sie sind ausdrücklich als **CORE / SHARED DEPENDENCY** klassifiziert.

1. app/.../AppRuntime.kt

- **Klassifikation:** **CORE / SHARED DEPENDENCY** (Zentraler Dependency Container & State Locator)
- **Verantwortung:** Initialisiert alle 16 Subsysteme asynchron; hält atomare State-Snapshots im Arbeitsspeicher; stellt State-Listener für UI-Refreshes bereit.
- **Abhängige Module:**
 - Host & Application Runtime
 - Network & VPN Core
 - Threat Intelligence & Policy Engine
 - Telemetry Shield & Privacy Engine
 - InstallGuard & Scanner Plane
 - XDR / EDR Correlation & Incident Plane
 - Device Hardening & Baseline Engine
 - TITAN Device Owner & Enterprise Härtung
 - Sämtliche 21 Activities und Screens

2. app/.../BoundedExecutors.kt

- **Klassifikation:** **CORE / SHARED DEPENDENCY** (Zentrales Threading- & Concurrency-Modell)
- **Verantwortung:** Stellt den serialisierten **gedefense-control** Single-Thread-Executor sowie überlastungssichere Scanner- und Recovery-Worker-Pools mit Kapazität 0 (**SynchronousQueue**) bereit.
- **Abhängige Module:**
 - Host & Application Runtime
 - Network & VPN Core
 - InstallGuard & Scanner Plane
 - Resilience & Self-Healing Supervisor
 - TitanPolicyManager

3. app/.../GeDefenseUi.kt & VgtUiComponents.kt

- **Klassifikation:** **CORE / SHARED DEPENDENCY** (Design System, Tokens & UI-Komponenten)
- **Verantwortung:** Definiert alle visuellen Tokens (Farben, Abstände, Badges, Typografie) und generiert native View-Hierarchien ohne XML.
- **Abhängige Module:**
 - Sämtliche 21 Aktivitäten, Screens und Hubs des Frontends.

4. app/.../CyberBackgroundView.kt

- **Klassifikation:** **CORE / SHARED DEPENDENCY** (Globaler Canvas-Hintergrund)
- **Verantwortung:** Rendert den GPU-beschleunigten, stromsparenden polygonalen VGT Cyber-Hintergrund in allen Benutzeroberflächen.
- **Abhängige Module:**
 - Alle 16 eigenständigen Android Activities.

5. app/.../SecureTelemetryVault.kt & PersistentVaultKeys.kt

- **Klassifikation:** CORE / SHARED DEPENDENCY (Kryptografisches Speicherfundament)
- **Verantwortung:** Verwaltet die 12 isolierten AEAD-Verschlüsselungsdomänen; deriviert KEKs und DEKs über den hardware-gebundenen KeyStore-Root; schützt ruhende Daten.
- **Abhängige Module:**
 - FirewallPolicyStore
 - WireGuardProfileStore
 - XdrEventStore
 - BehaviorBaselineStore
 - PackageBaselineStore
 - MalwareAnalysisStore
 - NetworkDiscoveryStore
 - PortSentinelStore
 - TitanPolicyStore
 - ScannerStateCache
 - EvidenceLedger

6. core/.../AeadVaultEnvelope.kt

- **Klassifikation:** CORE / SHARED DEPENDENCY (VGTVLT01 Binär-Codec)
- **Verantwortung:** Reines JVM-Primitiv zur Ver- und Entschlüsselung von Datensätzen via AES-256-GCM mit 96-Bit Nonce und AAD-Bindung.
- **Abhängige Module:**
 - SecureTelemetryVault
 - AuthenticatedSnapshotStore
 - EvidenceLedger

7. core/.../EvidenceLedger.kt

- **Klassifikation:** CORE / SHARED DEPENDENCY (Revisionssicheres Sicherheits-Journal)
- **Verantwortung:** Führt das HMAC-verkettete, manipulationssichere Audit-Protokoll aller Sicherheits- und Blockierungsereignisse.
- **Abhängige Module:**
 - XdrEngine
 - GeDefenseVpnService
 - InstallGuard
 - ResilienceSupervisor
 - HybridArtifactSigner

8. core/.../ThreatIndex.kt & IpPrefix.kt

- **Klassifikation:** CORE / SHARED DEPENDENCY (Bedrohungs-Indizierung & CIDR-Mathematik)
- **Verantwortung:** Bildet IP-Präfixe in kompakte binäre Radix-Tries ab; filtert RFC1918/Private IP-Bereiche; stellt O(log n) Evaluation sicher.
- **Abhängige Module:**
 - ThreatIntelRepository
 - ThreatCacheStore
 - GeDefenseVpnService

- `netstack/policy.go`

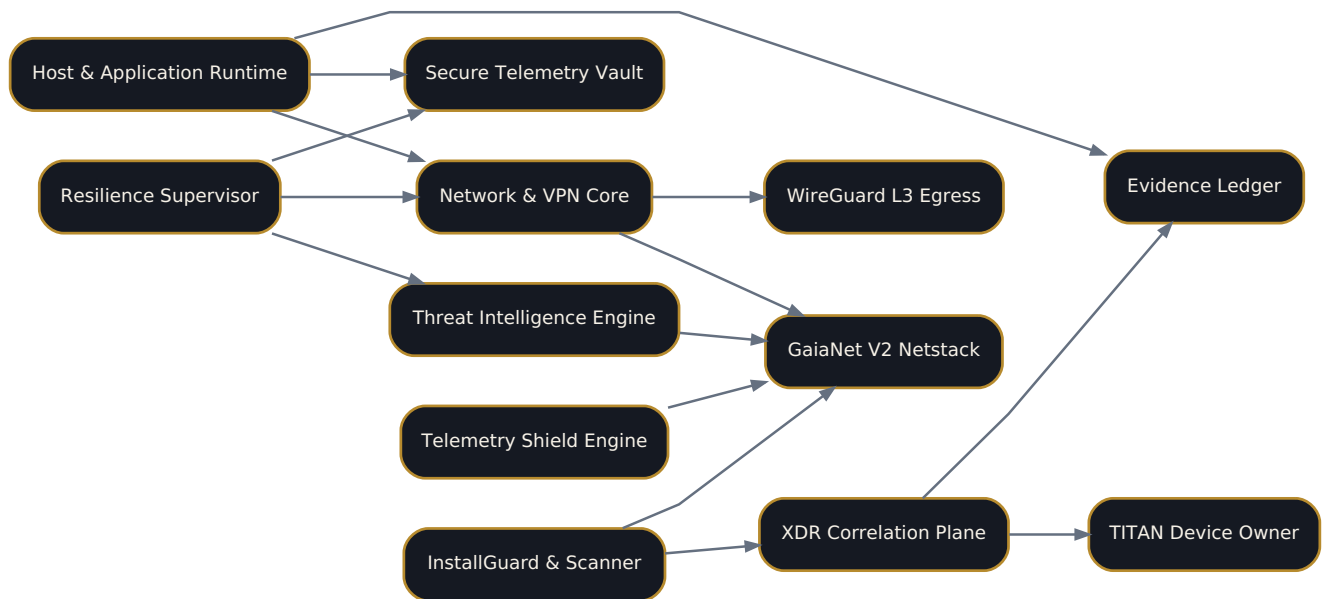
9. `app/.../NativeGaiaNet.kt`

- **Klassifikation:** `CORE` / `SHARED DEPENDENCY` (Prozess- & IPC-Brücke zur Data Plane)
 - **Verantwortung:** Startet den Go-Netstack-Helper; verwaltet Unix-Domain-Sockets; wickelt Filedeskriptor-Transfers ab; dient als TUN-Liveness-Ancor.
 - **Abhängige Module:**
 - `GeDefenseVpnService`
 - `PackageEgressGate`
 - `WireGuardTransport`
-

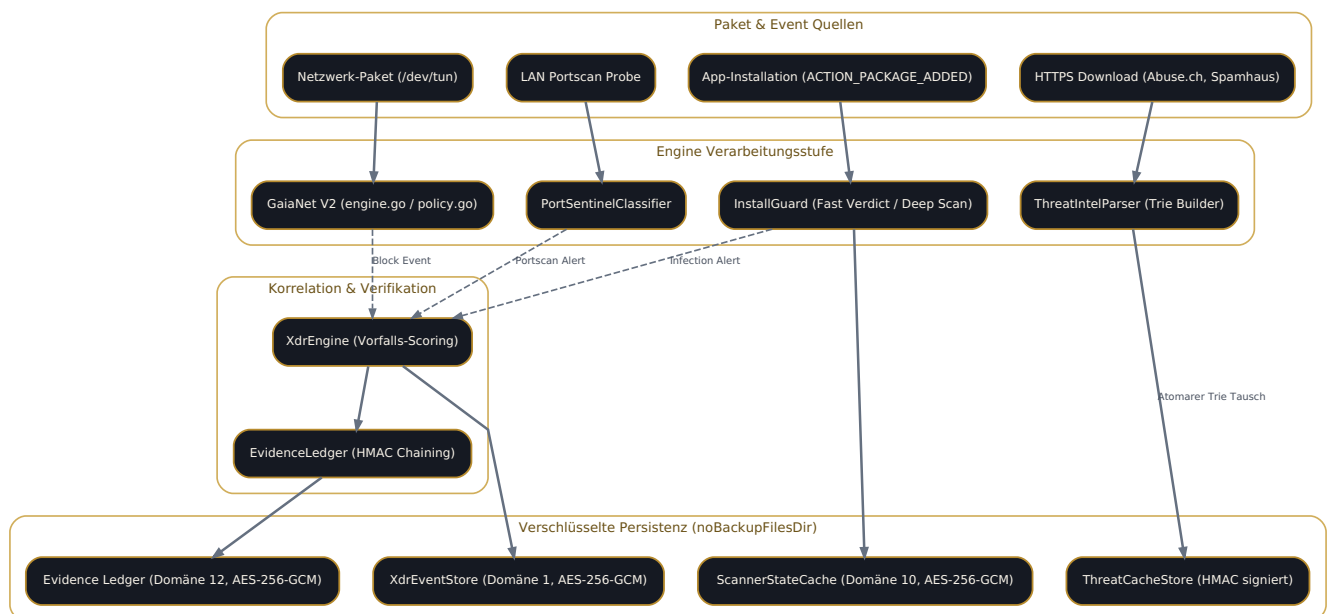
9.1 Gesamtsystem-Architektur



9.3 Modul-Abhängigkeitsgraph



9.4 Backend / API / Datenspeicher Datenfluss



10. DATEIREFERENZEN

Die folgende Referenztabelle verknüpft zentrale Architekturentscheidungen mit ihren exakten Implementierungsstellen:

- **Asynchroner Bootstrap ohne Main-Thread I/O:**

```
app/src/main/java/de/visiongaia/gedefense/mobile/AppRuntime.kt (ensureReadyAsync()),
app/src/main/java/de/visiongaia/gedefense/mobile/StartupActivity.kt (onCreate()).
```

- **TUN Allokation & Liveness Anchor:**

```
app/src/main/java/de/visiongaia/gedefense/mobile/GeDefenseVpnService.kt (establishVpn()),
app/src/main/java/de/visiongaia/gedefense/mobile/NativeGaiaNet.kt (Session.tunAnchor).
```

- **SCM_RIGHTS Descriptor Transfer & Protokoll v5:**

```
app/src/main/java/de/visiongaia/gedefense/mobile/NativeGaiaNet.kt (sendStartupDescriptors()),
netstack/helper_main_process.go (parseReceivedFDs()).
```

- **WireGuard Per-Socket Protection Handshake:**

```
netstack/wireguard_protected_bind.go (Bind.Open()),
app/src/main/java/de/visiongaia/gedefense/mobile/NativeGaiaNet.kt (handleProtectedSocketFd()),
app/src/main/java/de/visiongaia/gedefense/mobile/GeDefenseVpnService.kt (protectSocket()).
```

- **Update-Stabile Hardware-PRF Kustodie (VC55):**

```
app/src/main/java/de/visiongaia/gedefense/mobile/PersistentVaultKeys.kt (keys(), rootWrapKey()),
app/src/main/java/de/visiongaia/gedefense/mobile/AndroidSecrets.kt (hmacSha256()).
```

- **Pure-JVM AEAD Envelope & AAD Binding:**

```
core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/AeadVaultEnvelope.kt (seal(), open()).
```

- **InstallGuard Fast-Verdict & FlowGate Pre-Egress:**

```
app/src/main/java/de/visiongaia/gedefense/mobile/InstallGuard.kt (evaluatePackage()),
netstack/package_egress_gate.go (EvaluateFlow()).
```

- **DoT/DoQ Port 853 Non-MITM Filter:**

```
netstack/privacy_policy.go (EvaluatePort853()),
core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/PrivacyIntelligence.kt (isBlockedDoT()).
```

- **Post-Quantum Hybrid Detached Signatures:**

```
app/src/main/java/de/visiongaia/gedefense/mobile/HybridArtifactSigner.kt
(writeDetachedSignature(), signMlDsa87()).
```

- **Single-Flight Selbstreparatur:**

```
app/src/main/java/de/visiongaia/gedefense/mobile/ResilienceSupervisor.kt
(reconcileSingleFlight()),
core/src/main/kotlin/de/visiongaia/gedefense/mobile/core/RecoveryAttemptBudget.kt
(consumeBudget()).
```

11. ARCHITEKTUR-AUFFÄLLIGKEITEN

Im Rahmen des tiefgehenden statischen Architektur-Audits wurden folgende Auffälligkeiten, verwaiste Dateien und historische Artefakte im Quelltext identifiziert:

1. Verwaiste Dashboard-Screens ohne Aufrufer

- **app/src/main/java/de/visiongaia/gedefense/mobile/MoreScreen.kt (250 LOC):**
Befund: Vollständige UI-Klasse mit 250 Zeilen. War in frühen Beta-Versionen (0.15.x) der 5. Navigations-Tab ("Control Center"). Wurde in 0.27.x vollständig durch `SystemHubScreen.kt` ersetzt. Besitzt im gesamten Projekt **exakt 0 Aufrufer** und wird im Release-Build nicht angezeigt.
Empfehlung: In einem kommenden Refactoring-Sprint sicher archivieren oder entfernen.
- **app/src/main/java/de/visiongaia/gedefense/mobile/SecurityScreen.kt (701 LOC):**
Befund: Große Dashboard-Klasse mit 701 Zeilen und Weltkarten-Integration. War der historische Vorgänger von `ThreatScreen.kt`. Besitzt im gesamten Quelltext **exakt 0 Aufrufer**.
Empfehlung: Da alle Funktionen in `ThreatScreen` und `ProtectionHubScreen` überführt wurden, sollte diese Datei zur Vermeidung von Verwirrung entfernt werden.

2. Unbereinigte Konfigurations- und Backup-Dateien

- **settings.gradle.kts.orig (347 Bytes):**
Befund: Ein Git- oder Merge-Backup der Gradle-Settings-Datei im Projekt-Root. Wird vom Build-System ignoriert.
Empfehlung: Löschen.

3. Build-Tag Stubs für Non-Android Targets

- **netstack/helper_main_stub.go (6 Zeilen):**
Befund: Enthält `//go:build !android && !gdandroidhelper` und liefert einen leeren Stub `runHelperMain()` `{ return 0 }`.
Architektur-Grund: Notwendig, damit Go-Tools (`go vet`, Standard-Compiler) auf Windows- und Linux-Entwickler-Hosts außerhalb der Android-Cross-Compilation nicht mit fehlenden Symbolen abbrechen. Korrektes Design-Pattern.

4. Parallele Kryptografie-Generationen zur Migration

- **HardwareDerivedVaultKeys.kt (122 LOC) vs. PersistentVaultKeys.kt (292 LOC):**
Befund: Zwei Koexistierende Krypto-Derivatoren.
Architektur-Grund: `HardwareDerivedVaultKeys` (v4 Generation) wird bewusst als Read-Only-Migrationspfad vorgehalten, damit Bestandsnutzer beim Update auf VC55 ihre verschlüsselten Daten nicht verlieren.
`PersistentVaultKeys` ist die aktive Schreib-Generation. Saubere Abwärtskompatibilität.

5. Nicht-aktive CGO-Netzwerk-Binder

- **netstack/socket_binder_android.go vs. CGO-Quellen:**
Befund: Standardmäßig wird für `arm64-v8a` mit `CGO_ENABLED=0` gebaut. Einige CGO-Socket-Binder-Funktionen sind nur für experimentelle Sonderpfade hinter Build-Tags hinterlegt.
Architektur-Grund: Stellt sicher, dass das finale APK keine ungeprüften dynamischen JNI-Abhängigkeiten aufweist.